Gymnasium Oberwil

Maturaarbeit

Maschinelles Lernen mit TensorFlow

Entwicklung eines Convolutional Denoising Autoencoders

Luis Wirth, 4a

Betreut von
Dr. Jonas Gloor

Abgegeben im September 2019

Inhaltsverzeichnis

Inhaltsverzeichnis **I**

Vorwort **IV**

Einleitung **IV**

Konvention und Notation **V**

1 Machinelles Lernen **1**
1.1 Allgemeine Begriffe . . . 1
1.1.1 Daten . . . 1
1.1.2 Modelle . . . 2
1.2 Training . . . 3
1.2.1 Verlust- und Kostenfunktionen . . . 3
1.2.2 Gradientenverfahren . . . 4
1.2.3 Stochastisches Gradientenverfahren fuer Machine Learning . . . 6
1.2.4 Adam Optimizer? . . . 7
1.3 Trainingsphaenomene . . . 7
1.3.1 Konvergenz und Divergenz . . . 7
1.3.2 Underfitting und Overfitting . . . 7

2 Deep Learning und Künstliche Neuronale Netze **8**
2.1 Perzeptron . . . 8
2.1.1 Was kann ein Perzeptron erlernen? . . . 9
2.2 Erweiterung der Kuenstlichen Neuronen . . . 9
2.2.1 Kuenstliche Neuronen im Allgemeinen . . . 10
2.2.2 Perzeptronen als Kuenstliche Neuronen . . . 10
2.2.3 ReLU Neuronen . . . 10
2.2.4 Sigmoid Neuronen . . . 11
2.3 Topologie der Kuenstlichen Neuronalen Netzen . . . 11
2.4 Lernverhalten . . . 13
2.5 Vorwaertspropagierung . . . 13
2.5.1 Modellparameter initialisieren . . . 16
2.6 Rueckwaertspropagierung . . . 17
2.7 Universal Approximation Theorem? . . . 21

3 Convolutional Neural Networks **22**
3.1 Bilder als Tensoren . . . 22
3.2 Topologie von CNNs . . . 23
3.3 Convolutional-Schichten und Filter . . . 24
3.3.1 Filter in der Bildverarbeitung . . . 24
3.3.2 Filter in CNNs . . . 25
3.3.3 Filteroperation intutitiv . . . 25
3.3.4 Filteroperationen als diskrete Faltungen . . . 26

3.3.5 Mehrere Filter . . . 28
3.3.6 Padding . . . 29
3.3.7 Stride . . . 29
3.3.8 Vorzuege von Filtern . . . 30
3.3.9 Die Convolutional-Schicht . . . 30
3.4 Dimensionalitaetskontrolle . . . 31
3.4.1 Die Pooling-Schicht . . . 31
3.4.2 Die Upsampling-Schicht . . . 32

4 Autoencoder **33**
4.1 Topologie . . . 33
4.2 Funktionsweise . . . 35
4.3 Anwendungen . . . 36
4.3.1 Kompression . . . 36
4.3.2 Neue Datenrepraesentation . . . 36
4.3.3 Visualisierung von Daten . . . 36
4.4 Convolutional-Autoencoder . . . 36
4.5 Denoising-Autoencoder . . . 36
4.5.1 Generierung der verrauschten Bilder . . . 37

5 Implementation fuer Maschinelles Lernen **38**
5.1 TensorFlow . . . 38
5.1.1 Client und Master . . . 39
5.1.2 Verwendung - der Client . . . 39
5.1.3 Implementation - der Master . . . 44
5.1.4 Tensorboard . . . 46
5.2 Keras . . . 47
5.2.1 Grundsaetliche Funktionsweise . . . 47
5.2.2 Schichten . . . 47
5.2.3 Train and evaluate . . . 48
5.2.4 Weiss nit . . . 49

6 Entwicklung eines Denoising-Autoencoders **51**
6.1 Das konkrete Modell . . . 51
6.1.1 Daten . . . 51
6.2 Setup . . . 51
6.3 Entwicklung . . . 53
6.3.1 Testprogramm . . . 53
6.3.2 Trainingsdaten . . . 53
6.3.3 Modell definieren . . . 56
6.3.4 Training . . . 58
6.3.5 Modell ausfuehren . . . 58
6.3.6 Hyperparameter einstellen . . . 59
6.3.7 TensorBoard Konfiguration . . . 60

7 Diskussion **61**
7.1 Auswertung mit TensorBoard . . . 61

A Programmcode **62**

B Ausfuehrungen zu TensorFlow **64**

C Intuitiver Beweis fuer UAT? **65**

Literatur **66**

Abbildungsverzeichniss **68**

Tabellenverzeichniss **69**

Selbstständigkeitserklärung **70**

Vorwort

Sollte insgesamt eine Seite umfassen.
Auf die Leitfragenaenderung eingehen. (Weg von der alten zur neunen Frage)

Einleitung

ALLGEMEINES ZUM THEMENGEBIET MACHINELLES LERNEN
INFORMATIONEN ZU DIESER ARBEIT: INHALT AUFBAU ETC Das Ziel dieser Arbeit ist es, dem Leser ein umfangreiches Grundverstaentniss ueber Maschinelles Lernen zu bieten. Dieses umfasst die theoretischen Grundlagen zu modellbasiertem Lernen, Kuenstlichen Neuronalen Netzen, Convolutional Neural Networks und Autoencodern. Mithilfe dieser erarbeiten Theorie wird dann noch die die praktische Umsetzung eines Modells mithilfe von TensorFlow und Keras erklaert. Bei TensorFlow und Keras handelt es sich um Frameworks fuer Maschinelles Lernen mit Python. Sie sind in der Industrie sehr verbreitet und wurden deshalb fuer diese Arbeit gewaehlt.

Somit wurde die Leitfrage folgendermassen formuliert: **Wie funktioniert Maschinelles Lernen und wie entwickelt man mit diesem Wissen ein konkretes Modell in TensorFlow?**

Wir werden folgenden groben Aufbau verfolgen:

1. Wir erarbeiten die allgemeine Theorie zum Maschinellen Lernen.
2. Wir betrachten Kuenstliche Neuronale Netze als Modelle fuer ML.
3. Wir betrachten Convolutional Neural Networks und Autoencoder als konkrete KNN Architekturen.
4. Wir betrachten die ML-Frameworks TensorFlow und Keras bezueglich ihrer Funktionsweise.
5. Wir programmieren mithilfe der erarbeiten Theorie ein konkretes Modell in TensorFlow / Keras.
6. Wir untersuchen das programmiere Modell.

Fuer das konkrete Modell werden wir einen sogennante Convolutional-Denoising-Autoencoder entwickeln. In den Theorie Teilen werden wir lernen wie dieser Funktioniert. Der Grund fuer die Wahl dieses kokreten Modells, besteht darin, dass es sich dabei um ein Convolutional Neural Network handelt, welche in den letzten Jahren sehr populaer geworden sind und deshalb interessant zu betrachten sind. Wir werden dieses Konzept eines Convolutional Neural Network mit einem Autoencoder koppeln, um einen Convolutional-Denoising-Autoencoder zu erhalten. Mithilfe von ihm koennen wir Bildrauschen von Bildern wegrechnen lassen.

Konvention und Notation

Beschriftung

Zahlen und Tensoren

a	ein Skalar (Zahl)
$\boldsymbol{a}$	ein Vektor
$\boldsymbol{a} \in \mathbb{R}^n$	ein Vektor mit n Komponenten
$\boldsymbol{A}$	eine Matrix
$\boldsymbol{A} \in \mathbb{R}^{m \times n}$	eine Matrix mit m Zeilen und n Spalten
$\mathbf{A}$	ein Tensor
$\mathfrak{A}, \mathfrak{B}, \mathfrak{C}, \mathfrak{D}, \mathfrak{E}, \cdots$	a
$\mathfrak{a}, \mathfrak{b}, \mathfrak{c}, \mathfrak{d}, \mathfrak{e}, \mathfrak{f}, \mathfrak{g}, \cdots$	a

Mengen

(a, b)	ein geordnetes Paar (2-Tupel) der Elemente a und b
$\mathbb{A}$	eine Menge
$\mathbb{R}$	die Menge aller reellen Zahlen
$\mathbb{Z}$	die Menge aller ganzen Zahlen
$\mathbb{N}$	die Menge aller natuerlichen Zahlen
$\{a, b\}$	die Menge, welche aus den Elementen a und b besteht
$\{1, \ldots, n\}$	die Menge aller ganzen Zahlen von 1 bis n
$a \in \mathbb{A}$	a ist ein Element der Menge $\mathbb{A}$

Indexierung

a_i	die i-te Vektorkomponente mit Startindex 1
$(\boldsymbol{A})_{i,j}$	das Matrixelement in der i-ten Zeile und der j-ten Spalte
$A_{i,j}$	das Matrixelement in der i-ten Zeile und der j-ten Spalte
$\boldsymbol{A}_{i,:}$	die Zeile i der Matrix $\boldsymbol{A}$
$\boldsymbol{A}_{:,i}$	die Spalte i der Matrix $\boldsymbol{A}$
$\mathbf{A}_{i,:,:}$	der i-te Querschnitt entlang der Hoehe des Tensors $\mathbf{A}$
$\mathbf{A}_{:,i,:}$	der i-te Querschnitt entlang der Breite des Tensors $\mathbf{A}$
$\mathbf{A}_{:,:,i}$	der i-te Querschnitt entlang der Tiefe des Tensors $\mathbf{A}$
$\boldsymbol{A} \Rightarrow ($	Definition einer Matrix $\boldsymbol{A}$
$\mathbf{A} = \begin{bmatrix} \boldsymbol{A}_{:,:,1} & \cdots & \boldsymbol{A}_{:,:,c} \end{bmatrix}$	Definiton eines Tensors $\mathbf{A}$

Lineare Algebra Operationen

$\boldsymbol{v} \cdot \boldsymbol{w}$	das Skalarprodukt von $\boldsymbol{v}$ mit $\boldsymbol{w}$
$\boldsymbol{A}^\mathsf{T}$	das Transponierte einer Matrix $\boldsymbol{A}$
$\boldsymbol{A} \odot \boldsymbol{B}$	das elementeweise (Hadamard) Produkt
$\boldsymbol{A} * \boldsymbol{B}$	die diskrete Faltung von $\boldsymbol{A}$ ueber $\boldsymbol{B}$

Infinitesimalrechnung

$f'(x)$	die Ableitung der Funktion f bezueglich seinem Argument x
$\frac{\mathrm{d}y}{\mathrm{d}x}$	die Ableitung von y bezueglich x
$\frac{\partial y}{\partial x}$	die partielle Ableitung von y bezueglich x
$\boldsymbol{\nabla} y$	der Gradient von y
$\boldsymbol{\nabla}_{\boldsymbol{x}} y$	der Gradient von y bezueglich $\boldsymbol{x}$ (Vektor)
$\int f(x)\,\mathrm{d}x$	das unbestimmte Integral der Funktion f bezueglich x
$\int_a^b f(x)\,\mathrm{d}x$	das bestimmte Integral der Funktion f bezueglich x von a bis b

Funktionen

$f : \mathbb{A} \to \mathbb{B}$	eine Funktion f mit Definitionsmenge $\mathbb{A}$ und Zielmenge $\mathbb{B}$
$f(x)$	eine Funktion f mit Argument x (Skalar)
$f(\boldsymbol{v})$	eine Funktion f mit Argument $\boldsymbol{v}$ (Vektor)
$\boldsymbol{f}[\boldsymbol{v}]$	die vektorisierte Funktion $\boldsymbol{f}$ mit Argument $\boldsymbol{v}$ (Vektor)
$\mathcal{A}$	ein Funktionenraum
$f * g$	die Faltung von f ueber g

Statistik

$\mathcal{N}$	die Gauss'sche Normalverteilung
ϕ	die Gauss'sche Dichtefunktion
μ	der Erwartungswert/Mittelwert
σ^2	die Varianz

Eraehnung von Notation $\mathbb{R}$ fuer Skalare und $\mathbb{R}^n$ fuer Vektoren/Vektorraueme...

Kapitel 1

Machinelles Lernen

Der Themenbereich des **Machinellen Lernens** beschäftigt sich mit Algorithmen und mathematischen Modellen, welche von selber lernen, Probleme zu loesen. Hierbei wird nicht explizit einprogrammiert, wie das Modell das Problem zu loesen hat, stattdessen wird das Modell trainiert, optimiert sich von selbst und findet selber einen Weg, das Problem zu loesen. Die Grundidee dabei ist, dass man Daten erfasst, generiert oder misst, welche analysiert werden sollen. Innerhalb dieser Daten existieren gewisse Gesetzmaessigkeiten und Muster. Diese Muster sollen vom Modell erkannt werden und verallgemeinert werden. Nach dem erfolgreichen Lernen, kann das Modell Vorhersagen fuer neuen Daten machen.

Man unterscheidet zwischen zwei Arten von Maschinellem Lernen.

- **Ueberwachtes Lernen** (engl.: supervised learning) ist ein Lernverfahren, bei welchem die Daten aus zwei Teilen bestehen, aus Inputs und Outputs. Man bezeichnet dabei die Outputs als Labels. Die Aufgabe des Modells ist es eine **Korrelation** zwischen den Inputs und den Labels zu erlernen und so ihre Beziehung zueinander zu verstehen. Anhand der Informationen, welche in den Inputs enthalten sind, sind dann die Labels vorherzusagen. Man gleicht dann die vorhergesagten Labels des Modells mit den wahren Labels ab (man "ueberwacht" sie) und bewertet so die Faehigkeiten des Modells..

  Voraussetztung dafuer ist, dass die Daten eben "gelabelt" sein muessen. Man muss schon im vorhinein Daten haben, bei welchen man die gewuenschten Labels hat. Natuerlich muss auch die erwaehnte Korrelation bestehen. Falls kein Zusammenhang zwischen den Inputs und den Outputs besteht, kann das Modell auch keine Vorhersagen machen und so auch nichts erlernen.

- **Unueberwachtes Lernen** (engl.: unsupervised learning) ist ein anderes Lernverfahren, bei welchem eben diese Labels nicht vorhanden sind. Dem Modell stehen nur die Inputdaten zur Verfuegung. Diese werden ebenfalls analysiert und das Modell soll Muster herausextrahieren, welche sich von sonstigem zuefaelligen Rauschen unterscheiden.

Der Grossteil der Modelle des Maschinellen Lernens benutzen ueberwachtes Lernen, da es auch deutlich mehr Anwendungsmoeglichkeiten besitzt. Es gibt nur wenige Arten von unueberwachtem Lernen, dazu gehoeren: Hauptkomponentenanalysen und Generative Adversial Networks. Aus diesem Grund, werden wir im folgenden uns grundsaetzlich mit ueberwachtem Lernen beschaeftigen.

(Mehrere, *Supervised learning — Wikipedia, The Free Encyclopedia*)
(Mehrere, *Unsupervised learning — Wikipedia, The Free Encyclopedia*)

1.1 Allgemeine Begriffe

1.1.1 Daten

Um ein Modell zu trainieren, braucht man Daten. Diese Daten bestehen immer aus **Inputs** und **Outputs**. Man unterscheidet hierbei zwischen zwei Arten von Outputs. Die **Labels** sind die erwarteten

Outputs, welche die gewueschten Zielwerte sind. Die **Vorhersagen** sind die Outputs, welche das Modell produziert und hoffentlich moeglichst genau mit den Labels uebereinstimmen.

Diese Daten fuer das Training kommen in der Form eines **Trainingsdatensatzes** $\mathbb{X}$. Dabei handelt es sich um eine Menge von **Samples**, welche jeweils aus Inputs und Labels bestehen. Die Inputs werden in einem Vektor

$$\boldsymbol{x} = \begin{pmatrix} x_1 & x_2 & \cdots & x_m \end{pmatrix}^\mathsf{T}$$

und die Labels in einem Vektor

$$\hat{\boldsymbol{y}} = \begin{pmatrix} \hat{y}_1 & \hat{y}_2 & \cdots & \hat{y}_n \end{pmatrix}^\mathsf{T}$$

zusammengefasst.Somit ist ein Trainingssample nichts anderes als ein Paar $(\boldsymbol{x}_i, \hat{\boldsymbol{y}}_i)$ eines Inputsvektor $\boldsymbol{x}$ und eines Labelvektor $\hat{\boldsymbol{y}}$. Die Vorhersagen werden ebenfalls in einem Vektor

$$\boldsymbol{y} = \begin{pmatrix} y_1 & y_2 & \cdots & y_n \end{pmatrix}^\mathsf{T}$$

zusammengefasst.

Die Inputs beinhalten sogennanten **Features** (deutsch: Merkmale). Sie zeichnen die Inputs aus und umfassen ihren ganzen Informationsgehalt. Der Algorithmus soll anhand dieser Features seine Vorhersagen machen. Diese Vorhersagen werden dann mit den Labels abgeglichen und so bewertet. Anhand der Bewertungen wird dann eine Optimierung des Modells vorgenommen. Unter korrekten Bedingungen (kein Overfitting) findet kein Auswendiglernen der Trainingsdaten statt, sondern ein Generalisieren des Zusammenhangs anhand von Mustern und Gesetzmassigkeiten.

Um eine endgueltige Bewertung des Modells durchzufuehren, wird ein Testdatensatz $\mathbb{T}$, welcher nicht Teil des Trainingsdatensatz ist, genutzt, um Vorhersagen zu machen. Dies garantiert, dass kein Auswendiglernen moeglich ist.

BEISPIEL

1.1.2 Modelle

Ein **Modell** ist eine mathematische Funktion $h\colon \mathbb{R}^m \to \mathbb{R}^n$, Hypothesenfunktion gennant, welche die Inputs auf die Outputs abbildet $\boldsymbol{y} = h(\boldsymbol{x})$. Man kann diese Funktion gewissermassen als die Hypothese, welche das Modell bezueglich der Beziehung zwischen den Inputs und den Labels aufgestellt hat, betrachten. Diese Modelle koennen verwendet werden um vorallem entweder Klassifizierungsprobleme oder Regressionsprobleme zu loesen. Falls es sich um letzteres handelt, spricht man von einem Regressionsmodell. Ein solches Regressionsproblem werden wir in dieser Arbeit loesen.

Das Verhalten eines Modells wird bestimmt durch seine **Modellparameter** $\lambda_1, \lambda_2, \ldots, \lambda_k$. Sie bestimmen, wie die Hypothese des Modells lautet. Das Ziel ist es, die Modellparameter so einzustellen, dass die Vorhersagen $\boldsymbol{y}$ des Modells besser mit den Labels $\hat{\boldsymbol{y}}$ der Trainingsdaten uebereinstimmen. Dies wird iterativ gemacht, indem immer wieder leichte Anpassungen an den Parametern vorgenommen werden, bis das Modell die gewuenschten Resultate liefert.

Neben den gelernten Parametern, gibt es auch noch sogenannte **Hyperparameter**. Diese koennen nicht erlernt werden, sondern muessen manuell vor dem Training gewaehlt werden und koennen den Lernvorgang erheblich beeinflussen. Dies bedeutet, dass man ausprobieren muss, welche Werte fuer die Hyperparameter die besten Resultate liefern.

Das wohl einfachste Regressionsmodell ist eine Regressionsgerade. Diese ist angemessen, falls ein einfacher linearen Zusammenhang der Form $y = \lambda_1 x + \lambda_0$ zwischen den Features und den Labels besteht. Nun muessten nur noch λ_0 und λ_1 bestimmt werden. BEISPIELMODELL AUTO

Fuer Machine Learning haben sich gewisse Modelle besonders gut etabliert, darunter waeren: Support Vector Machines, Evolutionäre Algorithmen, und Künstliche Neuronale Netze. Diese Arbeit wird sich vorwiegend mit Neuronalen Netzen auseinandersetzen.

Abbildung 1.1: Schema eines Modells

1.2 Training

1.2.1 Verlust- und Kostenfunktionen

Einsicht ist der erste Schritt zur Besserung. Das gilt auch beim Machine Learning, deshalb muss beim Training zuerst die Genauigkeit des Modells bewertet werden. Dies wird mithilfe von sogennaten Kostenfunktionen bzw. Verlustfunktionen erreicht.

Eine **Verlustfunktion** $L(y, \hat{y})$ soll ein Mass fuer die Abweichung der Vorhersage y von dem Label $\hat{y}$ sein. Aus ihr bildet man die **Kostenfunktion** $C(\boldsymbol{y}, \hat{\boldsymbol{y}})$, indem man die Verluste der einzelnen Ouputs aufsummiert. Somit erhaelt man die Kosten einer gesamten Vorhersage mit m Outputs (siehe Gl. (1.1)). Bei gewissen Kostenfunktionen gibt es keine Verlustfunktion, da sie nicht auf die einzelnen Outputpaare aufgebrochen werden kann.
Der Fehler $\bar{C}(\mathbb{X})$ des gesamten Trainingsdatensatzes $\mathbb{X}$, der Groesse p, ergibt sich aus dem arithmetischen Mittel der einzelnen Kosten der Vorhersagen (siehe Gl. (1.2)).

$$C\left(\boldsymbol{y}, \hat{\boldsymbol{y}}\right) = \sum_{i=1}^{m} L(y_i, \hat{y}_i) \tag{1.1}$$

$$\bar{C}(\mathbb{X}) = \frac{1}{p} \sum_{j=1}^{p} C\left(\boldsymbol{y}_j, \hat{\boldsymbol{y}}_j\right) \tag{1.2}$$

Eine eine Kostenfunktion C sollte folgende Eigenschaften aufweisen:

- C ist minimal, wenn $\boldsymbol{y} = \hat{\boldsymbol{y}}$
- C waechst mit $|\boldsymbol{y} - \hat{\boldsymbol{y}}|$
- C ist nach jedem y_n partiell differenzierbar (erklaert in Sektion (1.2.2))

(Nielsen, *Neural Networks and Deep Learning*)

1.2.1.1 Mittlere quadratische Abweichung

Die bekannteste Kostenfunktion ist die "Mittlere quadratische Abweichung". (engl.: mean squared error). Sie ist definiert als das arithmetische Mittel aller quadrierten Differenzen zwischen den Vorhersagen und Labels. Zusaetzlich halbiert man noch das arithmetische Mittel, damit bei der Ableitung der Faktor 2 wegfaellt. Sie hat keine entsprechene Verlustfunktion, da man so das arithemtische Mittel nicht schreiben kann.
Die Kostenfunktion kann mithilfe einer Summe berechnen werden, oder in diesem Fall mithilfe einer Vektorsubtraktion Vorhersagen $\boldsymbol{y}$ von den Labels $\hat{\boldsymbol{y}}$.

$$C_{MSE} = \frac{1}{2n} \sum_{i=1}^{n} (\hat{y}_i - y_i)^2 = \frac{1}{2n} (\hat{\boldsymbol{y}} - \boldsymbol{y})^2 \tag{1.3}$$

Sie erfuellt alle Anforderungen an eine Kostenfunktion:

- Ihr Funktionwert ist 0 und minimal fuer $\boldsymbol{y} = \hat{\boldsymbol{y}}$
- Sie ist proportional zu $(\hat{\boldsymbol{y}} - \boldsymbol{y})^2$
- Ihre partielle Ableitung nach einem y_i lautet: $C'_{MSE} = \frac{1}{n}(y_i - \hat{y}_i)$

1.2.1.2 Cross-entropy

1.2.2 Gradientenverfahren

Um ein Modell nun zu trainieren, muss man verstehen, dass es sich um ein Optimierungsproblem handelt. Das Modell ist am besten, also macht die besten Vorraussagen, wenn die Funktionswerte der Kostenfunktion am kleinsten sind. Deshalb muss diese Kostenfunktion C minimiert werden. Hierbei muss die Funktion C nicht mehr in Abhaengigkeit der Inputs und Outputs betrachtet werden, sondern als Funktion der Modellparameter $C(\lambda_1, \lambda_2, \ldots, \lambda_k)$, denn diese sollen angepasst werden, um das Modell zu verbessern. Fuer diese Optimiertung wird das sogennante **Gradientenverfahren** (engl.: Gradient descent) verwendet. [1]

Allgemein koennen mithilfe des Gradientenverfahrens Funkionen $f(x_1, x_2, \ldots, x_n)$ vom Typ $\mathbb{R}^n \to \mathbb{R}$ (wie z.B. die Fehlerfunktion) minimiert werden. Dies geschieht, indem ein Startpunkt (Ortsvektor) $\boldsymbol{p}_0$, dessen Komponenten den Input von $f(\boldsymbol{p}_0)$ darstellen, gewaehlt wird. Nun werden iterativ neue Punkte $\boldsymbol{p}_t$ gesucht, welche immer naeher beim lokalen Minimun liegen, also Punkte, die den Funktionswert $f(\boldsymbol{p}_t)$ immer kleiner werden lassen. Dies wird durchgeführt, bis der Punkt genuegend nahe beim lokalen Minimun ist.

Dafuer muss ein Vektor $\boldsymbol{b}_t$ bestimmt werden, welcher auf den Punkt $\boldsymbol{p}_t$ addiert einen neuen Punkt $\boldsymbol{p}_{t+1}$ bildet, bei dem der Funktionswert $f(\boldsymbol{p}_{t+1})$ kleiner ist als der von $f(\boldsymbol{p}_t)$. Dies geschieht am effizientesten, wenn $\boldsymbol{b}_t$ in die Richtung der staerksten Funktionswertabnahme zeigt.
Hierzu braucht man den sogennanten **Gradient** $\boldsymbol{\nabla}$, wofür man wiederum partielle Ableitungen braucht.

Partielle Ableitungen

Partielle Ableitungen sind eine Erweiterung der "normalen" Ableitungen auf multidimensionale Funktionen $f(\boldsymbol{x}) = f(x_1, \ldots, x_n) : \mathbb{R}^n \to \mathbb{R}$. Man leitet dabei nur nach einem Argument x_i ab und betrachtet die restlichen Argumente als Konstanten. Es gelten die gleichen Ableitungsregeln wie bei der nicht-partiellen Ableitung. Die partielle Ableitung einer Funktion $f(x_1, \ldots, x_n)$ bezueglich einer Variable x_i in einem Punkt $\boldsymbol{a} = \begin{pmatrix} a_1 & \cdots & a_n \end{pmatrix}^\mathsf{T}$ ist analog zur normalen Ableitung folgendermassen definiert:

$$\frac{\partial f}{\partial x_i}(\boldsymbol{a}) := \lim_{h \to 0} \frac{f(a_1, \ldots, a_i + h, \ldots, a_n) - f(a_1, \ldots, a_i, \ldots, a_n)}{h}$$

Geometrisch ist dies die Steigung der Tangente an die Funktion f im Punkt $\boldsymbol{a}$. Die Tangente liegt in der Richtung der Achse des Parametern x_i, nach dem man ableitet.

[1] Im Gymnasium wird beigebracht die lokalen Extrema (inklusive lokalen Minimas) zu bestimmen, indem die erste Ableitung f' gebildet wird und gleich null gesetzt wird.Dies ist hier nicht moeglich, da die Funktion $C'(\lambda_1, \lambda_2, \ldots, \lambda_n)$ deutlich zu komplex ist, um die Nullstellen analytisch zu bestimmen. Deshalb wird das Gradientenverfahren verwendet.

Gradient

Der Gradient $\boldsymbol{\nabla}$ ist ein Differentialoperator, den man auf eine Funktion $f(\boldsymbol{x}) = f(x_1, \ldots, x_n) : \mathbb{R}^n \to \mathbb{R}$ anwendet kann, welcher ein Skalarfeld auf ein Vektorfeld (das sogennante Gradientenfeld) abbildet. Um den Gradienten $\boldsymbol{\nabla} f$ der Funktion f zu bilden, fasst man alle partiellen Ableitungen der Funktion f, nach jedem Argument x_i, in einem Vektor zusammen. Meistens bestimmt man den Gradienten $\boldsymbol{\nabla} f(\boldsymbol{p})$ in einem spezifischen Punkt $\boldsymbol{p} = \begin{pmatrix} p_1 & \cdots & p_n \end{pmatrix}^\mathsf{T}$.

$$\boldsymbol{\nabla}_{\boldsymbol{x}} f(\boldsymbol{p}) = \frac{\partial f}{\partial \boldsymbol{x}}(\boldsymbol{p}) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(\boldsymbol{p}) \\ \vdots \\ \frac{\partial f}{\partial x_n}(\boldsymbol{p}) \end{pmatrix}$$

Geometrisch ist der Gradient $\boldsymbol{\nabla} f(\boldsymbol{p})$ einer Funktion f in einem Punkt $\boldsymbol{p}$ dann der Vektor, welcher in die Richtung des steilsten Anstiegs von f zeigt. Sein Betrag gibt die Staerke des Anstiegs an.

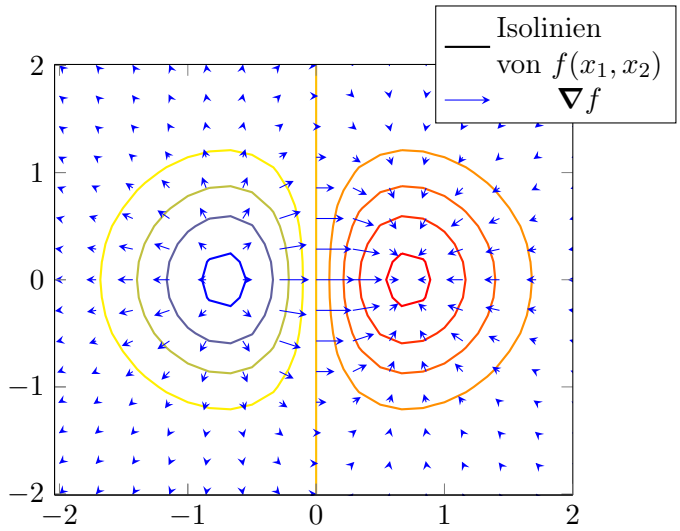


Abbildung 1.2: ein Skalarfeld als Konturdiagramm mit zugehoerigem Gradientenfeld

Da der Gradient in die Richtung des steilsten Anstiegs zeigt, sollte der Vektor $\boldsymbol{b}_t$ gerade in die entgegengesetze Richtung zeigen. Also sollte er in die Richtung des negierten Gradienten der Funktion f im Punkt $\boldsymbol{p}_t$ zeigen. Also kann jetzt das iterative Annaehern an das lokale Minimum folgendermassen beschrieben werden (wobei η eine spaeter erklaerte Schrittgroesse darstellt).

$$\boldsymbol{p}_{t+1} = \boldsymbol{p}_t - \eta \cdot \boldsymbol{\nabla} f(\boldsymbol{p}_t) \tag{1.4}$$

Waehrend des Gradientenverfahren konvergiert der Punkt $\boldsymbol{p}_t$ zu einem beliebigen *lokalen* Minimum, abhängig davon wie der Startpunkt $\boldsymbol{p}_0$ gewaehlt wurde. Wie man diesen Startpunkt zu waehlen hat, wird in Sektion (2.5.1) erlauetert. Da dieser meist zufaellig bestimmt wird, ist es eine Glückssache ein sehr niedriges Minimum zu finden.

Die sogennante **Lernrate** η aus Gleichung (1.4) ist ein Hyperparameter. Sie ist ein positiver Proportionalitatsfaktor, welcher die Schrittgroesse des Gradientenabstiegs bestimmt. Sie muss je nach zu minimierender Funktion anders gewaehlt werden. Dabei hilft nur Ausprobieren. Falls η nicht gut gewaehlt wurde, gibt es Probleme beim Training:

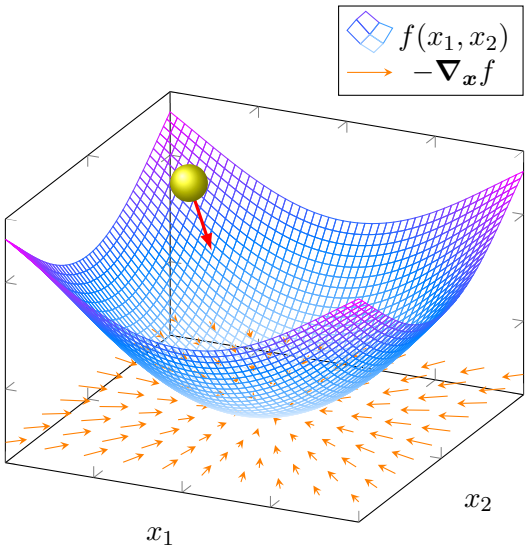


Abbildung 1.3: Visualisierung des Gradientenabstiegs: ein Ball rollt das Gradientenfeld hinab in das lokale Minimum.

- Falls η zu klein ist, verlaueft das Trainings unnoetig langsam und braucht sehr lange. Ausserdem kann es passieren, dass man bei einem hohen lokalen Minimum stecken bleibt.
- Falls η zu gross ist, kann es passiert, dass man ueber das lokale Minimum hinaus schiesst und somit nur darum herum springt.

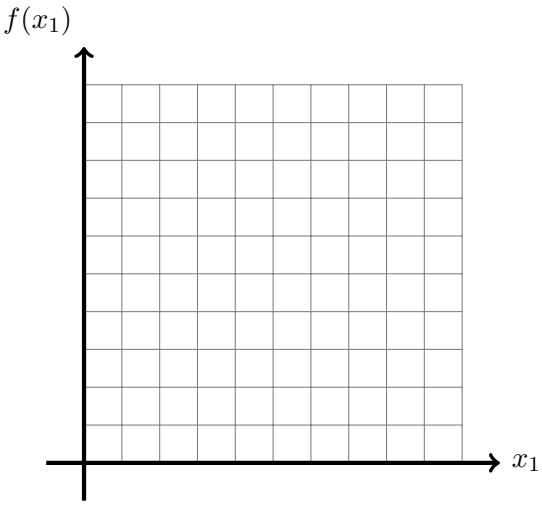


Abbildung 1.4: Visualisierung verschiedener Lernraten

(Nielsen, *Neural Networks and Deep Learning*)

1.2.3 Stochastisches Gradientenverfahren fuer Machine Learning

Wie oben erklaert, wird fuer das Trainieren eines Modells das Gradientenverfahren benutzt. Konkret wird die Kostenfunktion $C(\boldsymbol{y}, \hat{\boldsymbol{y}} \mid \lambda_0, \ldots, \lambda_n)$ (λ_i sind die Modellparameter, $\boldsymbol{y}$ die Vorhersagen und $\hat{\boldsymbol{y}}$ die Labels) minimiert, indem die Parameter angepasst werden. Dadurch macht das Modell immer bessere Vorhersagen. Fuer nur eine Iteration des Gradientenverfahren, muesste man den Gradienten fuer den *gesamten* Trainingsdatensatz berechnen. Dies waere zwar ein exakter Prozess, aber ein extrem langsamer zugleich. Bei grossen Datensaetzen wuerde es eine Ewigkeit dauern, bis das Modell nur annahernd guete Vorhersagen machen wuerde.

Aus diesem Grund verwendet man eine leicht abgeänderte Variante dieses Verfahren, naemlich das **Stochastische Gradientenverfahren** (engl.: Stochastic Gradient Descent) (SGD). Hierfuer wird der "echte" Gradient des gesamten Datensatzes mit dem Gradienten einiger Trainingssamples approximiert. Dazu wird der Trainingsdatensatz in sogennante **Mini-Batches** eingeteilt und der Gradient jeweils pro Mini-Batch berechnet. Als Konsequenz finden deutlich mehr Iterationen in *einer* Durchkaemmung der Trainingsdaten statt. Eine solche Durchkaemmung bezeichnet man als **Epoche**. Oft wird mehrere Epochen lang trainiert. Der Gradient eines genug grossen Mini-Batches ist zwar nicht ganz exakt, aber approximiert den Gradieten des gesamten Datensatzen genuegend gut. Sowohl die Mini-Batch Groesse, wie auch die Anzahl Epochen sind weitere Hyperparameter.

Die partiellen Ableitungen der gesamten Trainingsdaten wird mit dem arithmetischen Mittel der partiellen Ableitungen eines Mini-Batches der Groesse q approximiert.

$$\frac{\partial \bar{C}}{\partial \lambda_k} \approx \frac{1}{q} \sum_{i=1}^{q} \frac{\partial C_i}{\partial \lambda_k} \tag{1.5}$$

Eine Iteration des Stochastischen Gradientenverfahren wird analog zu Gleichung (1.4) folgendermassen durchgefuehrt.

$$\lambda_{k,t+1} = \lambda_{k,t} - \frac{\eta}{q} \sum_{i=1}^{q} \frac{\partial C_i}{\partial \lambda_{k,t}} \tag{1.6}$$

(Nielsen, *Neural Networks and Deep Learning*)

1.2.4 Adam Optimizer?

1.3 Trainingsphaenomene

1.3.1 Konvergenz und Divergenz

1.3.2 Underfitting und Overfitting

Kapitel 2

Deep Learning und Künstliche Neuronale Netze

Nun betrachten wir ein spezifisches Modell, welches die wohl besten Resultate fuer die meisten Problemstellungen des Maschinellen Lernens (Bilderkennung, Spracherkennung, etc.) liefert. Es handelt sich um das **Kuenstliche Neuronale Netz** (eng.: Neural Network) kurz KNN. Der Bereich des Maschinellen Lernens, welcher sich mit Neuronalen Netzen beschaeftigt, bezeichnet man als **Deep Learning**.

Kuenstliche Neuronale Netze sind zum Teil biologisch von Nervensystemen vieler Lebewesen inspiriert. Sie sind aber lediglich eine Abstraktion ihrer Informationverarbeitung und versuchen nicht eine moeglichst genaue biologische Abbildung darzustellen. Es gibt nicht nur eine Art von Neuronalem Netz, sondern es exisitieren die verschiedensten Architekturen, welche je nach Problemstellung ausgewaehlt werden muessen. Diese Arbeit wird vorallem von zwei solchen Architekturen gebrauch machen: Convolutional Neural Networks und sogenannte Autoencodern.

(Mehrere, *Kuenstliches Neuronales Netz — Wikipedia, Die freie Enyklopädie*)

2.1 Perzeptron

Um den Aufbau und die Funktion eines Kuenstlichen Neuronalen Netzes besser zu verstehen, wird im folgenden ein Vorgaenger des KNN erklaert: das **Perzeptron**.

Das einlagige Perzeptron wurde erstmals 1958 von Frank Rosenblatt vorgestellt. Dieses besteht aus einem einzigen Kuenstlichen Neuron. Dieses Kuenstliche Neuron hat mehrere binaere Inputs und einen einzigen binaeren Output. Binaer bedeutet, dass der Wert nur entweder 0 (*aus*) oder 1 (*ein*) sein kann. Des weiteren besitzt es mehrere sogenannte **Gewichte** $w_1, \ldots, w_m \in \mathbb{R}$, fuer jeden Input x_i ein Gewicht w_i. Diese sind reelle Zahlen, welche das Verhalten des Perzeptron bestimmen. Die **gewichtete Summe**, also die Summe aller Produkte der Inputs mit ihrem Gewicht, wird mit $\tilde{z}$ bezeichnet. Sie ist das gleiche wie das Skalarprodukt des Gewichtevektors $\boldsymbol{w} = \begin{pmatrix} w_1 & \cdots & w_n \end{pmatrix}^\mathsf{T}$ mit dem Inputvektor $\boldsymbol{x}$.

$$\tilde{z} = \sum_{i=1}^{m} w_i x_i = \boldsymbol{w} \cdot \boldsymbol{x}$$

Zusaetzlich besitzt das Perzeptron einen **Schwellenwert** $\tilde{b}$. Zusammen mit den Gewichten, bilden sie die Modellparameter. Das Perzeptron verhaelt sich so, dass falls die gewichtete Summe $\tilde{z}$ groesser als der Schwellenwert $\tilde{b}$ ist, das Neuron feuert, d.h. der Output 1 betraegt. Andernfalls ist er 0 (siehe erster Teil der Hypothesenfunktion h in Gleichung (2.1)). Es ist gaengig die Ungleichung der Bedingung in die Nullstellenform zu bringen und $\tilde{b}$ durch die **Neigung** (engl.: bias) $b = -\tilde{b}$ zu ersetzten. Somit lautet die Ungleichung: $\tilde{z} + b > 0$. Der neue Term $\tilde{z} + b$ wird mit z bezeichnet (siehe Rest der Gl. (2.1)). Die Neigung gibt an wie stark das Neuron dazu neigt zu feuern. Eine grosse Neigung laesst ein Neuron auch dann noch feuern, wenn nur wenige Inputs eingeschaltet sind. SCHLECHT

$$h(\boldsymbol{x}) = \begin{cases} 1 & \text{falls } \tilde{z} > \tilde{b} \\ 0 & \text{ansonsten} \end{cases} = \begin{cases} 1 & \text{falls } \tilde{z} + b > 0 \\ 0 & \text{ansonsten} \end{cases} = \begin{cases} 1 & \text{falls } \boldsymbol{w} \cdot \boldsymbol{x} + b > 0 \\ 0 & \text{ansonsten} \end{cases} \tag{2.1}$$

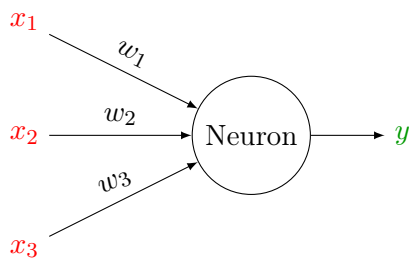


Abbildung 2.1: Perzeptron mit drei Inputs

Fuer das Trainieren des Perzeptrons existieren spezielle Verfahren, welche hier aber nicht relevant sind. Das Gradientenverfahren kann naemlich nicht verwendet werden. Der Grund dafuer sollte spaeter in Sektion (2.2) einleuchtend werden.

(Mehrere, *Perzeptron — Wikipedia, Die freie Enyklopädie*)
(Nielsen, *Neural Networks and Deep Learning*)

2.1.1 Was kann ein Perzeptron erlernen?

Nun stellt sich die Frage, was ein Perzeptron erlernen kann und wofuer es genutzt werden kann. Das Perzeptron ist lediglich ein **linearer Klassifikator** der Form $y = w_1x_1 + \cdots + w_mx_m$. Es ist also ein Klassifizierungsmodell, kein Regressionsmodell. Es kann die Features in zwei Klassen 0 oder 1 einordnen, wobei der Output der Hypothesenfunktion, diese Klassifizierung angibt. Ueberschreitet y den Schwellenwert, werden die Features der Klasse 1 zugeordnet, sonst der Klasse 0. Jedoch muessen diese Klassen linear separierbar sein.

Lineare Separierbarkeit bedeutet, dass alle Featurevektoren $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_p \in \mathbb{R}^m$ innerhalb ihres Vektorraums $\mathbb{R}^m$ durch eine Hyperebene in ihre Klassen aufteilbar sein muessen. Falls das Perzeptron zwei Inputs hat, bedeutet dies, dass die Ortsvektoren einfach durch eine Gerade voneinander trennenbar sein muessen (siehe Abb. (2.2)).
Falls die Features nicht linear separierbar sind, bedeutet dies, dass das Perzeptron die Klassifizierung nicht erlernen kann. BEISPIEL

Abbildung 2.2: erfolgreiche lineare Separierung (links) und das Versagen bei XOR (rechts)

(Mehrere, *Perzeptron — Wikipedia, Die freie Enyklopädie*)
(Mehrere, *Linear separability — Wikipedia, The Free Encyclopedia*)

2.2 Erweiterung der Kuenstlichen Neuronen

Ein Perzeptron ist, wie vorhin erklaert, nur in der Lage, lineare Klassifikationen durchzufuehren. Um nun auch kompliziertere Probleme zu loesen, muss das Prinzip ausgebaut werden. Ausserdem brauchen wir ein Kuenstliches Neuron, welches sich besonders gut als Baustein fuer KNNs eignet.

2.2.1 Kuenstliche Neuronen im Allgemeinen

Kuenstliche Neuronen sind immer so aufgebaut, dass sie einen oder mehrere Inputs haben und einen einzigen Output. Zu jedem Input x_i ist ein Gewicht w_i assoziert. Zuerst wird die gewichtete Summe der Inputs $\tilde{z}$ gebildet. Die Neigung b wird ebenfalls draufaddiert, um z zu erhalten. Nun muss die sogenannte **Aktivierung** a gebildet werden. Sie ist der Output des Neurons. Die Aktivierung $a = \varphi(z)$ ist das Resultat der **Aktivierungsfunktion** $\varphi : \mathbb{R} \to \mathbb{R}$ angewendet auf z. Die verschiedenen Kuenstlichen Neuronen unterscheiden sich fast nur in ihrer Aktivierungsfunktion.

Abbildung 2.3: ein kuenstliches Neuron und seine Bestandteile

(Nielsen, *Neural Networks and Deep Learning*)
(Mehrere, *Kuenstliches Neuron — Wikipedia, Die freie Enyklopädie*)

2.2.2 Perzeptronen als Kuenstliche Neuronen

Zuerst noch einmal ein Blick auf das Perzeptron im Angesicht der Aktivierungsfunktion. Ein wesentlicher Unterschied des Perzeptron gegenueber sonstigen Kuenstlichen Neuronen, besteht darin, dass seine Inputs und Outputs nur binaere Werte annehmen koennen. Um dieses Verhalten des Perzeptrons zu erhalten, muss eine Stufenfunktion als Aktivierungfunktion verwendet werden: die Heaviside-Funktion Θ. Sie hat einen einzigen Stufensprung bei $x = 0$ vom Wert 0 auf 1 (siehe Abb. (2.4)).

$$\varphi^{\text{hlim}}(z) = \Theta(z) = \begin{cases} 1 & \text{falls } z \geq 0 \\ 0 & \text{falls } z < 0 \end{cases}$$

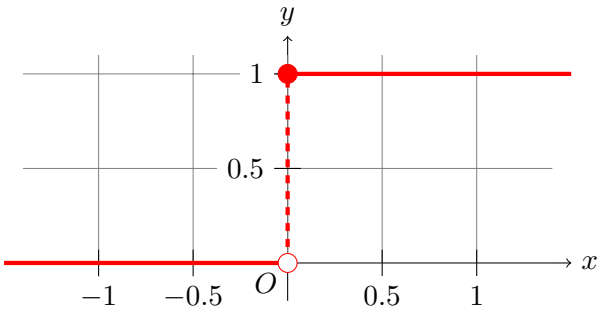


Abbildung 2.4: Definition und Graph der Heaviside-Funktion Θ

(Mehrere, *Kuenstliches Neuron — Wikipedia, Die freie Enyklopädie*)
(Mehrere, *Perzeptron — Wikipedia, Die freie Enyklopädie*)

2.2.3 ReLU Neuronen

Der naechste Schritt nach einer Stufenfunktion als Aktivierungsfunktion, waren lineare Aktivierungsfunktionen. Fuer diese koennen die Inputs nun belibige reelle Zahlen sein. Jedoch sind solche lineare Neuronen in einem KNN von keinem Nutzen. Das hat den Grund, dass eine Verkettung von linearen Neuronen, immer auf eine einzige lineare Funktion reduziert werden kann. Somit hat die Verkettung keinen Mehrwert.

Stattdessen verwendet man sogennante ReLU Neuornen. Sie benutzen die **Rectified Linear Unit** (**ReLU**) Aktivierungsfunktion. Diese ist eine nur teilweise lineare Aktiverungsfunktion. Die Werte groesser als 0 werden auf sich selber linear abbgebildet und die Werte kleiner als 0 werden auf 0 abbgebildet (siehe Abb. (2.5)). Eine sehr wichtige Eigenschaft von ihr ist, dass sie - im Gegensatz

zu den vorhin genannten Aktivierungsfunktionen - fast ueberall differenzierbar[1]und strikt monoton steigend ist. Erst fuer diese Aktivierungsfunktion, kann das Gradientenverfahren angewendet werden und somit das KNN trainiert werden.

Da sie nur teilweise linear ist, gehoert sie genaugenommen den Nicht-linearen-Aktiverungsfunktionen an. Diese Nicht-Linearitaet erlaubt es dem Neuronen deutlich komplexere Systeme zu modellieren und so deutlich komplexere Probleme zu loesen. Wir werden in Sektion (2.7) sogar erfahren, dass eine Kompostion von nicht-linearen Neuroen jede beliebige Funktion approximieren kann.

Wir werden diese Art von Neuron vorerst zur Seite legen und erst wieder in Kapitel (**??**) im Zusammenhang mit KNNs zur Bilderkennung betrachten.

$$\varphi^{\text{ReLU}}(z) = \begin{cases} z & \text{falls } z > 0 \\ 0 & \text{falls } z \leq 0 \end{cases} = \max(z, 0) \quad (2.2)$$

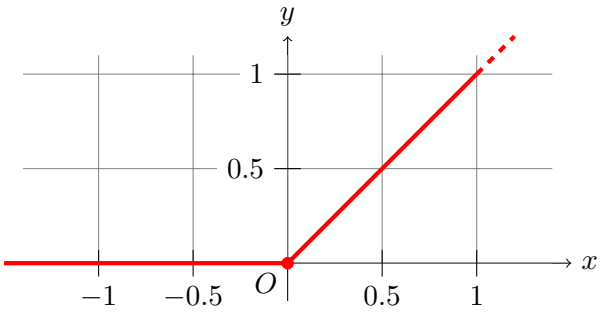


Abbildung 2.5: Formel und Graph der ReLU-Funktion

(Mehrere, *Kuenstliches Neuron — Wikipedia, Die freie Enyklopädie*)
(Nielsen, *Neural Networks and Deep Learning*)

2.2.4 Sigmoid Neuronen

Ein weiteres nicht-lineares Neuron, sind sogennante Sigmoid-Neuronen. Den Namen haben sie von ihrer Aktivierungsfunktion: der Sigmoidfunktion σ.

Auch sie kann aufgrund ihrer Nicht-linearitaet zur Approximation jeder Funktion in einem KNN verwendet werden. Sie ist die Aktivierungsfunktion, welche am meisten in KNNs eigesetzt wird. Da sie wirklich sehr stark von Lineartiaet abweicht und so am schnellsten komplexe Sachverhalte modellieren kann.

Die Sigmoid-Funktion besitzt eine einzige Wendestelle $\sigma''(x = 0) = 0$ und hat zwei Asymptoten, eine $\lim\limits_{x \to -\infty} \sigma(x) = 0$ und eine zweite $\lim\limits_{x \to \infty} \sigma(x) = 1$ (siehe Abb. (2.6)). Desweiteren zeichnet sie sich durch ihre simple Ableitung aus.

(Mehrere, *Kuenstliches Neuron — Wikipedia, Die freie Enyklopädie*)
(Mehrere, *Sigmoidfunktion — Wikipedia, Die freie Enyklopädie*)

2.3 Topologie der Kuenstlichen Neuronalen Netzen

Nun sollten diese Sigmoiden Neuronen als Bausteine verwendet werden, um ein Kuenstliches Neuronales Netz zu bilden. Dazu werden sie miteinander verbunden und bilden so ein Netz, aehnlich wie ein Nervensystem.

[1]Eigentlich ist die ReLU-Funktion in $x = 0$ wegen des Knicks nicht differenzierbar. Fuer die Gradientenberechnung definiert man jedoch einfach die Ableitung $\varphi^{\text{ReLU}\prime}(0) := 0$. Dies ist mathematisch zwar nicht korrekt, loest aber das Problem.

$$\varphi^{\text{sig}}(z) = \sigma(z) = \frac{1}{1+e^{-z}}$$
$$\sigma'(z) = \sigma(z)(1-\sigma(z))$$

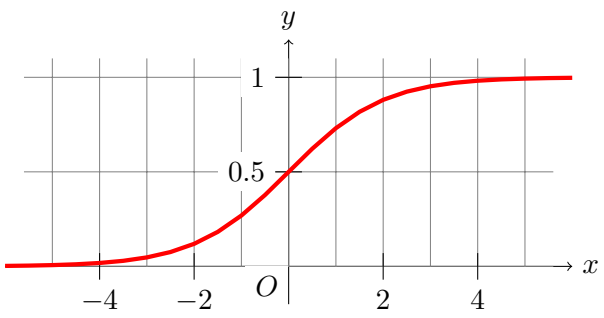


Abbildung 2.6: Definition, Ableitung und Graph der Sigmoid-Funktion σ

Diese Neuronen sind in verschieden Schichten (engl.: Layers) arangiert. Die erste ist die **Inputschicht**. Sie beinhaltet die Inputneuronen. Diese sind eigentlich keine richtigen Neuronen sondern eher Platzhalter fuer ihr jeweiliges Feature x_i. Als letztes kommt die **Outputschicht** mit den Outputneuronen, welche jeweils einen Outputwert y_i besitzen. Dazwischen liegen die **Zwischenschichten** (engl.: hiddenlayers). Von ihnen kann es beliebig viele geben und in ihnen beliebig viele Neuronen. Falls viele Zwischenschichten verwendet werden, bezeichnet man das Netzwerk als "deep". Daher ruert auch der Name des Deep Learnings. Der Aufbau eines KNN bezeichnet man als **Topologie** des Netzes. Die Topologie umfasst viele Hyperparameter. Darunter sind zum Beispiel die Anzahl Zwischenschichten, wie auch die Anzahl Neuronen pro Schicht.

Jedes Neuron aus einer Schicht ist mit jedem Neuron aus der naechsten Schicht ueber Verbindungen gekoppelt. Alle Verbindungen besitzten ein Gewicht analog zu den Inputs des Perzeptrons. Die Aktivierung, also der Output, eines Neurons wandert entlang den jeweiligen Verbindung zu allen Neuronen der naechsten Schicht und dient als deren Input. Die soeben beschriebe Art von KNN nennt man **Feedforward-Netz**, da alle Werte ausschliesslich nach vorne propagiert werden.

In Abbildung (2.7) ist ein Beispiel eines Neuronalen Netzes abgebildet. In diesem Fall besitzt es sowohl 4 Inputs, als auch 4 Outputs. Es hat ausserdem 3 Zwischenschichten. Die erste und die dritte haben jeweils 3 Neuronen und die zweite besitzt 4 Neuronen.

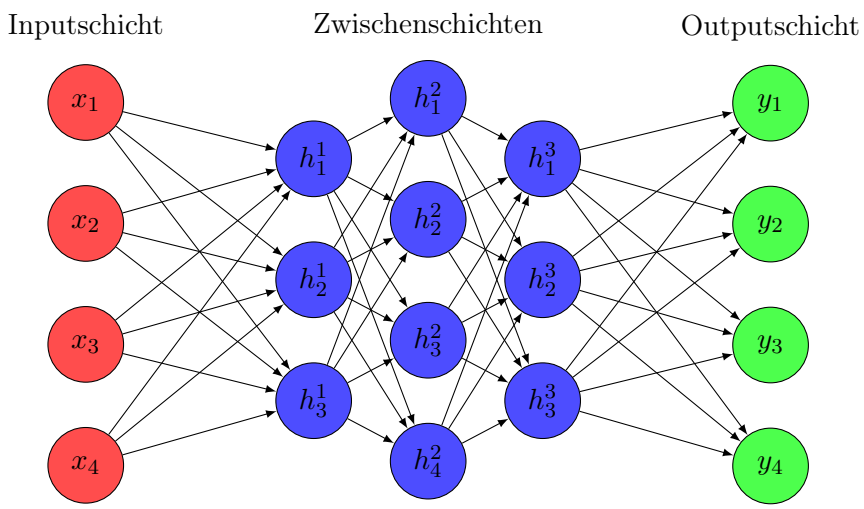


Abbildung 2.7: Schichten eines KNNs

(Mehrere, *Kuenstliches Neuronales Netz — Wikipedia, Die freie Enyklopädie*)
(Nielsen, *Neural Networks and Deep Learning*)

2.4 Lernverhalten

Die Hoffnung beim Trainieren von KNNs besteht darin, dass das Modell fuer jede weiter Schicht eine hoeheres Abstraktionsniveau erreicht. Wuerde man zum Beispiel ein Netzwerk zur Gesichtserkennung trainieren, koennte man sich den Erkennungsprozess folgendermassen vorstellen: Die erste Zwischenschicht erkennt Kanten und Konturen. Die zweite vereint diese Merkmale zu Ecken und primitiven geometrischen Formen. Die dritte Schicht sollte dann schon komplexere geometrische Formen erkennen, welche gewissen Gesichtmerkmalen, wie der Nase, aehneln. Die letzten Schichten soll dann alle diese Merkmale zusammensetzen und so ein Gesicht erkennen. WEITERFUEHREN

2.5 Vorwaertspropagierung

Jetzt, da der Aufbau eines KNNs erklaert wurde, sollte nun die mathematische Funktionsweise des Modells erklaert werden. Hierfuer muessen einige Konventionen zur Bezeichnung der Teile eines KNNs getroffen werden. Es sollten vorallem noch Abbildungen (**??**) und (**??**) zum Verstaentnis der Nomenklatur studiert werden.

- l ist der Index einer Schicht. Die Indexierung beginnt bei 0.
- L ist der letzte Schichtindex und somit auch die gesamte Anzahl an Schichten (ohne die Inputschicht).
- $|l|$ ist die Anzahl Neuronen in der l-ten Schicht. [2]
- n_j^l bezeichnet das j-te Neuron in der l-ten Schicht.
- z_j^l ist die gewichtete Summe der Inputs des j-ten Neuron in der l-ten Schicht.
- a_j^l ist die Aktivierung/Output des j-ten Neurons in der l-ten Schicht.
- b_j^l ist die Neigung fuer das j-te Neuron in der $(l+1)$-ten Schicht. [3]
- $w_{j,k}^l$ ist das Gewicht der Verbindung vom k-ten Neuron in der l-ten Schicht zum j-ten Neuron in der $(l+1)$-ten Schicht. [4]
- φ ist die gewaehlte Aktivierungsfunktion (diese ist immer eine nicht-lineare Aktivierungsfunktion)

Die Vorwaertspropagierung beginnt bei den Inputneuronen, welche jeweils einen Inputwert in sich tragen. Diese Werte werden, um fuer eine kohaerente Nomenklatur zu sorgen, analog zu den Aktivierungen der anderen Neuronen mit a_j^0 bezeichnet, wobei j der Index des Neurons ist.
Nun muessen die restlichen Aktivierungen der Neuronen bis und mit den Ouputneuronen berechnet werden. Dies geschieht rekursiv, anhand der Aktivierungen der voherigen Schicht und zwar folgendermassen (ersichtlich in Gleichung (FP1)).

Zuerst laeuft eine Summe ueber alle Neuronen n_k^l der jetztigen Schicht l. Dabei wird die gewichtete Summe der Aktivierungen a_k^l mit den assozierten Gewichten $w_{j,k}^l$ gebildet. ZU KOMPLIZIERT RELEVANT?: Hierbei ist das Gewicht jenes, welches das k-te Neuron der l-ten Schicht mit dem j-ten Neuron der $(l+1)$-ten Schicht verbindet. Zusaetzlich gehoert zu der gewichteten Summe auch die jeweilige Neigung b_j^l, welche dazuaddiert wird. Diese gewichtete Summe wird mit z_j^{l+1} bezeichnet.

$$z_j^{l+1} = \sum_{k=1}^{|l|} w_{j,k}^l a_k^l + b_j^l \tag{FP1}$$

[2] Diese Schreibweise hat nichts mit dem Betrag zu tun, sondern wird einfach gewaehlt, da sie sehr platzsparend ist.
[3] Diese Konvention wurde gewaehlt, damit die folgenden Gleichungen simpler sind.
[4] Man beachte die Reihenfolge!
Diese Konvention scheint zwar auf den ersten Blick unintuitiv, macht jedoch Sinn fuer die Matrixindezierung. (2.6).

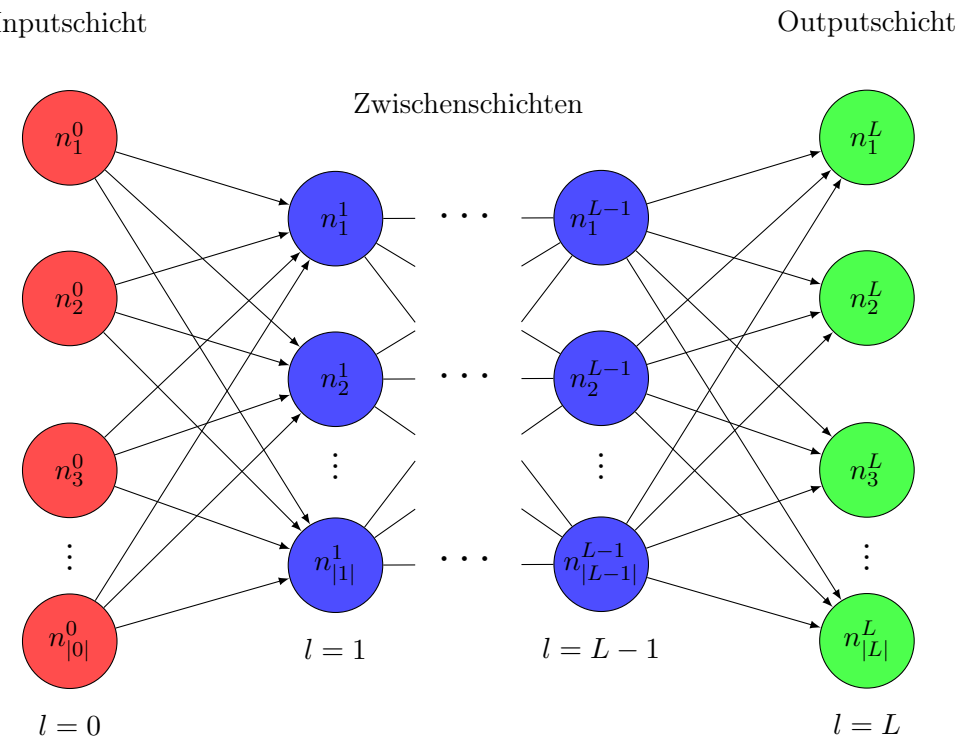


Abbildung 2.8: zum Verstaentniss der Nomenklatur

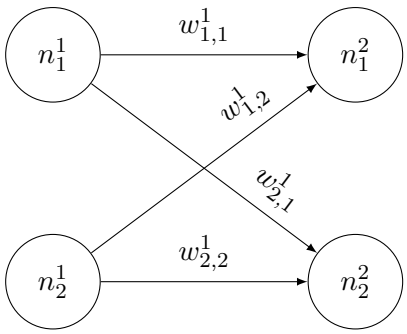


Abbildung 2.9: zum Verstaentnis der Gewichtebeschriftungen

Auf diese Summe wird dann die Aktivierungsfunktion φ angewandt. Das ist dann die Aktivierung a_j^{l+1} des j-ten Neurons in der $(l+1)$-ten Schicht.

$$a_j^{l+1} = \varphi\left(\sum_{k=1}^{|l|} w_{j,k}^l a_k^l + b_j^l\right) = \varphi\left(z_j^{l+1}\right) \tag{FP2}$$

Fuer Deep Learning braucht man vorallem sogenannte Deep Neural Networks. Diese zeichnen sich dadurch aus, dass sie sehr viele Zwischenschichten besitzen. Deshalb bezeichnet man sie auch als "deep". Bei solchen Netzwerken ist es nicht unueblich, dass diese sehr viele Neuronen und Verbindungen (ueber 100'000) besitzen. Um hierbei nicht den Ueberblick zu verlieren und um nicht in den Indizes zu ertrinken, macht man Gebrauch von **Linearer Algebra**. Man verwendet Matrizen und Vektoren um die vielen Variabeln zusammenzufassen. Ausserdem besteht ein weiterer Vorteil darin, dass Computer mithilfe von Vektor- und Matrixoperationen die Berechnungen paralellisieren koennen und in kuerzerer Zeit und mit weniger Ressourcen viele Berechnungen gleichzeitig ausfuehren koennen. Dies beschleunigt das Training der Modelle um ein Vielfaches. Dies wird spaeter in Sektion (**??**) noch weiter thematisiert.

Die Inputs $\boldsymbol{x}$, Vorhersagen $\boldsymbol{y}$ und Labels $\hat{\boldsymbol{y}}$ haben wir schon von Anfang an als Vektoren geschrieben.

Nun sollen noch die Modellparameter und die restlichen Komponenten eines KNNs als Vektoren und Matrizen zusammengefasst werden. Sowohl alle gewichteten Summen z_j^l, wie auch alle Aktivierungen a_j^l einer Schicht l, werden in Vektoren $\boldsymbol{z}^l \in \mathbb{R}^{|l|}$ und $\boldsymbol{a}^l \in \mathbb{R}^{|l|}$ zusammengefasst. Auch alle Neigung b_j^l fuer eine Schicht $(l+1)$ bilden einen Vektor $\boldsymbol{b}^l \in \mathbb{R}^{|l+1|}$.

Zu guter letzt, wird noch eine **Gewichtsmatrix** $\boldsymbol{W}^l \in \mathbb{R}^{|l+1|\times|l|}$ definiert. Sie enthaelt alle Gewichte welche die l-te Schicht *zu* der $(l+1)$-ten Schicht verbindet. Das heisst der Eintrag in der j-ten Zeile und in der k-ten Spalte ist $w_{j,k}^l$ und verbindet so das Neuron n_k^l zu dem Neuron n_j^{l+1}.

$$\boldsymbol{z}^l = \begin{pmatrix} z_1^l & z_2^l & \cdots & z_{|l|}^l \end{pmatrix}^\mathsf{T}$$
$$\boldsymbol{a}^l = \begin{pmatrix} a_1^l & a_2^l & \cdots & a_{|l|}^l \end{pmatrix}^\mathsf{T}$$
$$\boldsymbol{b}^l = \begin{pmatrix} b_1^l & b_2^l & \cdots & b_{|l+1|}^l \end{pmatrix}^\mathsf{T}$$

$$\boldsymbol{W}^l = \begin{pmatrix} w_{1,1}^l & w_{1,2}^l & \cdots & w_{1,|l|}^l \\ w_{2,1}^l & w_{2,2}^l & \cdots & w_{2,|l|}^l \\ \vdots & \vdots & \ddots & \vdots \\ w_{|l+1|,1}^l & w_{|l+1|,2}^l & \cdots & w_{|l+1|,|l|}^l \end{pmatrix}$$

Mit diesen Definitionen kann nun fast Gleichung (FP1) in Matrixform geschrieben werden, denn die Matrixmultiplikation von $\boldsymbol{W}^l$ mit $\boldsymbol{a}^l$ ergibt einen Vektor $\tilde{\boldsymbol{z}}^{l+1}$, welcher alle gewichteten Summen $\tilde{z}_j^{l+1}$ ohne die jeweilige Neigung enthaelt.

$$\boldsymbol{W}^l\boldsymbol{a}^l = \begin{pmatrix} \sum_{j=1}^{|l|} w_{1,j}^l a_j^l & \sum_{j=1}^{|l|} w_{2,j}^l a_j^l & \cdots & \sum_{j=1}^{|l|} w_{|l+1|,j}^l a_j^l \end{pmatrix}^\mathsf{T} = \tilde{\boldsymbol{z}}^{l+1}$$

Nun muessen wir noch den Neigungsvektor $\boldsymbol{b}^l$ draufaddieren und wir erhalten Gleichung (FP1a), mithilfe der wir den Vektor der gewichteten Summen $\boldsymbol{z}^{l+1}$ bilden koennen.

$$\boldsymbol{z}^{l+1} = \mathbf{W}^l\boldsymbol{a}^l + \boldsymbol{b}^l \tag{FP1a}$$

Der letzte Schritt besteht noch darin die Aktivierungsfunktion auf $\boldsymbol{z}^{l+1}$ anzuwenden um den Aktivierungsvektor $\boldsymbol{a}^{l+1}$ zu bilden. Hierfuer muss aber noch ein neues mathematisches Konzept eingefuehrt werden: die Vektorisierung einer Funktion.

Vektorisierung einer Funktion

Die Vektorisierung einer skalaren Funktion f, geschrieben als $\boldsymbol{f}[\boldsymbol{v}]$ hat als Argument einen Vektor $\boldsymbol{v}$, auf dessen Komponenten jeweils *einzeln* die Funktion f angewandt wird. Dieser neue Vektor ist der Rueckgabewert der Funktion. Er besitzt die gleichen Dimensionen wie der Argumentvektor.

$$\boldsymbol{f}[\boldsymbol{v}] = \begin{pmatrix} f(v_1) \\ \vdots \\ f(v_n) \end{pmatrix}$$

Nun kann ganz einfach die vektorisierte Aktivierungsfunktion $\boldsymbol{\varphi}$ auf $\boldsymbol{z}^{l+1}$ angewandt werden.

$$\boldsymbol{a}^{l+1} = \boldsymbol{\varphi}\left[\boldsymbol{W}^l\boldsymbol{a}^l + \boldsymbol{b}^l\right] = \boldsymbol{\varphi}\left[\boldsymbol{z}^{l+1}\right] \tag{FP2a}$$

(Nielsen, *Neural Networks and Deep Learning*)

2.5.1 Modellparameter initialisieren

Ein Schritt der getan werden muss, bevor das Training beginnt, ist das Initialisieren aller Modellparameter, in diesem Fall die Gewichte und Neigungen. Dies ist ein sehr essentieller Schritt, denn diese Startwerte entscheiden erheblich ueber die Leistungsfaehigkeit des Modells.

Wie in Sektion (1.2.2) gezeigt, muss am Anfang des Gradientenverfahrens ein Startpunkt $\boldsymbol{p}_0$ innerhalb des Gradientenfeldes $\boldsymbol{\nabla} C$ gewaehlt werden, von welchem aus der Gradientenabstieg beginnt. Da wir das Gradientenverfahren zur Optimierung der Modellparameter verwenden, ist unser Startpunkt der Vektor aller Hyperparameter $\boldsymbol{\lambda} = \begin{pmatrix}\lambda_1 & \cdots & \lambda_k\end{pmatrix}^\mathsf{T}$. Dieser Startpunkt entscheidet darueber, in welches lokale Minimum konvergiert wird und bestimmt somit auch die bestmoegliche Exaktheit der Vorhersagen. Falls schlechte Initialwerte gewaehlt werden, konvergiert der Punkt in ein hohes lokales Minimum, was grosse Kostenfunktionswerte und schlechte Vorhersagen verursacht.

Es ist nicht moeglich im Vorhinein zu wissen, welche Initialwerte gute Resultate liefern. Man muss ausprobieren, deshalb initialisert man gaengierweise die Hyperparameter mit Zufallswerten. Dafuer nimmt man aber nicht irgendwelche Zufallsvariabeln sondern man benutzt die Gauss'sche Normalverteilung $\mathcal{N}(\mu, \sigma^2)$ bzw. ihre Dichtefunktion.

$$\phi(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left\{-\frac{(x-\mu)^2}{2\sigma^2}\right\}$$

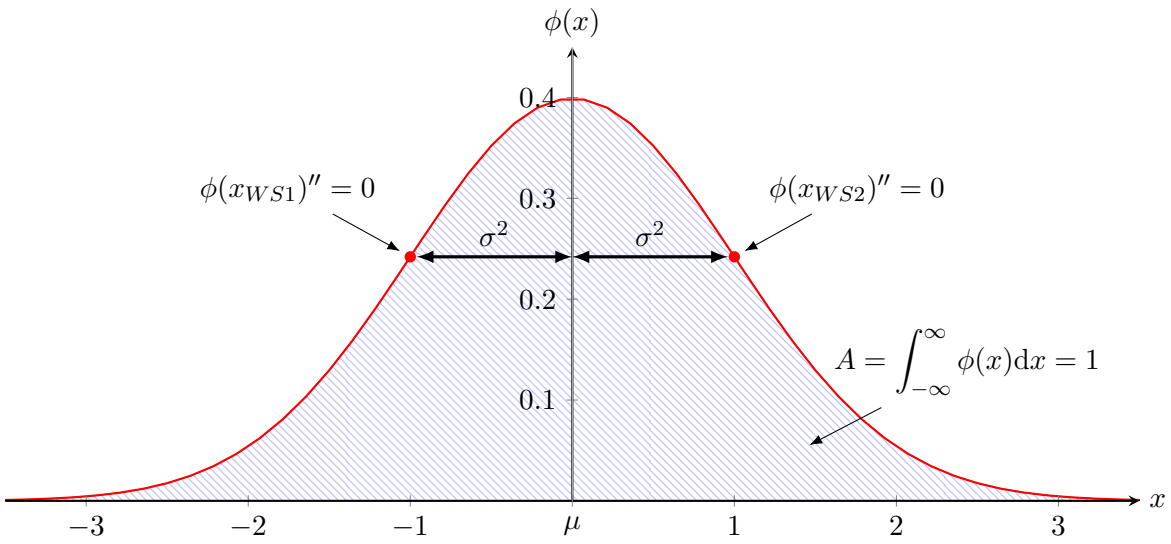


Abbildung 2.10: Graph der Dichtefunktion $\phi(x \mid \mu = 0, \sigma^2 = 1)$ mit ihren wichtigsten Eigenschaften

Um die Neigungen $b_{t=0}$ zu initialisieren benutzt man eine Normalverteilung $\mathcal{N}(0, 1)$ mit Erwartungswert $\mu = 0$ und Varianz $\sigma^2 = 1$. Um die Gewichte $w_{t=0}$ zu initialisieren benutzt man ebenfalls eine Normalverteilung mit Erwartungswert $\mu = 0$, jedoch wird die Varianz σ^2 so skaliert, dass die Summe aller Zufallesvariabeln der Gewichte einer Schicht eine Varianz von $\sigma^2_{tot} = 1$ hat. ERKLAEREN

$$w^l_{t=0} \sim \mathcal{N}(\mu = 0, \sigma^2 = \frac{1}{|l|}) \tag{2.3}$$

$$b^l_{t=0} \sim \mathcal{N}(\mu = 0, \sigma^2 = 1) \tag{2.4}$$

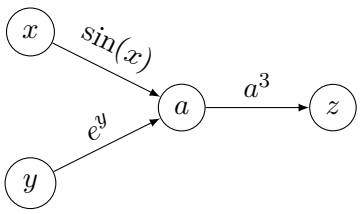
(Mehrere, *Normal distribution — Wikipedia, The Free Encyclopedia*)
(Nielsen, *Neural Networks and Deep Learning*)

2.6 Rueckwaertspropagierung

Die wahre Herausforderung besteht beim Gradientenverfahren darin, die partiellen Ableitungen der Modellparameter, also die Komponenten des Gradienten, zu bestimmen. Anders gesagt muessen alle Terme $\frac{\partial C}{\partial w^l_{j,k}}$, wie auch alle Terme $\frac{\partial C}{\partial b^l_k}$ berechnet werden. Das Verfahren zum bestimmen dieser Ausdruecke ist so spezifisch und aufwendig, dass das Gradientenverfahren fuer KNNs einen eigenen Namen hat: die **Rueckwaertspropagierung** (engl.: backpropagation) (auch Fehlerrueckfuehrung). Im Verlauf der Arbeit ist es nicht notwendig die Rueckwaertspropagierung bis ins Detail zu verstehen. Man sollte lediglich ein Gespuer dafuer entwickeln, wie der Gradient berechnet wird.

Da ein KNN, wie der Name es schon sagt, vernetzt ist, koennen die partiellen Ableitungen einer Schicht bezueglich seiner Nachbarsschichten berechnet werden. Dies ist auch der namensgebende Grundgedanke der Rueckwaertsprogagierung: Man beginnt in der letzten Schicht die partiellen Ableitungen zu bestimmen und berechnet dann im Rueckwaertsgang Schicht fuer Schicht die vorherigen partiellen Ableitungen, bis zur Inputsschicht.
Dies macht man mithilfe der Kettenregel der Ableitungen. Es ist sinnvoll fuer das Aufstellen dieser Gleichungen das Netzwerk als **Computation Graph** zu betrachten.

Ein Computation Graph ist eine Darstellung einer Verkettung von Funktionen als Netzwerk von Operationen. Die Knoten im Graph stellen Variabeln dar und die Pfade, welche die Knoten verbinden, sind die Funktionen, welche die Variabeln aufeinander abbilden. Die Funktion wird auf die Variable angewandet, von der der Pfad ausgeht. Der Knoten in welchem der Pfad endet, nimmt dann den Funktionswert an. Falls mehrere Pfade in einem Knoten enden, werden die einzelnen Werte der Pfade zusammenaddiert, um die Variable zu bilden. KETTENREGEL NOCHMALS ERWAEHNEN ODER IN ABBILDUNG (LEIBNIZ-SCHREIBWEISE!)
In Abbildung (2.11) ist ein Beispiel eines Computation Graphs zusammen mit der Herleitung der partiellen Ableitungen dargestellt.



$$a(x,y) = \sin(x) + e^y$$
$$z(a(x,y)) = a^3(x,y) = (\sin(x) + e^y)^3$$

$$\frac{\partial z}{\partial x} = \frac{\partial a}{\partial x} \cdot \frac{\partial z}{\partial a} = \cos(x) \cdot 3a^2$$
$$\frac{\partial z}{\partial y} = \frac{\partial a}{\partial y} \cdot \frac{\partial z}{\partial a} = e^y \cdot 3a^2$$

Abbildung 2.11: Computation Graph einer exemplarischen Verkettung von Funktionen

Der erste Schritt der Rueckwaertsprogagierung besteht darin, dass man die partiellen Ableitungen $\frac{\partial C}{\partial z^l_j}$ der Kostenfunktion C bezueglich den gewichteten Summen z^l_j aller Schichten berechnet. Daraus laesst sich dann spaeter sehr einfach die partiellen Ableitungen bezueglich den Gewichten $\frac{\partial C}{\partial w^l_{j,k}}$ und bezueglich den Neigung $\frac{\partial C}{\partial b^l_j}$ berechnen.

Zur Uebersichtlichkeit definiert man einen **Fehler** δ^L_j fuer jedes j-te Neuron in jeder l-ten Schicht, welcher die partielle Ableitung bezueglich der gewichteten Summe dieses Neurons ist (siehe Gl. (**??**). Ebenfalls definiert man analog einen Fehlervektor $\boldsymbol{\delta}^l$, welcher alle Fehler δ^l_j einer Schicht l zusammenfasst (siehe Gl. (BP0a)). Nun heisst es diesen fuer jedes Neuron jeder Schicht zu berechnen.

$$\delta_j^l := \frac{\partial C}{\partial z_j^l} \tag{BP0}$$

$$\boldsymbol{\delta}^l := \begin{pmatrix} \frac{\partial C}{\partial z_1^l} & \frac{\partial C}{\partial z_2^l} & \cdots & \frac{\partial C}{\partial z_{|l|}^l} \end{pmatrix}^\mathsf{T} \tag{BP0a}$$

Da die Kostenfunktion unmittelbar auf die letzte Schicht L angewandt wird, beginnt man auch dort mit der Berechnung des Fehlers $\boldsymbol{\delta}^L$. Wir stellen nun einen Computation Graph auf, um die partiellen Ableitungen zu bilden.

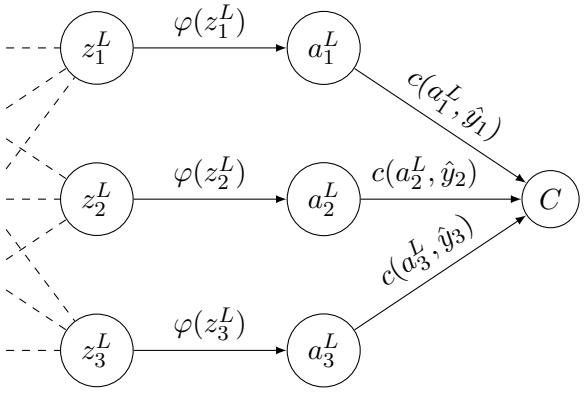


Abbildung 2.12: Computation Graph zur Berechnung von $\boldsymbol{\delta}^L$

Wir koennen dem Computation Graph aus Abbildung (2.12) entnehmen, dass die Kosten C eine Funktion in Abhaengigkeit von den letzten Aktivierungen a_j^L ist, welche wiederum in Abhaengigkeit von der jeweiligen gewichteten Summe z_j^L berechnet werden. Somit koennen wir mithilfe der Kettenregel die Beziehung (2.5) aufstellen.

$$\delta_j^L = \frac{\partial C}{\partial z_j^L} = \frac{\partial C}{\partial a_j^L} \cdot \frac{\partial a_j^L}{\partial z_j^L} \tag{2.5}$$

Da a_j^L durch die Anwendung der Aktivierungsfunktion φ auf z_j^L gebildet wird, ist $\frac{\partial a_j^L}{\partial z_j^L}$ einfach die Ableitung der Aktivierungsfunktion $\varphi'(z_j^L)$. Somit erhalten wir die erste (BP1) von vier wichtigen Gleichungen fuer die Rueckwaertsprogagierung.

$$\delta_j^L = \frac{\partial C}{\partial a_j^L} \cdot \varphi'(z_j^L) \tag{BP1}$$

Nun moechten wir diese Ausdruecke wieder in Matrixschreibweise realisieren, welche die ganze Schicht L zusammenfasst. Dazu muss eine neue Operation eingefuehrt werden: das Hadamard-Produkt.

Hadamard-Produkt

Das Hadamard-Produkt (auch elementweises Produkt) ist ein spezielles Produkt zweier gleichgrossen Matrizen $\boldsymbol{A} \in \mathbb{R}^{m\times n}$ und $\boldsymbol{B} \in \mathbb{R}^{m\times n}$. Die resultierende Matrix ergibt sich aus der elementweisen Multiplikation der Ausgangsmatrizen.

$$\boldsymbol{A} \odot \boldsymbol{B} = \begin{pmatrix} A_{1,1}B_{1,1} & \cdots & A_{1,n}B_{1,n} \\ \vdots & \ddots & \vdots \\ A_{m,1}B_{m,1} & \cdots & A_{m,n}B_{m,n} \end{pmatrix} \in \mathbb{R}^{m\times n} \qquad \boldsymbol{v} \odot \boldsymbol{w} = \begin{pmatrix} v_1 w_1 \\ \vdots \\ v_n w_n \end{pmatrix}$$

Mit $\boldsymbol{\varphi}'$ als die vektorisierte Ableitung der Aktivierungsfunktion, koenne wir den Fehlervektor der letzten Schicht nach Gleichung (2.6) berechnen.

$$\boldsymbol{\delta}^L = \begin{pmatrix} \frac{\partial C}{\partial a_1^L} & \frac{\partial C}{\partial a_2^L} & \cdots & \frac{\partial C}{\partial a_{|L|}^L} \end{pmatrix}^\mathsf{T} \odot \boldsymbol{\varphi}'[\boldsymbol{z}^L] \tag{2.6}$$

Dabei ist der erste Operand des Hadamard-Produkts nichts anderes als der Gradient $\boldsymbol{\nabla}_{\boldsymbol{a}^L} C$ der Kostenfunktion C bezueglich dem Aktivierungsvektor $\boldsymbol{a}^L$ der letzten Schicht. Dieser Gradient kann einfach berechnet werden, indem man die vektorisierte Ableitungsfunktion fuer die gewaehlte Kostenfunktion bildet. Wuerde man die Mittlere quadratischen Abweichung $C = \frac{1}{2|L|}(\hat{\boldsymbol{y}} - \boldsymbol{a}^L)^2$ als Kostenfunktion waehlen, haetten wir $\boldsymbol{\nabla}_{\boldsymbol{a}^L} C = (\boldsymbol{a}^L - \hat{\boldsymbol{y}}) \cdot \frac{1}{|L|}$.

Daraus folgt die kompakte matrix-version (BP1a) der Gleichung (BP1), welche den Fehlervektor fuer die letzte Schicht berechnet.

$$\boldsymbol{\delta}^L = \boldsymbol{\nabla}_{\boldsymbol{a}^L} C \odot \boldsymbol{\varphi}'(\boldsymbol{z}^L) \tag{BP1a}$$

Nun muessen wir eine rekursive Berechnungsmethode des Fehlers δ_j^{l-1} der vorherigen Schicht anhand des Fehlers δ_j^l der jetzigen Schicht erarbeiten. Dafuer stellen wir zuallererst wieder einen Computation Graph auf (siehe Abb. (2.13)).

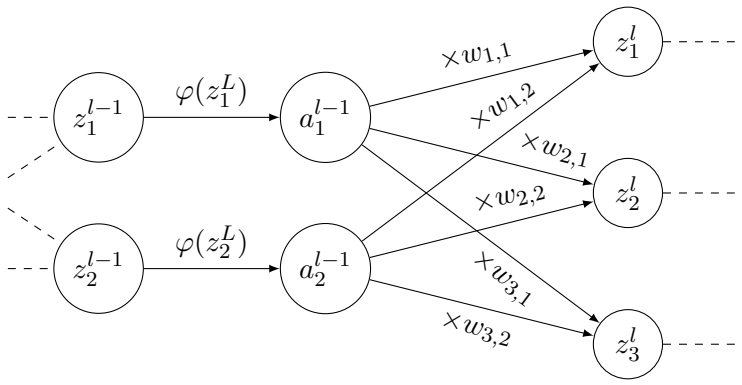


Abbildung 2.13: Computation Graph zur Berechnung von δ_j^{l-1}

Es gilt erneut die Gleichung (2.5) fuer die Berechnung des Fehlers δ_j^{l-1}.

$$\delta_j^{l-1} = \frac{\partial C}{\partial z_j^{l-1}} = \frac{\partial a_j^{l-1}}{\partial z_j^{l-1}} \cdot \frac{\partial C}{\partial a_j^{l-1}} \tag{2.5}$$

Zuerst einmal ist der erste Faktor wieder die Ableitung $\varphi'(z_j^{l-1})$ der Aktiverungsfunktion. Desweiteren

beeinflusst beim Uebergang der Schicht $(l-1)$ zur Schicht l eine Aktivierungen a_j^{l-1} alle gewichteten Summen z_k^l. Mit der Kettenregel folgt daher, dass die partielle Ableitung $\frac{\partial C}{\partial a_j^{l-1}}$ die Summe aller $\frac{\partial C}{\partial z_k^l} \cdot \frac{\partial z_k^l}{\partial a_j^{l-1}}$ sein muss. Wir erhalten Gleichung (2.7).

$$\delta_j^{l-1} = \varphi'(z_j^{l-1}) \cdot \sum_{k=1}^{|l|} \left(\frac{\partial C}{\partial z_k^l} \cdot \frac{\partial z_k^l}{\partial a_j^{l-1}} \right) \tag{2.7}$$

Um die gewichtete Summe z_k^l zu bilden, multipliziert man einfach die Aktivierung a_j^{l-1} der vorherigen Schicht mit den entsprechenden Gewichten $w_{k,j}^{l-1}$. Dadurch ist diese partielle Ableitung $\frac{\partial z_k^l}{\partial a_j^{l-1}}$ gerade das Gewicht selbst. Desweiteren ist $\frac{\partial C}{\partial z_k^l}$ per Definition der Fehler δ_k^l. Mit dieser Erkenntnis erhalten wir die zweite essentielle Gleichung (BP2) fuer die Rueckwaertsprogagierung.

$$\delta_j^{l-1} = \varphi'(z_j^{l-1}) \cdot \sum_{k=1}^{|l|} \left(\delta_k^l \cdot w_{k,j}^{l-1} \right) \tag{BP2}$$

Auch diese Gleichung haetten wir gerne in der Matrixschreibweise. Wir beginnen mit der Erweiterung auf alle gewichteten Summen.

$$\boldsymbol{\delta}^{l-1} = \boldsymbol{\varphi}'[\boldsymbol{z}^{l-1}] \odot \begin{pmatrix} \sum_{k=1}^{|l|} w_{k,1}^{l-1} \cdot \delta_k^l & \cdots & \sum_{k=1}^{|l|} w_{k,|l-1|}^{l-1} \cdot \delta_k^l \end{pmatrix}^{\mathsf{T}}$$

Der zweite Operant des Hadamard-Produkts ist hierbei gerade das Produkt der Matrixmultiplikation zwischen der transponierten Gewichtsmatrix $(\boldsymbol{W}^{l-1})^{\mathsf{T}}$ der Schicht $(l-1)$ und dem Fehlervektor $\boldsymbol{\delta}^l$ der Schicht l (ersichtlich in folgender Gleichung).

$$\begin{aligned}
\begin{pmatrix} \sum_{k=1}^{|l|} w_{k,1}^{l-1} \cdot \delta_k^l & \cdots & \sum_{k=1}^{|l|} w_{k,|l-1|} \cdot \delta_k^l \end{pmatrix}^{\mathsf{T}} &= \begin{pmatrix} w_{1,1}^{l-1} & w_{2,1}^{l-1} & \cdots & w_{|l|,1}^{l-1} \\ w_{1,2}^{l-1} & w_{2,2}^{l-1} & \cdots & w_{|l|,2}^{l-1} \\ \vdots & \vdots & \ddots & \vdots \\ w_{1,|l-1|}^{l-1} & w_{2,|l-1|}^{l-1} & \cdots & w_{|l|,|l-1|}^{l-1} \end{pmatrix} \begin{pmatrix} \delta_1^l & \cdots & \delta_{|l|}^l \end{pmatrix}^{\mathsf{T}} \\
&= \begin{pmatrix} w_{1,1}^{l-1} & w_{1,2}^{l-1} & \cdots & w_{1,|l-1|}^{l-1} \\ w_{2,1}^{l-1} & w_{2,2}^{l-1} & \cdots & w_{2,|l-1|}^{l-1} \\ \vdots & \vdots & \ddots & \vdots \\ w_{|l|,1}^{l-1} & w_{|l|,2}^{l-1} & \cdots & w_{|l|,|l-1|}^{l-1} \end{pmatrix}^{\mathsf{T}} \boldsymbol{\delta}^l = (\boldsymbol{W}^{l-1})^{\mathsf{T}} \boldsymbol{\delta}^l
\end{aligned}$$

Nun haben wir unsere rekursive Fehlerdefinition in Matrixschreibweise und somit die kompakte Version (BP2a) der zweiten wichtigen Formel (BP2).

$$\boldsymbol{\delta}^{l-1} = ((\boldsymbol{W}^{l-1})^{\mathsf{T}} \boldsymbol{\delta}^l) \odot \boldsymbol{\varphi}'[\boldsymbol{z}^{l-1}] \tag{BP2a}$$

In einem letzten Schritt muessen wir jetzt noch Formeln herleiten, mit welchen man anhand des Fehlers δ_j^l die partiellen Ableitungen der Gewichte und der Neigungen berechnen kann.

Eine Neigung b_j^l ist Funktionsbestandteil der entsprechenden gewichteten Summe z_j^{l+1}. Somit gilt fuer die Neigung Formel (2.8).

$$\frac{\partial C}{\partial b_j^l} = \frac{\partial C}{\partial z_j^{l+1}} \cdot \frac{\partial z_j^{l+1}}{\partial b_k^l} \tag{2.8}$$

Der erste Term ist hierbei per Definition unser Fehler δ_j^{l+1} und der zweite Term laesst sich zu 1 evaluieren, da die Summe z_k^{l+1} nur aus b_k^l besteht und aus Summanden, welche fuer die partielle Ableitung als konstant gelten. Somit ist die Ableitung der Neigung gerade unser Fehler und wir erhalten die dritte (BP3) von vier essentielen Gleichungen.

$$\frac{\partial C}{\partial b_j^l} = \delta_j^{l+1} \tag{BP3}$$

Somit ist der Kostengradient bezueglich der Neigung gerade der Fehlervektor (siehe Gl. (BP3a)).

$$\boldsymbol{\nabla}_{\boldsymbol{b}^l} C = \boldsymbol{\delta}^{l+1} \tag{BP3a}$$

Ein Gewicht $w_{j,k}^l$ ist ebenfalls ein Funktionsbestandteil der assozierten gewichteten Summe z_j^{l+1}. Dadurch gilt fuer die partiellen Ableitungen der Kosten nach dem Gewicht Gleichung (2.9).

$$\frac{\partial C}{\partial w_{j,k}^l} = \frac{\partial C}{\partial z_j^{l+1}} \cdot \frac{\partial z_j^{l+1}}{\partial w_{j,k}^l} \tag{2.9}$$

Dabei laesst sich der erste Teil wieder zum Fehler δ_j^{l+1} evaluieren. Die zweite partielle Ableitung ist gerade die Aktivierung a_k^l, da sich die gewichtete Summe aus der Multiplikation des Gewichtes mit der Aktivierung ergibt. Somit erhalten wir die letzte der vier essentiellen Gleichungen (BP4).

$$\frac{\partial C}{\partial w_{j,k}^l} = \delta_j^{l+1} \cdot a_k^l \tag{BP4}$$

Die Matrix-version davon ist Gleichung (BP4a).

$$\boldsymbol{\nabla}_{\boldsymbol{W}^l} C = \boldsymbol{\delta}^{l+1} (\boldsymbol{a}^l)^\mathsf{T} \tag{BP4a}$$

ALLES ZUSAMMENFASSEN
(Nielsen, *Neural Networks and Deep Learning*)

2.7 Universal Approximation Theorem?

Es stellt sich nun die Frage, was ein Neuronales Netz alles erlernen kann. Diese Frage kann man mit dem **Universal Approximation Theorem** (UAT) beantworten. Dieser Satz ist ein mathematischer Beweis dafuer, dass ein KNN grundsaetzlich in der LAge ist jede kontinuerliche Funktion beliebig gut zu approximieren. Er wurde von Kurt Hornik in 1991 beweisen.
Etwas genauer besagt der UAT, dass ein Feed-Forward Netzwerk mit einer einzigen Zwischenschicht, welche eine endliche Anzahl Neuron hat, jede Kontinuierliche Funktion approximieren kann. Voraussetzung dafuer ist, dass es sich bei den Neuronen und Nicht-lineare-Neuronen handelt. Dies ist nur eine theoretische Ausage ueber das Lernpotenzial eines KNNs. Jedoch macht es keinerlei Aussage darueber, ob ein KNN wirklich die Funtkionen erlernen wird.
IAN GOODFELLOW: A feedworward network with a single layer is sufficient to represent any function, but the layer may be infeasibly large and may fail to lean and generalize correctly.
Die meisten Klassen von Probleme koennen einfach als eine Funktion formuliert werden. Somit bedeutet der UAT, dass ein KNN theoretisch jedes Problem loesen kann.
Der eigentliche Beweis ist mathematisch ziemlich anspruchsvoll, deshalb werden wir diesen nicht behandeln.
AUF ANDERE RESOURCE (QR-CODE) VERWEISEN FUER ILLUSTRIEVEN BEWEIS ODER ANHANG VON MIR GESCHRIEBEN
(Mehrere, *Universal approximation theorem — Wikipedia, The Free Encyclopedia*)

Kapitel 3

Convolutional Neural Networks

Viele Anwendungen von Machine Learning sind verbunden mit Bild- oder Audioverarbeitung, wie z.B Bildklassifizierung, Gesichtserkennung oder Spracherkennung. Vorallem aber fuer hochaufloesende Bilder sind die KNNs, die wir soeben kennengelernt haben, nicht geeignet. Sie sind zum Teil gar nicht in der Lage eine Korrelation zwischen den Inputs und Outputs zu erlernen. Um diesen Umstand zu erklaeren, wird nun ein kleines Beispielmodell erlauetert:

Es soll ein KNN designed werden, welches eine Photographie klassifizieren soll, ob darauf ein Hund sichtbar ist oder nicht. Wir nehmen fuer dieses Gedankenexperiment bereits ein relativ niedrig aufgeloestes Bild mit 256×256 Pixel, dies entspricht weniger als 0.07 Megapixel (ein iPhone XS hat eine Kamera mit 12 Megapixel). Um die verschiedenen Farben zu codieren besitzt jeder Pixel drei Komponten: R, G und B. Somit hat dieses Bild insgesamt $256 \times 256 \times 3 = 196'608$ Komponenten. Jede Komponente ist ein Feature welches das KNN zu verrechnen hat. Somit bestuende die erste Schicht des Netzwerkes aus fast $200'000$ Neuronen. Um diese Schicht nun mit seiner Nachbarsschicht, welche gleiche Dimensionen besitzt, zu verbinden, braucht es $196'608 \times 196'608 = 38'654'705'664$ Verbindungen und damit gleich so viele Gewichte! Fuer ein Netwerk ohne eine einzige Zwischenschichten gaebe es also ueber 38 Milliarden Modellparameter zu erlernen! Dass dies nicht realistisch ist, sollte auf der Hand liegen.

Nicht nur die Anzahl Modellparameter sind ein Problem fuer KNNs in der Bildverarbeitung, sondern es bestehen noch weiter Probleme. Trotzdem sollte nun klar sein, dass eine andere Modellarchitektur noetig ist, um Machine Learning auf Bilder anzuwenden. Fuer genau solche Anwendungen wurde eine modifizierte Version eines KNNs entwickelt: das **Convolutional Neural Network** (CNN). Im Allgemeinen sind CNNs immer dann geeignet, wenn es Daten zu verarbeiten gibt, welche eine rasterartige Form haben, wie eben z.B Bilder. Diese Art von Netzwerk macht Gebrauch von Konzepten aus der klassischen Bildverarbeitung, wie sie mit beispielsweise Photoshop gemacht werden kann. Wie beim Perzeptron und bei den klassischen KNNs auch, ist die Architektur biologisch inspiriert. Der folgende Abschnitt wird die Funktionsweise eines solchen CNNs erklaeren.

(Goodfellow, Bengio und Courville, *Deep Learning*)
(deeplearning.ai, *Convolutional Neural Networks (Course 4 of the Deep Learning Specialization)*)
(Mehrere, *Convolutional neural network — Wikipedia, The Free Encyclopedia*)

3.1 Bilder als Tensoren

CNNs operieren an Bildern. Diese Bilder stellen den Input fuer die Modelle dar. Um mit diesen Bildern rechnen zu koennen, ist es sinnvoll, sie als sogenannte **Tensoren** vom Rang 3 zu untersuchen, anstatt sie als Anordnungen von Pixeln zu betrachten.
Um zu verstehen, was ein Tensor dritten Ranges ist, muessen wir zuerst verstehen, was ein Tensor im Allgemeinen ist.

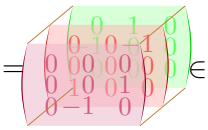
Tensor

Ein Tensor $\mathbf{T}$ ist eine Verallgemeinerung von Skalaren, Vektoren und Matrizen auf n Dimensionen. Es handelt sich wie bei Matrizen um eine Zahlenanordnung. Dabei wird die Anzahl Dimensionen, innerhalb welchen die Zahlen liegen als Rang oder Stufe n des Tensors bezeichnet. Vorstellen kann man sich einen Tensor als ein Hyperrechteck mit n Dimensionen, innerhalb dessen die Zahlen in einem Raster angeordnet sind. Diese Zahlen sind die Elemente des Tensors. Ein Tensor nullten Ranges ist ein Skalar, einer erster Stufe ein Vektor und einer mit Rang 2 ist eine normale Matrix.

$$1 \in \mathbb{R} \text{ (Skalar)} \qquad \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} \in \mathbb{R}^3 \text{ (Vektor)} \qquad \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \in \mathbb{R}^{2\times 3} \text{ (Matrix)}$$

Tensor 3. Ranges

Ein Tensor $\mathbf{T} \in \mathbb{R}^{h\times w\times d}$ mit Rang 3 ist eine 3-dimensionale Zahlenanordnung. Man kann sich diesen Tensor einfach als eine 3D-Matrix vorstellen; ein Volumen innerhalb dessen Zahlen in einem Raster angeordet sind. Analog zum Volumen bezeichnet man die Form des Tensors mit Hoehe, Breite und Tiefe.
Ein Tensor von der Form $\mathbb{R}^{3\times 3\times 3}$ koennte folgendermassen aussehen:

$\mathbf{T} =$  $\in \mathbb{R}^{3\times 2\times 3}$

Ein Bild kann somit einfach als Tensor dritten Ranges $\mathbf{B} \in \mathbb{R}^{h\times w\times c}$, der Form (Bildhoehe $\times$ Bildbreite $\times$ Anzahl Farbkomponenten) betrachtet werden. Elemente der Matrix nehmen dann einfach die Werte der Pixelkomponenten an. Ein schwarzweiss Bild hat nur eine Komponente, welche die Helligkeit angibt. Somit waere es eine normale 2D-Matrix $\boldsymbol{B} \in \mathbb{R}^{h\times w}$ (siehe Abb. (3.1)).

Abbildung 3.1: Beispiel einer Bildmatrix

(deeplearning.ai, *Convolutional Neural Networks (Course 4 of the Deep Learning Specialization)*)
(Mehrere, *Tensor — Wikipedia, Die freie Enyklopädie*)

3.2 Topologie von CNNs

Ein CNN besteht, wie auch ein KNN, aus Schichten. Jede dieser Schichten erhaelt als Input ein Bild und hat auch wieder eines als Output.
Ein wichtiger Unterschied eines CNNs ist, dass es aus unterschiedlichen Typen von Schichten besteht. Man unterscheiden zwischen vier Arten von Schichten:

- **Fully-connected-Schichten**,
- **Convolutional-Schichten**,
- **Pooling-Schichten**, und
- **Upsampling-Schichten**.

Die Fully-Connected-Schicht ist altbekannt und ist einfach die klassische Schicht eines KNNs, bestehend aus Neuronen. Aus diese werden wir nicht weiter eingehen, da wir sie schon im vorherigen Kapitel kennengelernt haben.
Die Convolutional-Schicht ist eine neuartige Schicht, welche es im KNN nicht gibt. Sie ist die Schicht, welche fuer das Training relevant ist, da sie die Modellparameter beinhaltet. Sie extrahiert die relevanten Features aus den Inputbildern und lernt so die Bilder zu verstehen.
Die Pooling-Schicht, wie auch die Upsampling-Schicht, beinhaltet keine Modellparameter und sind deshalb fuer das Training nicht direkt relevant. Diese Schichten werden lediglich gebraucht, um die verarbeiteten Bilder neu zu skalieren. Dies ist sinnvoll, da durch die Extraktion gewisser Features, die restlichen Features wegfallen. Somit muss weniger Information pro Bild gehalten werden und die Bilder sollten schrumpfen, um Overfitting vorzubeugen. Die Pooling- Schichten reduzieren die Bilder und die Upsampling-Schichten erweitern die Bilder um mehr Pixel. Dies wird in Sektion (4) fuer die Autoencoder hilfreich werden.
Grundsaetzlich folgt auf eine Convolutional-Schicht immer entweder eine Pooling- oder eine Upsampling-Schicht. Somit bilden sie gewissermassen eine Einheit.

In Abbildung (3.2) ist ein Schema eines CNNs abbgebildet.

Abbildung 3.2: Schichtung eines CNNs

(Goodfellow, Bengio und Courville, *Deep Learning*)
(deeplearning.ai, *Convolutional Neural Networks (Course 4 of the Deep Learning Specialization)*)
(Mehrere, *Convolutional neural network — Wikipedia, The Free Encyclopedia*)

3.3 Convolutional-Schichten und Filter

Zuerst betrachten wir die Convolutional Schicht. Diese Schicht macht ausgiebig Gebrauch von sogenannten Filtern. Diese werden in diesem Abschnitt behandelt.

3.3.1 Filter in der Bildverarbeitung

Filter (auch Kerne) sind in der Bildverarbeitung sehr verbreitet. Jeder kennt sie entweder von Photoshop, von Instagram oder sonstigen Bildbearbeitungsprogrammen. Auch CNNs machen Gebrauch von solchen Filtern, in diesem Fall um die Features eines Bildes zu erlernen.

Abbildung 3.3: der Instagramfilter: Blablabla

Ein Filter ist einfach eine Region, deutlich kleiner als das verarbeitete Bild $\mathbf{B}$, welche ueber alle Pixel wandert, diese manipuliert und so wieder ein neues Bild $\tilde{\mathbf{B}}$ liefert. Mathematisch gesehen handelt es sich bei einem Filter einfach um einen Tensor, den sogennanten **Filtertensor F** oder Faltungstensor (siehe Abb. (3.4)). Solche Filter sind immer quadratisch, wobei ihre Zeilen- und Spaltenlaenge mit f bezeichnet wird. Dabei ist f immer eine ungerade Zahl, damit ein Element, der sogennante **Zentralelement** (engl.: Center element), bezeichnet mit $(\mathbf{F})_C$, immer im Zentrum des Filters liegt.

Abbildung 3.4: eine 2D-Filtermatrix

Das Verhalten des Filters wird durch seine Matrixeintraege bestimmt. Die Elemente koennen also so gewaehlt werden, dass wenn der Filter auf ein Bild angewandt wird, er bestimmte Features aus dem Bild hervorhebt. Solche Features koennten zum Beispiel Raender oder Kanten sein, welche betont werden.

Ein gutes Beispiel ist der Sobel-Filter. Er ist ein typischer 2D-Kantendetektions-Filter mit einer Groesse von $f = 3$. Wir betrachten nun den Sobelfilter $\boldsymbol{G}_x$, welcher horizontale Kanten akzentuiert. Die Eintraege dieses Filters lauten wie folgt:

$$\boldsymbol{G}_x = \begin{pmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{pmatrix}$$

Angewandt auf ein schwarz-weisses Bild, erhaelt man ein neues Bild, bei welchem alle erkannten Kanten weiss eingefaerbt sind und die restlichen Bereiche schwarz sind. Der Filter erkennt dabei jede Stelle als Kante, welche zwei Regionen mit genug grossem Kontrast voneinander trennt. In Abbildung (3.5) wurde der horizontale Sobelfilter auf ein Beispielbild angewandt. Links ist das Ursprungsbild und rechts ist das prozessierte Bild.

Abbildung 3.5: Sobel-Filter angewandt auf Beispielsbild

(Mehrere, *Sobel operator — Wikipedia, The Free Encyclopedia*)
(deeplearning.ai, *Convolutional Neural Networks (Course 4 of the Deep Learning Specialization)*)
(Mehrere, *Kernel (image processing) — Wikipedia, The Free Encyclopedia*)

3.3.2 Filter in CNNs

Aus didaktischen Gruenden betrachten wir nun zuerst nur 2D-Filter, welche sich fuer graustufige Bilder $\boldsymbol{B} \in \mathbb{R}^{h \times w}$ eignen. Somit ist der hierfuer geeignete Filter einfach eine 2D-Matrix, bezeichnet mit $\boldsymbol{F} \in \mathbb{R}^{f \times f}$. Man bedenke bei jeder Filtermatrix einfach immer, dass es sich eigentlich im allgemeinen Fall um einen 3D-Tensor handelt.

Wie wir vorhin erfahren haben, koennen Filter genutzt werden um gewisse Features eines Bildes hervorzuheben und andere Features zu ignorieren. Dabei bestimmen die Filtereintraege, welche Features extrahiert werden. Somit besteht die Aufgabe darin, die richtigen Filtereintraege zu bestimmen, damit die gewuenschten Features erkannt werden und auf die gewuenschten Outputs abgebildet werden. Naheliegend sollte jetzt der Gedanke sein, die Filtereintreage als die Modellparameter eines CNNs zu definieren. Mithilfe des Gradietenverfahrens sollten erneut diese Parameter erlernt werden.
Da die Filter die gleiche Funktion haben, wie die Gewichte in einem KNN, bezeichnet man die Filter in einem CNN mit $\boldsymbol{W}$. Die Groesse des Filters stellt dabei einen Hyperparameter dar.

$$\boldsymbol{W} = \begin{pmatrix} w_{1,1} & \cdots & w_{1,f} \\ \vdots & \ddots & \vdots \\ w_{f,1} & \cdots & w_{f,f} \end{pmatrix}$$

3.3.3 Filteroperation intutitiv

Wie wendet man nun einen solchen Filter auf ein Bild an? Um das Prozedere einer Filteroperation leichter verstaendlich zu machen, wird es erneut fuer ein graustufiges Bild $\boldsymbol{B} \in \mathbb{R}^{h \times w}$ erlauetert. Spaeter werden auch noch farbige Bilder erklaert.

Bei einer Filteroperation, wendet man einen Filter $\mathbf{W}$ auf einen Bildtensor $\mathbf{B}$ an und erhaelt so ein neues Bild $\tilde{\mathbf{B}}$. Man schreibt dafuer: $\tilde{\mathbf{B}} = \mathbf{W} * \mathbf{B}$.

Um nun diese Operation zu erklaeren, verwenden wir als Beispiel eine 2D-Filtermatrix $\boldsymbol{W} \in \mathbb{R}^{3 \times 3}$, bei welcher $f = 3$ gewaehlt wurde. Wie bereits oben erwaehnt, wandert der Filter ueber das Bild. Dabei befindet sich der Filter immer ueber einer Region des Bilder, welche gleich gross ist, wie der Filter selber (in diesem Fall (3×3)), sodass den Filtermatrixeintraegen immer eindeutig Bildmatrixeintraege entsprechen. Diese Region des Bildes bezeichnet man als das **rezeptives Feld** des Filters (siehe Abb.

(3.6)). Das Rezeptive Feld, bezeichnet mit $\hat{\boldsymbol{B}} \in \mathbb{R}^{3\times 3}$ ist also einfach eine Untermatrix der Obermatrix $\boldsymbol{B}$.

Jedem Element des Filters entspricht ein Element des rezeptiven Feldes. Dem Zentralelement $(\boldsymbol{W})_c$ entspricht dabei gerade dem sogenannten Quellenpixel (engl.: source pixel) $(\mathbf{B})_s$ des Bildes. Ihn verwenden wir fuer eine Namenskonvention fuer das rezeptive Feld. Wir bezeichnen das rezeptive Feld mit $\hat{\boldsymbol{B}}^{(y,x)}$, wobei die hochgestellten Indizes (y, x) die Position des Quellenpixel $(\boldsymbol{B})_S$ im Bild $\boldsymbol{B}$ angibt.

Abbildung 3.6: ein Filter und sein rezeptives Feld

Der Filter beginnt nun oben links ueber das Bild zu wandern. Somit hat der Filter zu beginn das rezeptive Feld $\hat{\boldsymbol{B}}^{(2,2)}$, denn der Filter darf nicht ueber die Raender des Bildes hinausragen. Nun werden die Elemente der Filters mit den Elementen des rezeptiven Feldes verrechnet, welche die gleichen Dimensionen besitzen. Dabei wird einfach das Hadamard-Produkt (das elementweise Produkt) des beiden Matrizen miteinander gebildet. Nun erhalten wir eine neue Matrix $\boldsymbol{W} \odot \hat{\boldsymbol{B}}^{(2,2)} = \boldsymbol{P} \in \mathbb{R}^{3\times 3}$. Nun werden noch alle Elemente der neuen Matrix $\boldsymbol{P}$ aufsummiert. Diese Zahl ist dann der Grauwert des ersten Pixels $(\tilde{\boldsymbol{B}})_{1,1}$ des bearbeiteten Bildes (siehe Gl. (3.1)). In diesem Sinne stellt der Filter eine Gewichtung der Nachbarspixel dar, welche bestimmt, wie die neuen Pixel aussehen.

$$(\tilde{\boldsymbol{B}})_{1,1} = \sum_{y=1}^{3}\sum_{x=1}^{3}(\boldsymbol{P})_{y,x} \tag{3.1}$$

Nun wird der Filter um ein Element nach rechts verschoben und die gleiche Prozedur angewandt, um den zweiten Pixel zu berechnen. Dies wird durchgezogen, bis eine ganze Zeile der Bildmatrix durchstreift wurde. Danach wird der Filter wieder nach ganz links verschoben und er bewegt sich ein Element nach unten. Das geschieht, bis das ganze Bild verrechnet wurde. Dabei werden die Elemente des urspruenglichen Bildes durchaus mehrmals verrechnet, da es zu einer Ueberlappung des vorherigen Filters und des verschobenen Filters kommt.

Es ist zu bemerken, dass das neue Bild $\tilde{\boldsymbol{B}} \in \mathbb{R}^{(h-2)\times(w-2)}$ nicht mehr die gleichen Masse hat, wie das Ursprungsbild $\boldsymbol{B} \in \mathbb{R}^{h\times w}$. Das liegt daran, dass pro Lage des Filters jeweils nur ein Pixel des neuen Bildes berechnet wird. Man kann sich vorstellen, dass an der Position des Quellenpixel jeweils ein neuer Pixel entsteht. Dabei muss der Filter immer eine vollstaendige Region als rezeptives Feld haben. Dadurch kommt das Zentralelement nie auf die Raender des Bildes zu liegen. Somit fallen diese weg.

(deeplearning.ai, *Convolutional Neural Networks (Course 4 of the Deep Learning Specialization)*)
(Mehrere, *Convolution— Wikipedia, The Free Encyclopedia*)

3.3.4 Filteroperationen als diskrete Faltungen

Eigentlich handelt es sich bei einer Anwendung eines Filters auf ein Bild um eine spezifische mathematische Operation: eine **diskrete Faltung** (engl.: **convolution**). [1] Daher ruehrt auch der Name des Convolutional Neural Networks. Wir werden die Faltung als mathematische Operation nicht im vollen Umfang betrachten, da sie relativ kompliziert ist. Wir werden uns lediglich auf die Bedeutung der Faltung fuer Filter beschraenken.

[1] Streng genommen wird in CNNs nicht eine diskrete Faltung durchgefuehrt, sondern eine sogenannte **Kreuzkorrelation**. Der Unterschied dabei ist, dass bei einer Kreuzkorrelation die Faltungsmatrix nicht zuerst horizontal, wie auch vertikal gespiegelt wird, im Gegensatz zu der eigentlichen Faltung. Jedoch verwendet man begrifflich das Wort Faltung fuer beide Operationen. Im Verlauf dieser Arbeit wird mit dem Wort Faltung immer die Kreuzkorrelation gemeint sein.

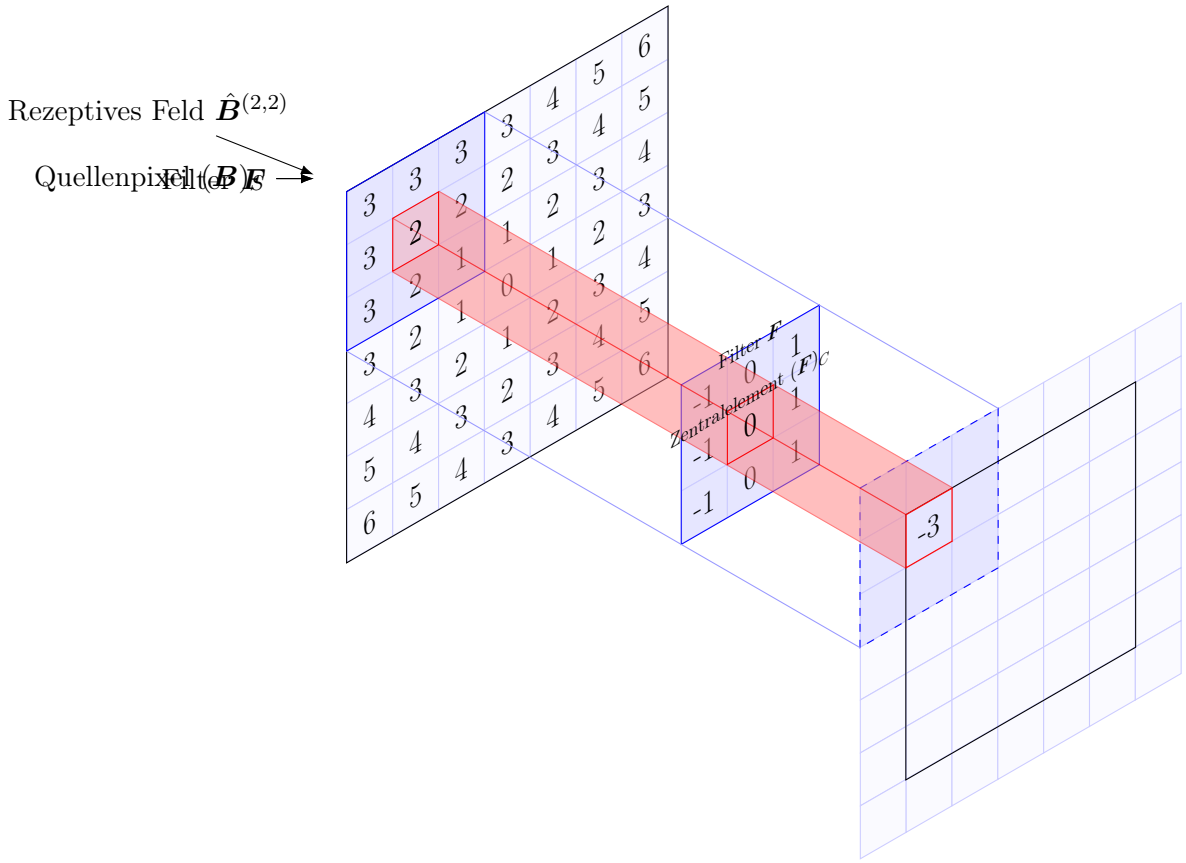


Abbildung 3.7: Schema, wie ein Filter ueber ein Bild laueft

Diskrete Faltung

Definintion Faltung

$$(f * g)(x) = \int_{-\infty}^{\infty} f(\tau)g(x-\tau)\mathrm{d}\tau \tag{3.2}$$

Mithilfe der Faltungsoperation koennen wir die Schritte welche in Sektion (3.3.3) erklaert wurden zusammenfassen. Bei der Filteroperation handelt es sich lediglich um eine diskrete Faltung des Filtertensors $\mathbf{W}$ ueber den Bildtensor $\mathbf{B}$. Somit koennen wir folgende Formel verwenden, um einen Pixel $\tilde{\boldsymbol{B}}(x,y)$ des neuen Bildes zu berechnen.

$$(\tilde{\boldsymbol{B}})_{y,x} = (\boldsymbol{W} * \boldsymbol{B})_{y,x} = \sum_{v=1}^{f}\sum_{u=1}^{f}(\boldsymbol{W})_{v,u}(\boldsymbol{B})_{y+v-1,x+u-1} \tag{3.3}$$

Faltung Beispielrechnung

$$\boldsymbol{B} = \begin{pmatrix} 10 & 10 & 10 & 0 & 0 & 0 \\ 10 & 10 & 10 & 0 & 0 & 0 \\ 10 & 10 & 10 & 0 & 0 & 0 \\ 10 & 10 & 10 & 0 & 0 & 0 \\ 10 & 10 & 10 & 0 & 0 & 0 \\ 10 & 10 & 10 & 0 & 0 & 0 \end{pmatrix} \text{ und } \boldsymbol{W} = \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}$$

Fuer die Berechnung des ersten Pixels $(\tilde{\boldsymbol{B}})_{1,1}$ fuehren wir folgende Rechnung aus.

$$\begin{aligned} (\tilde{\boldsymbol{B}})_{1,1} = (\boldsymbol{W} * \boldsymbol{B})_{1,1} &= \sum_{v=1}^{f} \sum_{u=1}^{f} (\boldsymbol{W})_{u,v} (\boldsymbol{B})_{u,v} \\ &= 10 \cdot 1 + 10 \cdot 0 + 10 \cdot -1 + 10 \cdot 1 + 10 \cdot 0 + 10 \cdot -1 + 10 \cdot 1 + 10 \cdot 0 + 10 \cdot -1 \\ &= 0 \end{aligned}$$

Das Gesamtergebniss lautet dann:

$$\tilde{\boldsymbol{B}} = \boldsymbol{W} * \boldsymbol{B} = \begin{pmatrix} 0 & 30 & 30 & 0 \\ 0 & 30 & 30 & 0 \\ 0 & 30 & 30 & 0 \\ 0 & 30 & 30 & 0 \end{pmatrix}$$

Fuer farbige Bilder ist das Vorgehen praktisch das Selbe. Es werden nun lediglich 3D-Tensoren anstatt 2D-Tensoren verwendet. Wir betrachten den allgemeinen Bildtensor $\mathbf{B}_{h \times w \times c}$, wobei c die Anzahl Farbkomponenten (engl.: channels) ist. Nun benutzt man einen Filter $\mathbf{W}_{f \times f \times c}$ mit beliebiger Groesse f aber mit der gleicher Tiefe c wie das Ursprungsbild $\mathbf{B}$. Dadurch muss der Filter nicht entlang der Tiefe des Bildes wandern, da er gerade gleich tief ist wie das Bild. Das hat zur Folge, dass das verarbeitete Bild $\tilde{\boldsymbol{B}}$ immer eine 2D-Matrix ist. Die allgemeine Filtergleichung lautet:

$$(\tilde{\boldsymbol{B}})_{y,x} = (\mathbf{W} * \mathbf{B})_{y,x} = \sum_{v=1}^{f} \sum_{u=1}^{f} \sum_{w=1}^{c} (\mathbf{W})_{v,u,w} (\mathbf{B})_{x+u-1,y+v-1,w} \tag{FO}$$

(Goodfellow, Bengio und Courville, *Deep Learning*)
(deeplearning.ai, *Convolutional Neural Networks (Course 4 of the Deep Learning Specialization)*)
(Mehrere, *Convolutional neural network — Wikipedia, The Free Encyclopedia*)

3.3.5 Mehrere Filter

Eine Filteroperation ist nicht auf einen einzigen Filter beschraenkt. Es koennen mehrere Filter auf das gleiche Ausgangsbild angewandt werden und zusammen ein Endbild erzeugen.

Die Anzahl Filter bezeichnen wir mit c. Nun macht man pro Filter $\mathbf{W}_i$ eine Faltung ueber das Ursprungsbild $\mathbf{W}$, wobei jede Faltung einen neue Matrix $\tilde{\boldsymbol{B}}_i = \mathbf{W}_i * \mathbf{B}$ liefert, wobei i der Index des Filters ist. All diese gefalteten Bilder $\tilde{\boldsymbol{B}}_i$ sind zweidimensionale Matrizen, unabhaengig davon, wieviele Komponenten das Ursprungsbild hatte. Aus diesem Grund koennen die einzelnen Matrizen $\tilde{\boldsymbol{B}}_i$ aufeinander gelegt werden und somit einen grossen 3D-Endtensor $\tilde{\mathbf{B}}$ bilden. Hierfuer muessen alle Filter $\mathbf{W}_i$ gleich gross sein. Die Tiefe des neuen Bildes $\tilde{\mathbf{B}}$ ist jetzt gerade die Anzahl Filter c. Wir haben bereits

Abbildung 3.8: aufeinander gelegte gefaltene Bilder

frueher immer die Tiefe des Bildes (bzw. die Anzahl Farbkomponenten) mit c bezeichnet. Somit ist c sowohl die Anzahl Filter einer Schicht, wie auch die Tiefe des verarbeiteten Bildes.

(Goodfellow, Bengio und Courville, *Deep Learning*)
(deeplearning.ai, *Convolutional Neural Networks (Course 4 of the Deep Learning Specialization)*)

3.3.6 Padding

Wie bereits erwaehnt schrumpfen die Bilder (an den Raendern), wenn man einen Filter auf sie anwendet. Zur Erinnerung: Dies liegt daran, dass pro Lage des Zentralelements nur ein Pixel des neuen Bildes entsteht und das Zentralelement nunmal nicht auf allen Ursprungspixeln zu liegen kommt. Die Raender fallen so weg. Diese Groesse des Wegfalls haengt von der Filtergroesse f ab. Hierbei gilt folgende Formel fuer die neue Groesse n_1 des Bildes, anhand der alten Groesse n_0. n bezeichnet eine der zwei Seitenlaengen.

$$n_1 = n_0 - f + 1 \tag{3.4}$$

Das Problem hierbei ist, dass nach einigen Faltungen das Bild extrem geschrumpft ist und im Grenzfall kleiner als der Filter wird. Das darf natuerlich nicht passieren. Ausserdem kommt es dabei zu einem relativen Informationverlust an den Raender, da es im Innern des Bildes zu viel Ueberlappungen der Filterlagen kommt und es bei den Raendern nur seltener zur Ueberlappung kommt.

Diese Probleme werden mit sogenanntem **Padding** behoben. Padding ist ein Schritt, welcher vor der eigentlichen Faltung stattfindet. Dabei werden einfach zusaetzliche Raender (Zeilen und Spalten) an das Ursprungsbild angebracht. Der Tensor wird der Laenge und der Breite nach (nicht der Tiefe entlang!) an den Enden erweitert. Die neuen Elemente werden dabei auf den Wert 0 gesetzt.

Abbildung 3.9: Padding

Das Padding p ist eine Zahl, welche angibt wieviele Elemente an allen Raendern hinzugefuegt werden. Padding $p = 1$ bedeutet, dass an allen Kanten jeweils eine Reihe bzw. Spalte hinzugefuegt wird. Begrifflich unterscheidet man zwischen zwei Arten von Padding:

- **Valid-Padding**: Es werden keine zuesaezlichen Elemente angebracht. p ist also 0.
- **Same-Padding**: Es werden so viele Reihen und Spalten angebracht, dass die Groesse des Bildes nach der Faltung unveraendert bleibt.

Um Same-Padding durchzufuehren, muss man p so waehlen, dass es den Wegfall durch den Filter gerade kompensiert. Da durch das Padding das Bild der Laenge und Breite nach jeweils um zwei p laenger wird, erhaelt man den Ausdruck $n_1 = n_0 - f + 1 + 2p$. Wenn man diese Formel jetzt nach p aufloest, erhaelt man folgende Formel fuer das Waehlen des Same-Paddings p.

$$p = \frac{f-1}{2} \tag{3.5}$$

(deeplearning.ai, *Convolutional Neural Networks (Course 4 of the Deep Learning Specialization)*)

3.3.7 Stride

Bis jetzt haben wir bei den Filterfaltungen, den Filter pro Anwendung immer nur um einen Pixel verschoben. Das muss aber nicht so sein. Man kann den Filter auch mit einer anderen Schrittgroesse verschieben. Diese Eigenschaft bezeichnet man als **Stride** s. Falls $s = 2$ gewaehlt wird, bedeutet das, dass der Filter sich pro Matrixmultiplikation um zwei Elemente bewegt. Somit wurde eine Position uebersprungen. Zur Folge hat dies, dass das neue Bild deutlich kleiner geworden ist, denn das Zentralelement ueberspringt somit auch diese Felder und bildet so deutlich weniger Pixel.

Folgende Formel beschreibt die Dimensionen des neuen Bildes unter Beruecksichtigung der Filtergroesse f, dem Padding p und dem Stride s.

$$n_1 = \frac{n_0 + 2p - f}{s} + 1 \tag{3.6}$$

(deeplearning.ai, *Convolutional Neural Networks (Course 4 of the Deep Learning Specialization)*)

3.3.8 Vorzuege von Filtern

Es stellt sich nun die Frage, weshalb sich Filter fuer Maschinelles Lernen mit Bildern eignen. Wie bereits erwaehnt besteht die Aufgabe von Filtern darin, bestimmte Features eines Bildes hervorzuheben und die Restlichen auszublenden. Genau die gleiche Aufgabe erfuellen die Neuronen in einem KNN. Auch sie sollen Features der Inputdaten erlernen, wobei gewisse Neuronen auf gewisse Features reagieren. Weshalb verwendet man also nicht einfach KNNs? (abgesehen von dem Problem mit der Ueberzahl an Modellparametern, siehe Sektion 3)

Man muss erkennen, dass Bilddaten sich deutlich von sonstigen Daten unterscheiden. Bildfeatures bzw. Pixel sind nur im Kontext von ihren Nachbarn relevant. Denn ein Pixel ist erst dann eine Kante oder eine Ecke, wenn er zwei verschiedene Farbregionen voneinander trennt. Oder ein Gegenstand wird erst durch eine ganzen Haufen von Pixeln und deren relative Position zueinander ausgezeichnet. Man bezeichnet diesen Umstand als **lokalisierte Features**.

Desweiteren sind Bildausschnitte nicht immer gleich orientiert. Wenn man ein Gesicht auf einem Bild erkennen moechte, sollte es aber keine Rolle spielen, wo auf dem Bild es sich befindet, welche Groesse es hat und was fuer eine Lage es hat. Um diese Eigenschaften irrelevant fuer das Modell zu machen, muss dieses gewisse **Invarianzen** erfuellen.

Die Eigenschaft, welche CNNs geeignet macht fuer lokalisierte Features und solche Invarianzen, bezeichnet man als **Parameter-Sharing**. Es bezeichnet den Umstand, dass auf mehrere oder alle Features die gleichen Modellparameter wirken. Bei CNNs wird dies durch die Bewegung des Filters bzw. seiner Eintraege ueber (fast) alle Pixel bewerkstelligt. Somit ist es egal, wo und wie sich die lokalisierten Features befinden. Dies fuehrt dann eben zu den gewuenschten Invarianzen: Translations-, Rotations- und Helligkeitsinvarianz.
Eine weiterer Vorteil durch dieses Parameter-Sharing ist, dass die Inputbilder beliebige Dimensionen besitzen koennen, da die Filter ihre Bewegung lediglich an die Groesse des Bildes anpassen muessen.
(deeplearning.ai, *Convolutional Neural Networks (Course 4 of the Deep Learning Specialization)*)

3.3.9 Die Convolutional-Schicht

Nun moechten wir alles zusammentragen, was wir ueber Filter gelernt haben, um nun die Convolutional-Schicht zu definieren.

Beginnen tut eine Convolutional-Schicht l mit seinem Input, also den Aktivierungen $\mathbf{A}^{l-1} \in \mathbb{R}^{h^l \times w^l \times c^l}$, welche die vorherige Schicht $(l-1)$ produziert hat, oder falls es die erste Schicht ist, erhaelt sie den Input $\mathbf{X}$ des Netzes, welchen man auch mit $\mathbf{A}^0$ bezeichnet.
Die Schicht besitzt c^l Varianten an Filtern $\mathbf{W}_i^l \in \mathbb{R}^{f^l \times f^l \times c^{l-1}}$. Diese Filter haben alle die gleichen Eigenschaften, bezueglich: der Groesse f^l, der Tiefe c^{l-1}, dem Padding p^l und dem Stride s^l. Sie unterscheiden sich nur in den Modellparametern $w^l_{i\,|\,\alpha,\beta,\gamma}$.

$$\mathbf{W}_i^l = \left[\begin{pmatrix} w^l_{i\,|\,1,1,1} & \cdots & w^l_{i\,|\,1,f,1} \\ \vdots & \ddots & \vdots \\ w^l_{i\,|\,f,1,1} & \cdots & w^l_{i\,|\,f,f,1} \end{pmatrix} \overset{\vec{\gamma}}{\cdots} \begin{pmatrix} w^l_{i\,|\,1,1,c^l} & \cdots & w^l_{i\,|\,1,f,c^l} \\ \vdots & \ddots & \vdots \\ w^l_{i\,|\,f,1,c^l} & \cdots & w^l_{i\,|\,f,f,c^l} \end{pmatrix} \right]$$

Nun faltet man jeden Filter $\mathbf{W}_i^l$ einzeln ueber das Bilder $\mathbf{A}^l$ und erhaelt so mehrere neue 2D-Bilder $\tilde{\boldsymbol{A}}_i^l$.

$$\tilde{\boldsymbol{A}}_i^l = \mathbf{W}_i^l * \mathbf{A}^l \tag{3.7}$$

Die Faltung berechnet man nach folgender Gleichung:

$$(\tilde{\boldsymbol{A}}_i)_{y,x}^l = (\mathbf{W}_i^l * \mathbf{A}^l)_{y,x} = \sum_{v=1}^{f}\sum_{u=1}^{f}\sum_{w=1}^{c}(\mathbf{W}_i^l)_{v,u,w}(\mathbf{A}^l)_{x+v-1,y+u-1,w} \tag{FO}$$

Jede dieser Matrizen ist ein Querschnitt $\tilde{\boldsymbol{A}}_{:,:,i}^l$ entlang der Tiefe des neuen 3D-Tensors $\tilde{\mathbf{A}}^l \in \mathbb{R}^{h^{l+1}\times w^{l+1}\times c^l}$. Hierbei ist die Tiefe c^l gerade die Anzahl Filter c^l und die Hoehe h^{l+1} und die Breite w^{l+1} werden jeweils nach der folgenden Formel berechnet anhand der Hoehe h^l und der Breite w^l des Ausgangsbildes.

$$n^{l+1} = \frac{n^l + 2p^l - f^l}{s^l} + 1 \tag{3.8}$$

Da die Faltungsoperation eine lineare Operation ist, braucht man wieder eine nicht-lineare Aktivierungsfunktion um das Modell zu befaehigen nicht-lineare Probleme zu loesen. Deshalb wendet man in einem letzten Schritt die vektorisierte Aktivierungsfunktion $\boldsymbol{\varphi}$ auf den Tensor $\tilde{\boldsymbol{A}}^l$ an und erhaelt so die neue Aktiverung der naechsten Schicht $\mathbf{A}^{l+1}$. Empirisch hat man festgestellt, dass es fuer CNNs eine bessere Aktivierungsfunktion als die Sigmoidfunktion gibt. Die ReLU-Aktivierungsfunktion $\boldsymbol{\varphi}^{ReLU}$ aus Sektion (2.2.3) erziehlt bessere Resultate als die Sigmoidfunktion. Deshalb verwendet man eigentlich nur diese in CNNs.

$$\mathbf{A}^{l+1} = \boldsymbol{\varphi}[\tilde{\mathbf{A}}^l] \tag{3.9}$$

(Mehrere, *Convolutional neural network — Wikipedia, The Free Encyclopedia*)
(deeplearning.ai, *Convolutional Neural Networks (Course 4 of the Deep Learning Specialization)*)
(Goodfellow, Bengio und Courville, *Deep Learning*)
(Nielsen, *Neural Networks and Deep Learning*)

3.4 Dimensionalitaetskontrolle

Beim Anwenden einer Convolutional Schicht gehen Informationen verloren, da nur die relevanten Features hervorgehoben werden und der Rest verworfen wird. Jedoch schrumpft das Bild entweder gar nicht (bei Same-Padding) oder es schrumpft nur sehr leicht (bei Valid-Padding). Dies ist ein Problem, da die Information in deutlich weniger Pixeln bzw. Tensorelementen codiert werden koennte. Deshalb haben wir einen unoetigen Ressourcenverbrauch und eine erhoehte Gefahr fuer Overfitting. Um diesem Problem entgegenzuwirken, gibt es einerseits sogenannte **Pooling-Schichten** und als Gegenstueck dazu **Upsampling-Schichten**. Ersteres wird verwendet, um die Dimensionalitaet der Bilder zu vermindern, und letzteres, um die Dimensionalitaet der Bilder zu erweitern. Somit ermoeglichen diese Schichten eine kontrollierte Art (im Gegensatz zum Padding) die Dimensionalitaet einzustellen.

3.4.1 Die Pooling-Schicht

Die Pooling-Schicht verringert die Dimensionalitaet durch das Zusammenfassen eines Feldes von Tensorelementen zu einem einzigen Tensorelement der naechsten Schicht. Dabei geht das Feld nur entlang der Laenge und Breite des Bildes und nicht entlang der Tiefe. Dies bedeutet, das Pooling wird auf jeden Channel einzeln angewandt, dadurch aendert sich die Tiefe des Bildes nicht.

Fuer die Beschreibung einer Pooling-Schicht wird viel Begrifflichkeit der Convolutional-Schicht und der Filter uebernommen. Die Groesse des Elementfeldes, welches zusammengefasst wird, bezeichent man analog zur Filtergroesse mit f^l. Das Stride s^l bezeichnet auch beim Pooling wieder, wie gross die Schrittgroesse beim Verschieben des Feldes ist. Meistens waehlt man den Stride s^l gerade gleich der Feldgroesse f^l, damit alle Pixel zusammengefasst werden. Kleiner als f^l kann s^l nicht gewaehlt werden.

Man unterscheidet zwischen zwei Arten von Pooling:

- **Average-Pooling**: Das Elementenfeld wird zusammengefasst, indem man das arithmetische Mittel der Elemente bildet.
- **Max-Pooling**: Das Elementenfeld wird zusammengefasst, indem das Element mit dem hoechsten Wert beibehalten wird und die anderen verworfen werden.

In Praxis verwendet man eigentlich nur Max-Pooling, da es deutlich bessere Resultate erzielt.

MaxPooling Beispiel

$$\begin{pmatrix} 5 & 2 & 4 & 3 \\ 8 & 9 & 5 & 1 \\ 3 & 8 & 6 & 7 \\ 8 & 1 & 4 & 2 \end{pmatrix} \rightarrow_{\text{MaxPool}} \begin{pmatrix} 9 & 5 \\ 8 & 7 \end{pmatrix}$$

Es ist festzuhalten, dass eine Pooling-Schicht keinerlei Modellparameter besitzt. Somit gibt es nichts zu trainieren. Sie besitzt lediglich einige Hyperparameter, wie die Feldgroesse f^l und den Stride s^l.

Mit einer Pooling-Schicht, bei welcher $f^l = 2$ und $s^l = 2$ gewaehlt wurde, also ein (2×2)-Feld zusammengefasst wird, kommt es zu einer Dimensionalitaetsreduktion von 75%, da aus jeweils vier Elementen ein Element wird.

Um zu berechnen, was die neuen Dimensionen nach dem Pooling sind, gelten die gleichen Formeln wie fuer die Filteroperationen:

$$n_1 = \frac{n_0 - f}{s} + 1 \tag{3.10}$$

(deeplearning.ai, *Convolutional Neural Networks (Course 4 of the Deep Learning Specialization)*)
(Goodfellow, Bengio und Courville, *Deep Learning*)

3.4.2 Die Upsampling-Schicht

Die Upsampling-Schicht bildet das Gegenstueck zur Pooling-Schicht, da sie die Dimensionalitaet der Bilder erhoeht. Auf den ersten Blick erscheint unklar, weshalb man die Dimensionalitaet erhoehen wollen koennte, da die eigentliche Motivation dieser Schichten in der Verhinderung von Overfitting besteht. Jedoch sollte in Kapitel (4) klar werden, weshalb solche Schichten notwendig sind.

Man verwendet auch hier wieder den Begriff der Feldgroesse f^l. Diesemal beschreibt dieser Wert, wie stark das Bild hochskaliert wird. Beim Upsampling wird ein Element zu einem $(f^l \times f^l)$-Feld hochgerechnet und erweitert so die Dimensionalitaet.

Es stellt sich natuerlich die Frage, was fuer Werte das neue Feld erhaelt. Dafuer muss man eine Interpolationsart waehlen. Zwei Arten sind besonders verbreitet:

- **Bilineare Interpolation**
- **Naechste-Nachbar-Interpolation** (engl.: nearest-neighbor-interpolation)

Wir werden hier nur die Naechste-Nachbar-Interpolation betrachten. Sie ist ziemlich trivial, denn alle neuen Feldelemente nehmen einfach den Wert des alten Elements an. Der Wert wird also einfach vermehrt.
(deeplearning.ai, *Convolutional Neural Networks (Course 4 of the Deep Learning Specialization)*)
(Goodfellow, Bengio und Courville, *Deep Learning*)

Kapitel 4

Autoencoder

Nun moechten wir uns vorerst von der Architektur des Convolutional Neural Networks abwenden, um den Autoencoder zu betrachten. Wir werden uns erneut mit der Funktionsweise und der Nuetzlichkeit dieser Architekur auseinandersetzen. Danach werden wir den Autoencoder mit dem Convolutional Neural Network fusionieren und so einen Convolutional-Autoencoder erhalten. Zu guter letzt betrachten wir dann noch eine Anwendung eines solchen Convolutional-Autoencoder, naemlich einen Denoiser.

Ein **Autoencoder** ist eine weitere Architektur eines KNNs. Diese Architektur beschreibt das Netzwerk jedoch nicht bezueglich der Schichtenarten, sondern beschreibt es auf der Ebene der Netzform.
Die Aufgabe eines Autoencoders besteht darin eine neue Repraesentation einer Datenmenge zu erlernen. Diese neuartige Repraesentation soll mit weniger Daten moeglichst die gleiche Information codieren. Somit entwickelt das Modell eine **effiziente Daten-Codierung** fuer einen Datensatz. Dadurch kann er zur **Dimensionalitaetsreduktion** verwendet werden.
Neben dieser neuen Repraesentation erlernt das Netzwerk noch, wie es aus dieser Codierung wieder die Ursprungsdaten reproduzieren kann.

Autoencoder gehoeren nicht dem klassischen ueberwachtem Lernen an. Strenggenommen gehoeren sie dem unueberwachtem Lernen an, da in den Trainingsdaten keine Labels enthalten sein muessen. Jedoch ist diese Begrifflichkeit etwas irrefuehrend, weil nur deshalb keine Labels benoetigt werden, da die Labels gleich den Features sind! Somit wird das Netzwerk trotzdem darauf trainiert gewisse gewuenschte Outputs zu liefern.
(Saalmann, *Einfuehrung in Autoencoder und Convolutional Neural Networks*)

4.1 Topologie

Ein Autoencoder besteht wie das klassische KNN auch aus Neuronen. Wie bereits erwaehnt sind die "Labels" eines Autoencoders gleich den Features. Somit versucht ein Autoencoder einfach die Werte der Inputneuronen moeglichst exakt in die Outputneuronen zu kopieren. Diese Operation waere an sich ziemlich bedeutungslos und nutzlos, da mit den Daten nichts passiert. Das Netzwerk wuerde einfach erlernen eine Identitaetsfunktion zu imitieren. Aus diesem Grund muss man gewisse Einschraenkungen einfuehren. Diese bringen das Netzwerk dazu, interessante Methoden zu entwickeln um die Features *approximativ* zu rekonstruieren.

Die angesprochene Einschraenkung besteht darin, dass dem Autoencoder nicht beliebig viel Kapatzitaet fuer die Codierung der Features gegeben wird. Es stehen dem Netzwerk nur wenige Zahlenwerte zur Verfuegung um die Features in der neuen Repraesentation zwischenzuspeichern, bevor sie wieder in die urspruenliche Form gebracht werden. Diese Kapazitaetseinschraekung wird durch die Topologie des Autoencoders bezweckt.

Der einfachste Autoencoder, besteht aus drei Schichten, welche sich wie folgt eingliedern lassen:

- eine Inputschicht mit d Neuronen
- eine Zwischenschicht mit p Neuonen, bezeichnet als **Flaschenhals**
- eine Outputschicht mit d Neuronen

In Abbildung (**??**) ist ein beispielhafter Autoencoder abbgebildet, mit $d = 5$ Features und einem Flaschenhals der Groesse $p = 2$.

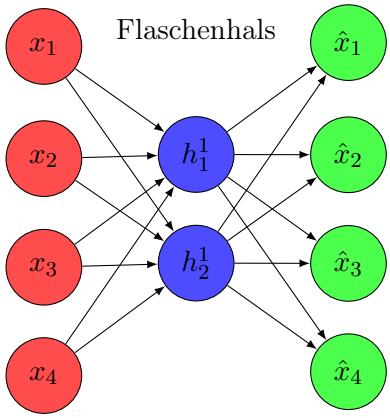


Abbildung 4.1: Schichten eines kurzen Autoencoders

Die Inputschicht enthaelt die Features $\boldsymbol{x} \in \mathbb{R}^d$. Die Outputschicht dagegen enthaelt die **Rekonstruktionen** der Features, welche wir mit $\hat{\boldsymbol{x}} \in \mathbb{R}^d$ bezeichnen. Als logische Konsequenz muessen die beiden Schichten die gleiche Anzahl d an Neuronen besitzen. Die Flaschenhalsschicht hingegen besitzt nur p Neuronen, wobei $p \ll d$. Somit ist die sie deutlich kleiner als die anderen beiden Schichten und bildet so die Kapazitaetseinschraenkung.

Wir haben nun einen Autoencoder nur mit einer Zwischenschicht betrachtet. Gaengier ist es jedoch das Netz aus mehreren Zwischenschichten zu bauen. Dabei werden die Schichten zum Flaschenhals hin immer schmaler und die Schichten nach dem Flaschenhals wieder immer dicker. In Abbildung (**??**) ist dies gut ersichtlich. Wir werden nun aber weiterhin mit dem Autoencoder, welcher nur eine Zwischtenschicht besitzt, die Funktionsweise herleiten.

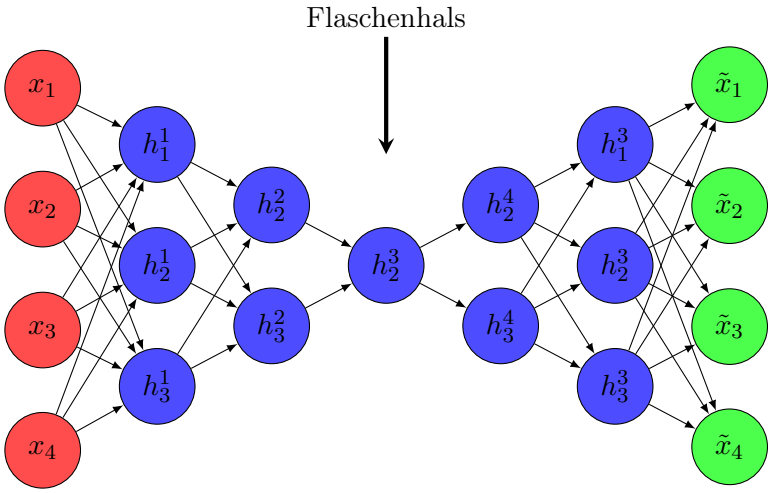


Abbildung 4.2: Schichten eines tieferen Autoencoders

4.2 Funktionsweise

Ein Autoencoder moechte die Features $\boldsymbol{x}$ von der Inputschicht in die Outputschicht kopieren. Jedoch zwingt der Flaschenhals den Autoencoder dazu eine neue Repraesenation der Features zu erlernen. Dies hat den Grund, dass die Features in ihrer jetzigen Form keinen Platz im Flaschenhals finden, da $p \ll d$. Somit muss das Modell die Informationen, welche in den Features stecken nur auf einige wenige Eigenschaften komprimieren. Es muss gewissermassen einen Code entwickeln. Diese neue Repraesentation nennt man das **Encoding**.
In einem zweiten Schritt muss der Autoencoder aus diesem Encoding wieder versuchen die urspruenlichen Features zu rekonstruieren. Diese Rekonstuktion bezeichnet man als das **Decoding**.

Encoder und Decoder

Man kann einen Autoencoder gewissermassen als zwei Teilmodelle betrachten. Das einte Modell erzeugt das Encoding und das andere das Decoding. Wir spalten dafuer die Hypothesenfunktion h in ein Funktionpaar (ϕ, ψ) auf.

- $\phi : \mathcal{X} \to \mathcal{E}$ Encoder-Funktion
- $\psi : \mathcal{E} \to \mathcal{X}$ Decoder-Funktion

Die erste Teilfunktion ϕ ist dafuer zustaendig das Encoding zu erzeugen. Sie bildet den Inputraum $\mathcal{X} \subseteq \mathbb{R}^d$, welcher die Features $\boldsymbol{x}$ enthaelt, auf den Encodingraum $\mathcal{E} \subseteq \mathbb{R}^p$, welcher die neuen Repraesenation $\boldsymbol{x}^*$ enthaelt, ab.
Das Gegenstueck ist die Teilfunktion ψ. Sie bildet den Encodingraum $\mathcal{E}$ zurueck auf den Inputraum $\mathcal{X}$ ab.

Der Output des Autoencoders $\hat{\boldsymbol{x}}$ wird somit durch die aufeinander folgende Anwendung der Teilfunktion auf die Features $\boldsymbol{x}$ gebildet:

$$\hat{\boldsymbol{x}} = \psi(\phi(\boldsymbol{x})) = (\psi \circ \phi)(\boldsymbol{x}) \tag{4.1}$$

Wenn wir dieses Modell trainieren, minimieren wir die Kostenfunktion. Als Beispiel koennen wir den Mittleren-quadratischen-Fehler C_{MSE} nehmen. Verwenden wir ihn als Kostenfunktion, minimieren wir folgenden Ausdruck:

$$\min_{\phi,\psi} |\boldsymbol{x} - \hat{\boldsymbol{x}}|^2 = \min_{\phi,\psi} |\boldsymbol{x} - (\psi \circ \phi)(\boldsymbol{x})|^2 \tag{4.2}$$

Anderst ausgedrueckt versuchen wir Funktion ϕ und ψ zu finden, welche die Differenz der urspruenglichen Features $\boldsymbol{x}$ und den Rekonstruktionen $\hat{\boldsymbol{x}}$ minimieren. Dabei muss der Autoencoder Dimensionalitaetsreduktion betreiben. Er entwickelt einen Code um die Features in der Flaschenhalsschicht komprimiert zu repraesentieren. Durch die Kapazitaetseinschraekung $p \ll d$ muss $\mathcal{E}$ eine niedrig-dimensionale Codierung der Features darstellen. Dem Modell ist es nicht moeglich die Inputs identisch wiederherzustellen, da bei der Abbildung ϕ Informationen verloren gegangen sind. Dadurch sollte der Code im Optimalen Fall nur noch die Merkmale der Features umfassen, welche fuer die groesste Information codieren. Dieser Code kann extrahiert werden, um eine Repraesentation der Features zu erhalten, welche nicht mehr mehrere Hundert Merkmale umfasst sonder nur noch ein paar wenige. Jedliche natuerliche Schwankungen in den Werten der Features gehen verloren, da sie nicht im COde enthalten sind. Sie koennen nicht verallgemeinert werden.

Der Code

Bei der Suche dieser Funktionen muss der Autoencoder einen gewissen "Code" entwickeln, um die Features neu zu repraesentieren. Dies da der Falschenhals nicht den noetigen Kapazitaet bietet, um die Features in der urspruenlichen Form zu halten. Somit muss der Autoencoder Dimensionalitaetsreduktion betreiben, um die Features komprimiert darzustellen.

Durch diese Komprimierung ist es dem Modell unmoeglich die Rekonstruktionen perfekt zu machen. Sie sind immer approximativ, da bei der Komprimierung Informationen verloren gegangen sind. Am

effektivsten wird die Kostenfunktion minimiert, wenn das Modell einen Code entwickelt, welcher die Features mit dem hoechsten Informationsgehalt abstrahiert. Die Features, welche unwichtig sind oder nur durch zuefalliges Rauschen der Daten entsteht koennen verworfen werden, da sie nicht verallgemeinert werden koennen.

Es ist wichtig zu erkennen, das der Code welcher der Autoencoder entwickelt **datenspezifisch** ist. Das heisst er kann nicht wie z.B ein Bildkompressionalgorithmus auf beliebige Daten angewendet werden. Stattdessen ist er nur sinnvoll fuer die Kompression von Daten, mit welchen er trainiert wurde. Der Autoencoder erlernt die Eigenschaften eines Datensatz zu abstrahieren und neu darzustellen. BEISPIEL EINER ABSTRAKTION (a la Menschliches Gesicht beschreiben).

4.3 Anwendungen

Die Eigenschaft, welche einen Autoencoder sich fuer die verschiedensten Anwendungen geeignet macht, ist die Dimensionalitaetsreduktion.
Wir werden nun einige Anwendungen grob betrachten.

4.3.1 Kompression

Es handlet sich um Verlustbehaftete Kompression. Dimensionalitaetsreduktion. Falls lineare Neuronen, aehnlich zur Hauptkomponentenanalyse $\frac{p}{d}$ des urspruenglichen Speicherbedarfs. Diese Komprimierung ist dabei Datenspezifisch.
Datenverkehr beschleunigen, weil weniger Daten.

4.3.2 Neue Datenrepraesentation

neues Datenformat

4.3.3 Visualisierung von Daten

Visualisierung von hochdimensionaler Daten in $\mathbb{R}^2$ und $\mathbb{R}^3$
Im uebernachsten Kapitel werden wir eine Anwendung noch genauer Betrachten die des Denoisers.

4.4 Convolutional-Autoencoder

Ein Convolutional-Autoencoder ist wie der Name es schon sagt, ein Autoencoder, welcher anstatt die normalen Fully-connected-Schichten eines KNNs die Schichten eines CNN verwendet.
Er ist interessant, weil er zur Kompession von Bildern verwendet werden kann. Wie bereits erwaehnt ist auch hier die Kompression datenspezifisch. Beispielsweise koennte man einen Convolutional-Autoencoder mit einem Datensatz von menschlichen Gesichtern trainieren. Dieser wuerde dann einen datenspezifischen Code fuer diese Gesichter entwickeln. Mann kann sich vorstellen, dass das Modell nicht mehr die Farbwerte der Features im Encoding speichert, sondern stattdessen diese Features abstrahiert. Somit koennten Eigenschaften, wie die Augenfarbe, Breite der Nase, Mundposition und vieles mehr im Flaschenhals gespeichert werden. Diese Informationen brauchen offenkundig weniger Kapazitaet als alle Pixelwerte.

Des weiteren sind Convolutional-Autoencoder interessant, da sie uns auf eine weitere Anwendung eines Autoencoders bringen: den Denoising-Autoencoder. Diesen betrachten wir im naechsten Abschnitt.
(CodeParade, *Computer Generates Human Faces*)

4.5 Denoising-Autoencoder

Ein Denoising-Autoencoder oder einfach Denoiser ist ein leicht abgeaenderter Autoencoder. Er wird verwendet um ein Rauschen aus Bildern zu entfernen.

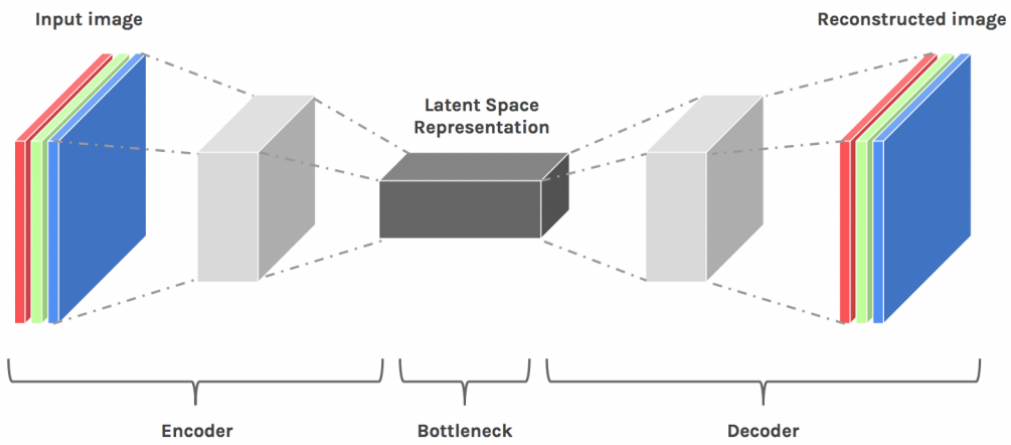


Abbildung 4.3: Darstellung eines Convolutional Autoencoders

Er unterscheidet sich insofern von dem klassischen Autoencodern, dass er richtige Labels hat und nicht einfach die Features kopiert. Sein Input stellen die verrauschten Bilder dar und die Labels sind die unverrauschten Bilder. Somit wird er darauf trainiert das Rauschen zu entfernen.
Man mag sich nun die Frage stellen, weshalb man hierfuer einene Autoencoder und kein normales CNN verwendet. Empirisch hat sich einfach gezeigt, dass sich Denoiser dafuer besser eignen. Das hat nachvollziehbare Gruende. Bei der Entwicklung des Code fuer Encoding-Repraesentation, kann das Rauschen nicht codiert werden, da es voellig zufaellig ist. Es existieren keine Muster oder Gesetzmaessigkeiten, welche anderst codiert werden koennten. Somit muss der Autoencoder diese Rausch-Features verwerfen. Somit hat er leichteres Spiel gegenueber sonstigen CNNs.
Corrupted Input

4.5.1 Generierung der verrauschten Bilder

Um den Autoencoder zu trainieren, muss man die verrauschten Daten generieren. Dies macht man indem man auf die Features ein additives Gauss'sches Rauschen anwendet. Man verwendet dafuer einfach eine Gauss'sche Normalverteilung $\mathcal{N}(\mu = 0, \sigma^2 = 1)$ mit Erwartungswert $\mu = 0$ und Varianz $\sigma^2 = 1$. Man entnimmt dieser dann einfach jeweils einen Zufallswert und addiert ihn zu jedem Feature. Somit erhaelt man einen Datensatz, welcher aus verrauschten Daten besteht.

$$r \sim \mathcal{N}(\mu = 0, \sigma^2 = 1)$$
$$\tilde{x} = x + r$$

Kapitel 5

Implementation fuer Maschinelles Lernen

In diesem Kapitel werden wir einige Frameworks betrachten, welche eine Implementation der ML-Algorithemen realisiert haben. Wir werden sie in ihrer groben Funktionsweise untersuchen. Diese Tools werden wir im naechsten Kapitel benutzten um dann ein konkretes Modell zu programmieren.

5.1 TensorFlow

Abbildung 5.1: TF-Logo

TensorFlow, kurz TF, ist ein von Google entwickeltes **Framework** fuer **datenstromorientriere Programmierung**. Es ist im Grunde genommen eine Mathematik-Programmbibliothek fuer numerische Berechnungen, dessen Hauptanwendungsbereich im Maschinellen Lernen liegt. Die Vorteile von TF bestehen vorallem darin, dass die praktische Anwendung des Frameworks sehr einfach ist und fast keine mathematische Vorkenntnisse voraussetzt. Somit kann man sehr einfach ein ML-Modell definieren, trainieren und verwenden. Ausserdem besitzt TF sehr performante Implementierungen, welche Algorithmen, wie die Vorwaertspropagierung und die Rueckwaertspropagierung, sehr schnell ausfuehren laesst. Somit geht das Training von ML-Modellen deutlich schneller als bei selbstgeschriebenen Implementationen. TF ist gratis und ein Open-Source-Projekt. Es kann auf Github gefunden werden unter: https://github.com/tensorflow/tensorflow.

In der Industrie ist TensorFlow sehr weit verbreitet. Es wird von den verschiedensten Unternehmen fuer ihre Anwendungen gebraucht, darunter waren: Google, Twitter, AirBnB, Intel, CocaCola, Snapchat und noch viele mehr.

Grundsaetzlich wurde TF fuer Maschinelles Lernen entwickelt. Speziell eigenet es sich fuer Deep Learning mit kuenstlichen neuronalen Netzen. Es ist ein Interface, welches das definieren von Modellen erlaubt, welche dann mit den eingebauten ML-Algorithmen trainiert werden koennen.
Trotz dieser eigentlichen Grundidee von TF, sollten die Abstraktionen eigentlich fuer beliebige Anwendugen, welche mit numerischen Berechnungen zu tun haben, sich eignen.

Im folgenden werden die grobe Funktionsweise von Tensorflow untersuchen. Um genau zu sein betrachten wir Tensorflow-Core Version r1.14. [1] Wir werden TF nur sehr grob betrachten, da wir spaeter nicht direkt damit arbeiten werden, sondern mit der High-Level-API Keras. Trotzdem ist er hilfreich ein grobes Verstaentniss ueber TF zu besitzen.

Die wichtige Resource um TensorFlow zu lernen ist wohl die offizielle TensorFlow Website. Dort findet man auch die Dokumentation.

[1] Momentan befindet sich TF 2.0 in der Beta. Es ist eine grosse Ueberarbeitung des Frameworks. Es ist zu empfehlen, sobald TF 2.0 offiziel veroeffentlich wurde, es zu verwenden.

(Many, *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*)

5.1.1 Client und Master

Wie die meisten grossen Programmbibliotheken ist Tensorflow in Backend und Fontend aufgeteilt. Das Frontend von Tensorflow bezeichnet man als **Client** und das Backend als **Master**.

Fontend und Backend

Das **Frontend** ist der Teil eines Programmes, mit welchem der Benutzer interagiert. Es stellt eine Schnittstelle zu dem sogennanten Backend eines Programmes dar. Es exponiert so gewisse Funktionalitaeten des Programmes dem Benuzter gegenueber, ohne dass er dabei etwas von der zugrundeliegenden Logik mitbekommt.

Das **Backend** ist das Gegenstueck zum Frontend. Es implementiert jedigliche Programmlogik, welche das gewuenschte Verhalten der Applikation hervorruft. Der Benutzer hat keinen direkten Zugriff auf diesen Teil des Programmes und bekommt somit nichts von dessen Existenz mit.

Der Client - also das Frontend - von TF ist in **Python** geschrieben. [2] Python ist allgemein als eine sehr einsteigerfreundliche Programmiersprache bekannt. Sie ermoeglicht es dem Benutzer mit wenigen Zeilen Code und ziemlich intuitiv TensorFlow zu verwenden. Wenn wir also mit TensorFlow programmieren, werden wir mit Python-Code arbeiten.

Der Master - also das Backend - von TF ist dagegen in **C++** (und CUDA) geschrieben. Dies hat den Grund, dass C++ im Gegensatz zu Python sehr performant ist. Also ist die Zeit zum Aufuehren eines Programmes sehr kurz. Dafuer ist der Programmcode deutlich anspruchsvoller und verboser. Da der Benutzer jedoch keine direkte Interaktion mit dem Master hat, bekommt er davon nichts mit.

5.1.2 Verwendung - der Client

Nun moechten wir betrachten wie man TensorFlow zu verwenden hat, damit wir spaeter ein ML-Modell damit basteln koennen. Dafuer werden gewisse Python-Kenntnisse vorausgesetzt. Wir betrachten nun also den Client von TF. Man verwendet verschiedene Konzepte um programmatisch mit TF zu arbeiten. Im folgenden Abschnitt werden wir diese Konzepte behandeln.

5.1.2.1 Graph

TensorFlow ist datenstromorientiert. Das bedeutet, dass man eine Berechnung (engl.: computation) durch einen gerichteten Graphen beschreibt. Zuerst stellt man diesen Graph auf und fuegt ihn zusammen und erst nach der Definition fuehrt man ihn aus. Es handelt es sich dabei um einen **Computation Graph**, wie wir ihn schon in Sektion (2.6) kennengelernt haben. In TensorFlow bezeichnet man diesen Graphen als einen `tf.Graph`.

Er besteht aus einer Menge an Knoten, welche miteinander ueber Pfade verbunden sind. Die Knoten halten Werte in sich, welche ueber die Pfade, verrechnet werden und in den naechsten Knoten eingespeist werden. Die Pfade sind also die Operationen, welche auf die Werte angewandt werden. So fuehrt der Graph eine Datenstrom-Berechnung aus. Jeder Knoten hat null oder mehr Inputs und null oder mehr Outputs.

Man muss einen Graph nicht explizit definieren (kann es aber, falls man mehrere Graphen braucht), da beim defnieren der Operationen sie implizit dem sogennanten **Default Graph** hinzugefuegt werden.

[2] Tensorflow besitzte mehrere Frontends, welche in den verschiedensten Programmiersprachen geschrieben sind. Wir werden jedoch nur das Python-Frontend betrachten, da es am besten von TF unterstuetzt ist.

5.1.2.2 Tensoren

Die zentrale Dateneinheit in TensorFlow sind Tensoren, wie wir sie bereits in Sektion (3.1) kennengelernt haben. Auch die Knoten des Graphs reprasentieren Tensoren, welche aber keine feste Wert enthalten, da sie erst beim Ausfuehren des Graphens, mit Werten befuellt werden. Nach der Ausfuehrung der Graphs, kann man die Zahlenwerte auslesen.

Man verwenet in TF deshalb Tensoren, weil so die Daten beliebige Dimensionalitaeten und Formen annehmen koennen. Es koennen sowohl Listen von Zahlen (Vektoren), wie auch Bilder (Tensoren dritter Ordnung) verrechnet werden.
Nun sollte auch klar sein, weshalb wir bei der Herleitung der Gleichungen, auch immer eine Matrixschreibweise erarbeitet haben.

In Tensorflow bezeichnet man einen Tensor mit `tf.Tensor`. Er ist definiert durch seine Form (Stufe inbegriefen) und den Datentyp seiner Elemente (z.B. tf.float32).

Jeder tf.Tensor ist mit einer einzigen Graph-Ausfuehrung assoziert. Sie besitzen ihren Wert nur fuer den jeweiligen Durchlauf, danach wird er verworfen. Man sagt die Tensoren sind "immutable", da die Werte nicht erhalten bleiben und so nicht modifiziert werden koennen.

Die soeben beschriebene Tensoren agieren einfach als Durchflussstellen innerhalb des Graphen. Sie muessen nicht explizit definiert werden, da sie automatisch eingefuegt werden, wenn der Graph durch seine Operationen definiert wird.

Neben dieser Art von Knoten, gibt es noch verschiedene Inputknoten. stellen den Input in den Graphen dar und muessen explizit definiert werden. Von ihnen gibt es verschiedene Arten, welche wir nun betrachten werden.

tf.constant

Der `tf.Constant` Tensor ist ein Input-Knoten in den Graphen. Er hat immer einen konstanten Wert als Output. Dieser wird beim Erstellen angegeben.

```
<Varname> = tf.constant(<Wert>)
```

Folgender Python-Code ist ein Beispiel, welcher drei konstante Tensoren mit unterschiedenlichen Formen erstellt. Als Argument, gibt man die Elemente des Tensors, worueber auch die Form ersichtlich an.

```
skalar = tf.constant(1.0)
vector2 = tf.constant([1.0, 2.0])
matrix2x2 = tf.constant([[1.0, 2.0], [3.0, 4.0]])
```

$$\text{skalar} = 1 \quad \text{vector2} = \begin{pmatrix} 1 & 2 \end{pmatrix} \quad \text{matrix2x2} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

tf.placeholder

Um den Graphen fuer Maschinelles Lernen zu verwenden, braucht man eine Moeglichkeit ihn mit Inputs fuer die Modelle zu befuettern. Um dies moeglichst komfortabel zu machen, gibt es den sogennanten `tf.Placeholder` Tensor. Wie der Name es schon sagt, ist er ein Platzhalter fuer die Inputs, welcher beim Start der Graphausfuehrnug eingespeist werden. Man muss einzig seine Dimensionen und seinen Datentyp (z.B. tf.float32) bei der Erstellung angeben.

```
<Varname> = tf.placeholder(dtype=<Datentyp>, shape=<Form>)
```

Fuer den Datentyp waehlt man eigentlich immer `dtype=tf.float32`. Somit kann man mit Gleitkommazahlen rechnen. Die Form gibt man einfach als n-Tupel von Integern an, welche die Anzahl Komponenten in der jeweiligen Dimension angeben. Somit bedeutet die Form `shape=(4,2,3)` einen Tensor im Tensorraum $\mathbb{R}^{4\times2\times3}$.

tf.variable

Ein weiteres Kriterium um Maschinelles Lernen mit dem tf.Graph durchzufuehren, ist dass man Modellparameter einfuehren kann. Dafuer ist der `tf.Variable` Tensor geeignet. Er ist im Gegensatz zu den anderen Tensoren "mutable". Das bedeutet, dass seine Werte unabhaengig von einer einzigen Graphenausfuehrung existieren und erhalten bleiben. Seine Elementen koennen modifiziert werden. Somit sind sie geeignet, fuer Modellparameter. Die Werte bleiben grundsaetzlich jede Ausfuehrung erhalten und koennen beim Trainingsdurchlauf angepasst werden.

Folgender Python-Code kreirt eine tf.Varaible mit einem Anfangswert und aendert ihren Wert danach.

```
<Varname> = tf.Variable(<Initialwert>, name=<optionaler Name>)
<Varname>.assign(<neuer Wert>)
```

Bei der neuen Wertebesetzung kann auch der alte Wert des Tensors referenziert werden, um den Wert zum Beispiel zu erhoehen: `var.assign(var + 2.0)`.

5.1.2.3 Operationen

Die Knoten des tf.Graph - also die Tensoren - werden ueber die Pfade aufeinander abbgebildet. Die Operationen, welche sie aufeinander abbildet, bezeichnet man als `tf.Operation`. Eine tf.Operation besitzt jeweils einen Namen und repraesentiert eine abstrakte Berechnung. Sie hat jeweils null oder mehr Tensor-Objekte als Input, wie auch als Output.

Man definiert meistens keine neuen Operationen, sondern verwendet einfach die vordefinierten. Ein Beispiel dafuer waere die `tf.matmul(a,b)`, welche eine Matrixmultiplikation zwischen `a` und `b` ausfuehrt. Im folgenden Code wird das Resulat im neuen Tensor `c` gespeichert. "MatMul" bezeichnet).

```
c = tf.matmul(a, b)
```

Mit diesem Konzept koennen wir nun endlich den eigentlichen Graph definieren. Dafuer muss man lediglich ein paar Inputtensoren erstellen, und diese mithilfe von Operationen verrechnen. Natuerlich kann man die Resulate der Operationen dann weiter verrechnen, bis man den gewuenschten Grapen hat.

Hier ein Beispiel eines solchen Graphs.

```
# Inputknoten
a = tf.constant([[1.0, 2.0], [3.0, 4.0]])
b = tf.constant([[1.0, 1.0], [0.0, 1.0]])
# Operationen
c = tf.matmul(a, b)
d = tf.placeholder(dtype=tf.float32, shape(2,2))
e = tf.add(c, e)
```

Abbildung 5.2: Visualisierung des beschriebenen `tf.graph`

5.1.2.4 Session

Der Client kommuniziert mit dem Master ueber eine sogennante `tf.Session`. Sie ist die Schnittstelle zwischen Frontend und Backend und ermoeglicht es einen tf.Graph zu erstellen und ihn auszufuehren. Um die Session zu erstellen verwendet man die Funktion `tf.Session()`.

```
sess = tf.Session()
```

Es ist Best-Practise eine Session, wie eine Datei ueber das `with`-Statment zu oeffen und zu schliessen.

```
with tf.Session() as sess:
# Code
```

Um den Graphen nun zu berechnen, muss man lediglich die Session ausfuehren, das geschieht ueber die Methode `tf.Session.run()`. Diese Funktion erwartet als Argument einerseits den Knotenpunkt, welchen man berchnen moechte. Dessen Wert ist der Rueckgabewert der Funktion. Andererseits erwartet die Funktion noch einen Python-Dictionary, den sogennanten `feed_dict`, welcher die `tf.placeholder` mit ihren Werten befuellt.

```
result = sess.run(<Knotenpunkt>, feed_dict=<Placeholder-Dictionary>)
```

Es folgt nun ein Codebeispiel, welches die Forwaertspropagierung eines Sigmoide Neurons mit drei Inputs implementiert. Es wird es mit den Inputs $x = \begin{pmatrix} 1 & 2 & 3 \end{pmatrix}^{\mathsf{T}}$ ausgefuehrt.

```
import tensorflow as tf

# Inputs
x = tf.placeholder(dtype=tf.float32, shape=(3))

# Modellparameter
w = tf.Variable(tf.random_uniform([3], -1, 1), name='weight')
b = tf.Variable(tf.random_uniform(1, -1, 1), name='bias')

# Forwaertspropagierung
z = tf.add(tf.tensordot(x, w, 1), b)
a = tf.sigmoid(z)
y = a

with tf.Session() as sess:
res = sess.run(y, feed_dict = {x: [1, 2, 3]})
print(res)
```

5.1.2.5 Optimizer

Um ein ML-Modell in TF zu implementieren, muss man das ganze Modell mit allen Operationen als Graphen definieren. Dies hoert sich im Moment noch sehr aufwendig an, ist es jedoch nicht. Wir werden naemlich die High-Level-API Keras zur Hilfe ziehen, bei der Graphen erstellt. Keras stellt Implementationen von verschiedenen KNN-Schichten zu verfuegen, damit man sie nicht mehr selber

erstellen muss. Diese kann man dann einfach zu einem Graphen zusammenfuegen. Aber dazu spaeter mehr.

Die Ausfuehrung des erstellen Graphen stellt dann einfach eine Vorwaertspropagierung dar. Bevor jedoch das Modell brachbare Resualte liert, muss man es trainieren. Dafuer verwendet man einen `tf.train.Optimizer`.

Der erste Schritt ist dafuer eine Kostenfunktion zu definieren. Es sthene dabei Verschiedne aus `tf.losses` zur Verfuegung. Als Argument nimmt die Kostenfunktion die Label-Tensoren `labels` und die Vorhersagen-Tensoren `predictions`. Beispielsweise koennte man den Mittleren Quadratischen Fehler waehlen:

```
cost = tf.losses.mean_squared_error(labels=y_hat, predictions=y)
```

Nun erstellt man den Optimizer. Dabei stehen wieder mehrere zur Auswahl. Der `tf.train.GraidentDescentOptimi` ist eine Implementation des Gradientenverfahren. Er nimmt als Arugment die Lernrate η.

```
optimizer = tf.train.GradientDescentOptimizer(learning_rate=<>)
```

Nun kann man den Optimizer damit beauftragen die Kostenfunktion `loss` zu minimieren. Ausserdem muss man dem Optimizer eine List von `tf.Variable`s geben, welche die zu optimierenden Modellparameter sind. Man erhaelt so man eine Optimierungs-Operation, welche man dann ausfuehren kann.

```
opt_op = opt.minimize(cost, var_list=<Hyperparameter-Liste>)
opt_op.run()
```

Nun folgt ein kleines Beispielmodell, welches trainiert wird.

```
import tensorflow as tf

# Inputs
x = tf.placeholder(dtype=tf.float32, shape=(3))

# Modellparameter
w = tf.Variable(tf.random_uniform([3], -1, 1), name='weight')
b = tf.Variable(tf.random_uniform(1, -1, 1), name='bias')

# Forwaertspropagierung
z = tf.add(tf.tensordot(x, w, 1), b)
a = tf.sigmoid(z)
y = a

cost = tf.losses.mean_squared_error(labels=y_true, predictions=y_pred)


with tf.Session() as sess:
print(sess.run(cost)) # cost is just another node
optimizer = tf.train.GradientDescentOptimizer(0.01)
train = optimizer.minimize()
res = sess.run(y, feed_dict = {x: [1, 2, 3]})
print(res)

# Daten
```

```
26  x = tf.constant([[1], [2], [3], [4]], dtype=tf.float32)
27  y_true = tf.constant([[0], [-1], [-2], [-3]], dtype=tf.float32)
28
29  # Modell
30  linear_model = tf.layers.Dense(units=1)
31  y_pred = linear_model(x)
32
33  # Evalutaion
34  sess = tf.Session()
35  init = tf.global_variables_initializer()
36  sess.run(init)
37
38  print(sess.run(y_pred))
39
40  # Loss
41  loss = tf.losses.mean_squared_error(labels=y_true, predictions=y_pred)
42
43  print(sess.run(loss))
44
45
46  # Training
47  optimizer = tf.train.GradientDescentOptimizer(0.01)
48  train = optimizer.minimize(loss)
49
50
51  for i in range(100):
52  _, loss_value = sess.run((train, loss))
53  print(loss_value)
```

5.1.3 Implementation - der Master

TF ist gerade deshalb sehr attraktiv fuer ML, da es sehr performant (schnell ausfuehrbar) ist und dadurch die Trainingszeit fuer die Modelle relativ kurz haltet. Dies wurde durch verschieden Designentscheidungen von TF erreicht.

In diesem Abschnitt werden wir grob die zugrundeliegende Implementation von TensorFlow betrachten - also den Master - und dabei herausfinden weshalb TF so performant ist. Ausserdem werden wir herausfinden, wie TF ableiten kann.

5.1.3.1 Devices

Die Kernstuecke des Masters sind die verschiedenen **Devices**, auf welchen die Berechnungen ausgefuehrt werden. Ein Device ist jedliche Art von Computerprozessor, auf welchem die Graphen ausfuehrt werden koennen. Zu diesen Prozessoren gehoeren die CPU (Hauptprozessor) und die GPU (Grafikprozessor). [3]

Der Master analysiert die verfuegbaren Devices und bewertete sie, bezueglich ihren Faehigkeiten. Anhand dieser Bewertung entscheidet der Master dann, auf welchem Device die Berechnugen ausgefuert werden. [4] Insofern GPUs zur Verfuegung stehen, waehlt TF grundsaetzlich immer diese, da sie Tensoroperationen deutlich schneller als CPUs verarbeiten koennen.

[3] Die normale TensorFlow Version ist nicht in der Lage die GPU zu verwenden. Um sie verfuegbar zu machen, muss man die GPU Version installieren. Wir werden im folgenden von der GPU-Version ausgehen, da sie erhebliche Performance Vorteile gewaehrt.

[4] Es ist moeglich die Graphen auf mehreren Devices gleichzeitig auszufuehren und so noch schneller die Modelle zu trainieren. Dies ist jedoch erst fuer sehr aufwendige Projekte noetig und erfordert ein fortgeschritteneres Versteantniss.

5.1.3.2 Why dataflow

Reasons:

- Parallelism. By using explicit edges to represent dependencies between operations, it is easy for the system to identify operations that can execute in parallel.
- Compilation. TensorFlow's XLA compiler can use the information in your dataflow graph to generate faster code, for example, by fusing together adjacent operations.
- Portability. The dataflow graph is a language-independent representation of the code in your model. You can build a dataflow graph in Python, store it in a SavedModel, and restore it in a C++ program for low-latency inference.

5.1.3.3 Performance und Hardwarebeschleunigung

Nun moechten wir die Gruende fuer die beachtliche Performance von TF erschliessen. Der erste wichtiger Faktor dabei ist, dass der Master hauptsaechlich in C++ geschrieben ist. C++ ist eine Programmiersprache die sehr nahe am Maschinen-Code (engl.: low-level) ist. Somit ist der Code sehr hardwarenah und kann so schneller ausgefuehrt werden. Der Nachteil besteht dafuer darin, dass die Programmiersprache ziemlich verbos ist. Das bedeutet man braucht viel Code um eine relativ einfach Idee auszudruecken. Deshalb wurde der Client von TF in Python geschrieben, was Abhilfe verschafft und so ein relativ einfaches und unverboses Entwickeln ermoeglicht.

Der andere wichtige Aspekt fuer die Performance ist wohl die sogennante **Hardwarebeschleuniging** (engl. Hardware Acceleration), von welcher TF gebrauch macht.
Hardwarebeschleunigung bezeichnet eine Sammlung an Methoden, bei welchen man spezialisierte Hardware verwendet um rechenintensive Aufgaben schneller auszufuehren. Die Architektur einer CPU ist zwar so ausgelegt, dass sie beliebige Aufgaben ausfuehren kann, jedoch meistens nicht besonders effizient. Deshalb wurden einerseits neue externe Hardwarebausteine, wie die GPU, welche sich fuer Grafikberechnungen besonders eignet, entwickelt. Andererseits wurde auch die interne Architektur der CPU so angepasst, dass sie auch spezielisierte Aufgaben besser meistert.

Der Grossteil der Hardwarebeschleunigung wird durch die **Parallelisierung** von Operationen erreicht. Dabei spaltet man eine rechenintensive Aufgabe in viele Teilaufgaben auf, welche gleichzeitig in gleicher Weise ausgefuehrt werden. Voraussetzung dafuer ist, dass die Teilaufgaben unabhaengig voneinander bearbeitet werden koennen.
Ein typisches Beispiel fuer die Paralleliserbarkeit von Operationen ist die Matrixmultiplikation. Moechte man eine Matrix $\boldsymbol{A} \in \mathbb{R}^{n \times m}$ mit einer zweiten Matrix $\boldsymbol{B} \in \mathbb{R}^{m \times p}$ multiplizieren, muss man praktisch m-mal die gleichen Schritte ausfuehren: Man berechnet jeweils das Skalarprodukt zwischen der i-ten Zeile von $\boldsymbol{A}$ und der i-ten Spalte von $\boldsymbol{B}$. Alle diese Skalarprodukte haben keinen Einfluss aufeindander und koennen so unabhaenigig mit der gleichen Prozedur berchnet werden.

Allgemein eignen sich Tensoroperatioen fuer Parallelisierung und damit zur Hardwarebeschleunigung. Nun ist auch ersichtlich, weshalb TensorFlow als Hauptdatentyp Tensoren verwendet; damit es gebrauch von der Hardwarebeschleunigung machen kann. wir fast alle Gleichungen des Maschinellen

CPU-Beschleunigung

Die Hardwarebeschleunigung der CPU ist fuer TensorFlow nur von geringfuegiger Relevanz, da TF sich vorallem fuer GPU-Hardwarebeschleunigung eignet und dafuer optimiert ist. Ausserdem ist es praktisch unmoeglich die gleiche Performance von TF mit einer CPU zu erreichen, welche mit einer GPU moeglich ist.

Hauptsaechlich parallelisiert TensorFlow mithilfe von der **SIMD**. SIMD ist ein Prinzip zur Parallelisierung einer Insturktion fuer mehrere Dateneinheiten und steht deshalb fuer "Single Instruction, Multiple Data". SIMD umfasst einerseits die spezifische CPU-Architektur und den dazugehoerigen Befehlssatz, um die Operatioen zu parallelisieren. Es gibt verschiedene
SSE4.1, SSE4.2, AVX, AVX2, FMA

GPU-Beschleunigung durch cuDNN und CUDA

cuDNN ist die NVIDIA CUDA Deep Neural Network library. Sie ist eine GPU-beschleunigte Bibliothek fuer deep neural networks. Sie stellt implementation fuer standard routines wie Vorwaerts- und Rueckwaertspropagierung bereit.

CUDA

cuDNN

5.1.3.4 Automatische Gradientenberechnung

Als wir die Rueckwaertsprogagierung hergeleitet haben, haben wir mithilfe von Computation Graphs die Kettenregel angewandt um die partiellen Ableitungen zu bilden. Genau das gleiche macht TF, jedoch automatisiert. Aus diesem Grund ist TF auch datenstromorient.

5.1.4 Tensorboard

5.1.4.1 Notizen

TensorBoard is a companion visualitzayion yool for TensorFlow for understanding the structure of the computation graphs as well as the overall behaviour of the ML-Modell.
Muss im terminal als Programm gestartet werden: tensorboard –logdir .

5.1.4.2 Visualizing computational graphs

Due to the size and topology of these graphs with maybe over 10'000 nodes the nodes are groupped and collapsed into high-level blocks. The visualtionsation is interactive and can pan, zoom and expand.

```
writer = tf.summary.FIleWriter('.')
writer.add_graph(tf.get_default_graph())
writer.flush()
```

5.1.4.3 Visualization of Summary Data/Training

When training Ml-models users often want to examine the state of various aspect of the model and how they change over time. In this way the can observe the learning behaviour of the model, whetever or not it converges and how fast it does this. With this informations the best hyper parameters can be chosen. The operations used to compute these summarisations are then inserted into the TF.graph. Interesting values to examine are value of the loss function or the time taken to execute the graph, histograbased summaries. The data for TensorBoard is written by TensorFlow in a log file, which then can be read by the TensorBoard program. The tensorboard starts a http server and shows the data in a web browser.

5.1.4.4 Visualization of Training

5.2 Keras

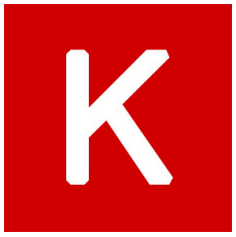


Abbildung 5.3: Keras-Logo

Keras ist ein Open-Source Deep-Learning Framework geschrieben in Python. Keras wurde entwickelt, um eine einheitliche Schnittstelle fuer verschieden Backends, wie TensorFlow, Microsoft Cognitive Toolkit und Theano zu bieten. Seit TF Version 1.4 ist Keras ein fester Bestandteil der TensorFlow-Core-API.
Keras ermoeglicht ein sehr simples und benutzerfreundliches defnieren von Deep-Learning-Modeln welche dann sehr einfach trainiert werden koennen. Wir werden also anstatt TF direkt zu verwenden, ueber Keras das Modell definieren. Deshalb ist ein tiefgehendes Verstaetniss von TF ueberfluessig.

5.2.1 Grundsaetliche Funktionsweise

In Keras ist es sehr einfach ein Modell zu bauen. Man reiht lediglich vordefinierte KNN-Schichten aneinander und baut so einen Graphen. Dafuer verwendet man meistens den `tf.keras.Sequential` Modus.

5.2.2 Schichten

Keras ist hat verschiedene Schichten von Kuenstlichen Neuronalen Netzen implementiert. Dies koennen also einfach in den tf.Graph eingefuegt werden. Wir werden nun diese verschieden Schichten betrachten.

Sequential-Modell/Dense-Layer

To build a simple, fully-connected network (i.e. multi-layer perceptron):

```
model = tf.keras.Sequential()
# Adds a densely-connected layer with 64 units to the model:
model.add(layers.Dense(64, activation='relu'))
# Add another:
model.add(layers.Dense(64, activation='relu'))
# Add a softmax layer with 10 output units:
model.add(layers.Dense(10, activation='softmax'))
```

Configure the layers

There are many tf.keras.layers available with some common constructor parameters:
activation: Set the activation function for the layer. This parameter is specified by the name of a built-in function or as a callable object. By default, no activation is applied. kernelinitializer and biasinitializer: The initialization schemes that create the layer's weights (kernel and bias). This parameter is a name or a callable object. This defaults to the "Glorot uniformïnitializer. kernelregularizer and biasregularizer: The regularization schemes that apply the layer's weights (kernel and bias), such as L1 or L2 regularization. By default, no regularization is applied.

```
# Create a sigmoid layer:
layers.Dense(64, activation='sigmoid')
# Or:
layers.Dense(64, activation=tf.sigmoid)

# A linear layer with L1 regularization of factor 0.01 applied to the kernel matrix:
layers.Dense(64, kernel_regularizer=tf.keras.regularizers.l1(0.01))

# A linear layer with L2 regularization of factor 0.01 applied to the bias vector:
layers.Dense(64, bias_regularizer=tf.keras.regularizers.l2(0.01))
```

```

# A linear layer with a kernel initialized to a random orthogonal matrix:
layers.Dense(64, kernel_initializer='orthogonal')

# A linear layer with a bias vector initialized to 2.0s:
layers.Dense(64, bias_initializer=tf.keras.initializers.constant(2.0))
```

Conv2D

MaxPooling2D

UpSampling2D

5.2.3 Train and evaluate

Train and evaluate Set up training
After the model is constructed, configure its learning process by calling the compile method:

```
model = tf.keras.Sequential([
# Adds a densely-connected layer with 64 units to the model:
layers.Dense(64, activation='relu', input_shape=(32,)),
# Add another:
layers.Dense(64, activation='relu'),
# Add a softmax layer with 10 output units:
layers.Dense(10, activation='softmax')])

model.compile(optimizer=tf.train.AdamOptimizer(0.001),
loss='categorical_crossentropy',
metrics=['accuracy'])
```

tf.keras.Model.compile takes three important arguments:
optimizer: This object specifies the training procedure. Pass it optimizer instances from the tf.train module, such as tf.train.AdamOptimizer, tf.train.RMSPropOptimizer, or tf.train.GradientDescentOptimizer. loss: The function to minimize during optimization. Common choices include mean square error (mse), categorical crossentropy, and binary crossentropy. Loss functions are specified by name or by passing a callable object from the tf.keras.losses module. metrics: Used to monitor training. These are string names or callables from the tf.keras.metrics module.

The following shows a few examples of configuring a model for training:

```
# Configure a model for mean-squared error regression.
model.compile(optimizer=tf.train.AdamOptimizer(0.01),
loss='mse',       # mean squared error
metrics=['mae'])  # mean absolute error

# Configure a model for categorical classification.
model.compile(optimizer=tf.train.RMSPropOptimizer(0.01),
loss=tf.keras.losses.categorical_crossentropy,
metrics=[tf.keras.metrics.categorical_accuracy])
```

5.2.3.1 Fit

Input NumPy data For small datasets, use in-memory NumPy arrays to train and evaluate a model. The model is fitto the training data using the fit method:

```
import numpy as np

def random_one_hot_labels(shape):
n, n_class = shape
classes = np.random.randint(0, n_class, n)
labels = np.zeros((n, n_class))
labels[np.arange(n), classes] = 1
return labels

data = np.random.random((1000, 32))
labels = random_one_hot_labels((1000, 10))

model.fit(data, labels, epochs=10, batch_size=32)
```

tf.keras.Model.fit takes three important arguments:
epochs: Training is structured into epochs. An epoch is one iteration over the entire input data (this is done in smaller batches). batch size: When passed NumPy data, the model slices the data into smaller batches and iterates over these batches during training. This integer specifies the size of each batch. Be aware that the last batch may be smaller if the total number of samples is not divisible by the batch size. validation data: When prototyping a model, you want to easily monitor its performance on some validation data. Passing this argument—a tuple of inputs and labels—allows the model to display the loss and metrics in inference mode for the passed data, at the end of each epoch.

5.2.3.2 Predict

The tf.keras.Model.evaluate and tf.keras.Model.predict methods can use NumPy data and a tf.data.Dataset. To evaluate the inference-mode loss and metrics for the data provided:

```
data = np.random.random((1000, 32))
labels = random one hot labels((1000, 10))

model.evaluate(data, labels, batch size=32)

model.evaluate(dataset, steps=30)


And to predict the output of the last layer in inference for the data provided, as a NumPy array:

result = model.predict(data, batch size=32)
print(result.shape)
```

5.2.4 Weiss nit

```
inputs = tf.keras.Input(shape=(32,))  # Returns a placeholder tensor

# A layer instance is callable on a tensor, and returns a tensor.
x = layers.Dense(64, activation='relu')(inputs)
x = layers.Dense(64, activation='relu')(x)

predictions = layers.Dense(10, activation='softmax')(x)

```

```
Instantiate the model given inputs and outputs.

model = tf.keras.Model(inputs=inputs, outputs=predictions)

# The compile step specifies the training configuration.
model.compile(optimizer=tf.train.RMSPropOptimizer(0.001),
loss='categorical_crossentropy',
metrics=['accuracy'])

# Trains for 5 epochs
model.fit(data, labels, batch_size=32, epochs=5)
```

Kapitel 6

Entwicklung eines Denoising-Autoencoders

Nun sind wir in dem Kapitel angelangt, in welchem das ganze erarbeitete Wissen zusammenfliesst. Wir werden ein konkretes Machine-Learning-Modell in TensorFlow zusammen mit Keras programmieren. Dabei wird darauf eingegangen, welche Schritte die Entwicklung eines solchen Modells umfasst und wie dabei am besten vorgegangen wird.

6.1 Das konkrete Modell

Als konkretes Modell werden wir einen Convolutional-Denoising-Autoencoder, dessen Theorie wir bereits behandelt haben, programmieren. Wir werden ihn verwenden um das Bildrauschen von Bildern wegzurechnen und sie so wieder erkenntlich zu machen. Als Bilder werden wir handgeschriebene Ziffern vom MNIST-Datenset verwenden.

6.1.1 Daten

Als Trainingsdaten verwenden wir den **MNIST**-Datensatz. Er ist der wohl bekannteste Datensatz fuer beispielhaftes Maschinelles Lernen. Er besteht aus schwarz-weiss Bildern von handgeschrieben Ziffern. Zuesaetzlich gaebe es auch noch Labels zu den Ziffern, da das Modell meistens fuer Ziffernerkennung trainiert wird. Jedoch werden wir diese nicht brauchen, da wir einen Autoencoder trainieren.

Der Datensatz besteht aus einem Trainingsdatensatz von 60'000 Sampels und einem Testdatensatz von 10'000 Sampels. Alle Ziffern wurden in soefern schon korrekt formatiert, dass ihre Groesse auf 28×28 Pixel normalisiert wurde und sie im Bild zentriert sind.

(Yann LeCun, *The MNIST Database of handwritten digits*)

6.2 Setup

Zuallererst muessen wir unser Entwicklungsumfeld richtig konfigurieren. Das bedeutet, dass wir Python zusammen mit den verschiedenen Programmbibliotheken installieren muessen.

Die Installationsschritte werden in dieser Arbeit fuer eine arch-basierte Linuxdistribution, welche den **Pacman**-Package-Manager verwendet, erklaert.

Falls der Leser ein anderes Betriebssystem verwendet, ist auf die offizielle TensorFlow Website fuer die Installationsschritte zu verweisen:

Abbildung 6.1: grobes Schema des Modells

Abbildung 6.2: ein Auszug an MNIST-Bildern (Steppan, *File:MnistExamples.png — Wikipedia, The Free Encyclopedia*)

Python3

Um TensorFlow und Keras verwenden zu koennen, muss Python3 installiert sein.
Um Python3 mithilfe von Pacman zu installieren, muss man folgenden Befehl in der Kommandozeile ausfuehren.

```
sudo pacman -S python
```

TensorFlow

Um TensorFlow zu installieren muss man unter Arch Linux nur das entsprechende Package installieren. Falls man einen Computer mit einer Nvidia-Grafikkarte mit einer "Compute Capability" von mehr als 3.5 besitzten, kann man von der GPU-Hardwarebeschleunigung gebraucht machen. Somit installiert man die CUDA-Version von TF mithilfe folgenden Kommandos:

```
sudo pacman -S python-tensorflow-cuda
```

Pacman installiert nun automatisch zuerst alle Programmabhaengigkeiten, wie CUDA, cuDNN und das TensorFlow-Backend, bevor es das eigentliche Python-TensorFlow installiert.

Falls man nicht eine geeignte Grafikkarte besitzt, installiert man am besten einfach die normale TF-Version ohne Unterstuetzung fuer die GPU. Dies macht man mit diesem Kommando:

```
sudo pacman -S python-tensorflow
```

Nun ist TF hoffentlich erfolgreich installiert. Keras muss nicht explizit installiert werden, da es in TensorFlow implementiert ist.

Python-Module

Wir werden fuer unser Python-Programm von zwei Packages Gebrauch machen, welche nicht in der Standardbibliothek von Python enthalten sind:

- NumPy: Ein Package welches verschiedene Mathematische Konzepte implementiert; vorallem Vektor- und Matrix-Arithmetik
- Matplotlib: Ein Package zum erstellen von Plots und Grafiken

Man installiert beide mit folgenden Kommandos.

```
sudo pacman -S python-numpy
sudo pacman -S python-matplotlib
```

6.3 Entwicklung

Nun beginnen wir mit der eigentlichen Programmierung.

6.3.1 Testprogramm

Zuallerst werden wir ein kleines Testprogramm schreiben, welches ueberprueft, ob alle Programmabhaengigkeiten korrekt installiert wurden und verwendet werden koennen.
In den ersten Zeilen des Pythonprogamms werden wir die Importstatments schreiben, um NumPy, Matplotlib und TensorFlow verfuegbar zu machen. Keras muss nicht explizit geladen werden, da es in TensorFlow drin steckt. Dann schreiben wir die Versionnummern der verschiedenen Programmbibliotheken in die Konsole und koennen so ueberpruefen, ob alles richtig konfiguriert ist.

```
import numpy as np # NumPy wird impotiert
import matplotlib as mpl # Matplotlib wird impotiert

import tensorflow as tf # TensorFlow wird impotiert

print(np.__version__) # Schreibt die NumPy-Version nach stdout.
print(mpl.__version__) # Schreibt die Matplotlib-Version nach stdout.
print(tf.__version__) # Schreibt die TensorFlow-Version nach stdout.
```

Falls der Output folgenden Charakter hat (die Versionsnummern muessen nicht die gleichen sein) und keine Fehlermeldungen angezeigt werden, sollte alles funktionieren.

```
1.17.
3.1.1
1.14.0
```

6.3.2 Trainingsdaten

Da wir nun verfiziert haben, dass alle Programmabhaengigkeiten funktionieren, koennen wir mit dem eigentlichen Programm beginnen.

Im den folgenden Auschnitten werden wir den Code immer weiter ausbauen und dabei erfahren, was er bewirkt. Wir beginnen unser eigentlichens Programm mit folgenden Importstatments.

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
```

6.3.2.1 Laden des MNIST-Datensatzes

Der nun erste Schritt besteht darin, die Datensaetze zu laden. Wie bereits erwaehnt, werden wir den MNIST-Datensatz verwendet. Da dieser dermassen bekannt ist, gibt es eine Funktion in Keras, welche automatisch die Daten herunterladt und sie als NumPy Arrays zurueckgibt. Die Funktion gibt die verschiedenen Komponenten der Daten in folgendem Format zurueck `(x_train, y_train), (x_test, y_test)`. Da wir nur an den Features `x` interessiert sind, verwerfen wir die Labels `y` einfach mithilfe der Wegwerf-Variabel `_`.

```
(x_train, _), (x_test, _) = tf.keras.datasets.mnists.load_data()
```

Nun sind die Features fuers Training in der Variabel `x_train` und die Features des Trainingsdatensatzes in der Variabel `x_test` gespeichert.

6.3.2.2 Formatieren der Daten

Nun muessen wir die Daten transformieren, damit sie die richtige Form fuer unser Modell haben. Wie bereits erklaert, ist der MNIST-Datensatz ziemlich handsam, da die Bilder alle die gleichen Masse haben und die Ziffern zentriert sind.

Wir muessen die Grauwerte normalisieren, denn im Moment liegen sie noch im Intervall $[0, 255]$ und sind vom Typ Integer. Unser Modell kann am besten mit Kommazahlen im Intervall $[0, 1]$ umgehen. Um diese Anpassung vorzunehmen brauchen wir folgenden Code.

```
x_train = x_train.astype('float32') / 255.0 # Normalisierung
x_test = x_test.astype('float32') / 255.0 # Normalisierung
```

Desweiteren werden wir in unserem Modell ConvLayers verwenden. Diese in Keras implementieren ConvLayers erwarten die Inputs in der Form (m, w, h, c), wobei m die Anzahl Bilder ist, w die Bildbreite, h die Bildhoehe und c die Anzahl Farbkomponenten. Wir formen nun mithilfe von NumPy das Array um.

```
x_train = np.reshape(x_train, (len(x_train), 28, 28, 1)) # neue Form: (60'000, 28, 28, 1)
x_test = np.reshape(x_test, (len(x_test), 28, 28, 1)) # neue Form: (10'000, 28, 28, 1)
```

Nun besitzen die Daten die richtige Form fuer unser Modell.

6.3.2.3 Generieren der verrauschten Bilder

Den Input in unser Modell stellen nicht die normalen MNIST-Bilder dar, sondern eine verrauschte Variante von ihnen. Diese werden wir nun generieren in dem wir ein additives Gauss'sches Rauschen anwenden.

Zuerst erstellen wir eine Matrix $\boldsymbol{R} \in \mathbb{R}^{28\times28\times1}$, welche die gleiche Form, wie die Bilder hat. Diese Matrix befuellen wir mit Zufallswerten. Fuer die Zufallswerte verwenden wir eine Gauss'sche Normalverteilung $\mathcal{N}(\mu = 0, \sigma^2 = 1)$ mit Erwartungswert $\mu = 0$ und Varianz $\sigma^2 = 1$. Da wir fuer jedes Bild eine eigene Rauschmatrix brauchen, generieren wir eine Liste $(\boldsymbol{R}_1, \boldsymbol{R}_2, \ldots, \boldsymbol{R}_m)$ der Laenge m an Rauschmatrizen. Dafuer verwenden wir die NumPy Funktion `np.random.normal(loc=<$\mu$>, scale=<$\sigma^2$>,size=<Form>)`. Die somit erhaltenen Rauschmatrizen multiplizieren wir dann mit einer

Rauschkonstane `noise_factor` und addieren das Resulat auf die MNIST-Bilder. Nach dem Hinzufuegen der Rauschwerte, muessen wir die Grauwerte noch auf das Intervall $[0, 1]$ kappen. Dafuer verwenden wir die NumPy-Funktion `np.clip(var, min, max)`. Diese Schritte wenden wir sowohl auf den Trainingsdatensatz, wie auch auf den Testdatensatz an.
Somit lautet unser Code:

```
noise_factor = 0.5

# fuer x_train
noise_matrices = np.random.normal(loc=0.0, scale=1.0, size=x_train.shape)
noise_matrices *= noise_factor
x_train_noisy = x_train + noise_matrices
x_train_noisy = np.clip(x_train_noisy, 0.0, 1.0)

# fuer x_test in Kurzfassung
x_test_noisy = x_test + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=x_test.shape)
x_test_noisy = np.clip(x_test_noisy, 0.0, 1.0)
```

Nun kann mithilfe der Matplotlib ein Blick auf die verrauschten Bilder neben den Originalbildern geworfen werden. Wir erstellen einfach einen Plot, welcher jeweils 10 Bilder beider Arrays rendert.

Dazu erstellen wir zuerst eine `pyplot.figure`. Innerhalb dieser haben wir jeweils 10 `subplots`, fuer die verrauschten MNIST-Bilder und die Originale. Wir zeigen dann die entsprechenden Bilder an, nachdem wir sie in Form gebracht haben. Wir muessen der Grafik auch angeweben, dass sie schwarz-weiss Bilder anzeigen muss. Zu guter letzt zeigen wir den Plot einfach an.

```
n = 10 # jeweils 10 Bilder
plt.figure()
for i in range(n):
  # verrauschte Bilder
  ax = plt.subplot(2,n,i+1+n)
  plt.imshow(x_test[i].reshape(28,28))
  plt.gray()

  # Original Bilder
  ax = plt.subplot(2, n, i+1)
  plt.imshow(x_test_noisy[i].reshape(28,28))
  plt.gray()

plt.show()
```

Wir erhalten so folgende Grafik:

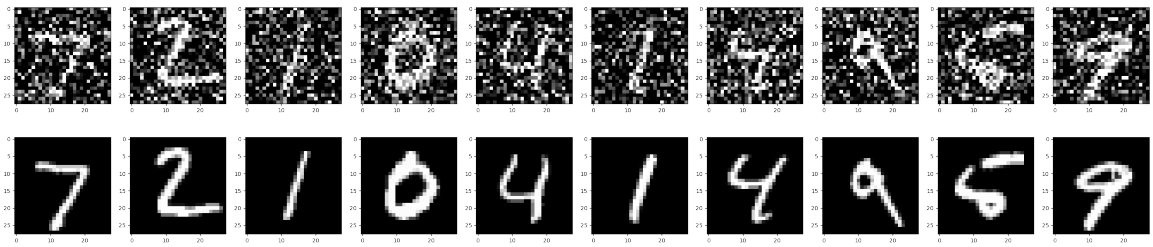


Abbildung 6.3: Verrauschte Bilder neben den Orignalbildern

Nun haben wir die Daten vollstaendigt im richtigem Format. Damit koennen wir nun zur Definition des Modells uebergehen.

6.3.3 Modell definieren

Nun werden wir mithilfe von Keras das Modell unseres Convolutional-Denoising-Autoencoders definieren. Wir werden das Modell nach dem Vorbild der Theorie bauen. Wie wir wissen umfasst die Topologie eines CNNs die verschiedentsten Hyperparameter. Wir werden der einfachheitshalber vorerst unbegruendete Werte dafuer waehlen. Spaeter werden wir diese Hyperparameter noch einstellen.

Wir beginnen unser Modell mit der Inputsschicht $l = 0$, welche einfach ein Tensor ist, welcher die Inputswerte haelt. Die Schicht hat die gleiche Form, wie ein MNIST-Bild.

```
input_data = tf.keras.Input(shape=(28,28,1))
```

Nach dieser Inputschicht folgt der Encoder des Autoencoders. Dieser besteht sowohl aus Convolutional-Schichten, wie auch aus Pooling-Schichten, welche sich abwechseln. Wir werden wie gesagt fuer alle Schichten vorerst die gleichen unbegruendeten Hyperparameter waehlen. Fuer die Convolutional-Schichten waehlen wir eine Filtergroesse $f^l = 3$, eine Anzahl Filter von $c^l = 32$, einen Stride $s = 1$, Same-Padding und die ReLU-Aktivierungsfunktion $\varphi = \varphi^{\text{ReLU}}$. Der Code fuer die Defition einer solchen Convolutional-Schicht lautet:

```
conv_layer = tf.keras.layers.Conv2D(filters=32, kernel_size=(3,3),
strides=(1,1),padding='same', activation='relu')(input_data)
```

Fuer die Pooling-Schichten waehlen wir analog eine Feldgroesse $f = 2$, einen Stride $s = 2$, damit jedes Feld einmal zusammengefasst wird und Same-Padding. Der Code fuer eine dieser Pooling-Schichten lautet:

```
pool_layer = tf.keras.layers.MaxPooling2D(pool_size=(2,2),strides=(2,2),padding='same')(econv0)
```

Nun folgt der Decoder. Fuer ihn verwenden wir die gleichen Convolutional-Schichten wie im Encoder. Anstatt Pooling-Schichten enthaelt er UpSampling-Schichten, bei welchen die Feldgroesse $f = 2$ gewaehlt wurde. Somit werden 4×4-Felder zu einem einzigen Pixel zusammengefasst. Als Interpolationsmethode benutzten wir den Naechsten-Nachbar-Algorithmus. Eine solche UpSampling-Schicht definiert man also so:

```
dupsample0 = tf.keras.layers.UpSampling2D(size=(2,2), interpolation='nearest')(dconv0)
```

Wir definieren jetzt alle Schichten und verknuepfen sie miteinander. Wir bauen einen Autoencoder, dessen Flaschenhals zwei Convolutional-Schichten tief ist. Somit besteht der Encoder aus zwei Convolutional-Layer-Pooling-Layer-Paar, dann folgt der Flaschenhals und aus zwei Convolutional-Layer-UpSampling-Layer-Paar. Der ganze Graph wird durch folgenden Code gebildet:

```
# Encoder
econv0 = tf.keras.layers.Conv2D(
  filters=32, kernel_size=(3,3), strides=(1,1),padding='same',activation='relu')(input_data)
emaxpool0 = tf.keras.layers.MaxPooling2D(
  pool_size=(2,2),strides=None,padding='same')(econv0)
econv1 = tf.keras.layers.Conv2D(
  filters=32, kernel_size=(3,3), strides=(1,1),padding='same',activation='relu')(emaxpool0)
emaxpool1 = tf.keras.layers.MaxPooling2D(
  pool_size=(2,2), strides=None,padding='same')(econv1)

encoded =  emaxpool1 # Flaschenhals der Form (7,7,32)
```

```
12
13  # Decoder
14  dconv0 = tf.keras.layers.Conv2D(
15    filters=32, kernel_size=(3,3), strides=(1,1), padding='same', activation='relu')(encoded)
16  dupsample0 = tf.keras.layers.UpSampling2D(
17    size=(2,2), interpolation='nearest')(dconv0)
18  dconv1 = tf.keras.layers.Conv2D(
19    filters=32, kernel_size=(3,3), strides=(1,1), padding='same', activation='relu')(dupsample0)
20  dupsample1 = tf.keras.layers.UpSampling2D(
21    size=(2,2), interpolation='nearest')(dconv1)
22  dconv2 = tf.keras.layers.Conv2D(
23    filters=1,kernel_size=(3,3), strides=(1,1), padding='same', activation='sigmoid')(dupsample1)
24  decoded = dconv2
```

Wie im Code zu sehen, haben wir fuer alle Convolutional-Layers die ReLU Aktivierungsfunktion gewaehlt, ausser der letzten Schicht. Fuer die letzte Schicht gebauren wir die Sigmoid-Aktivierungsfunktion. Dies hat den Grund, dass der Output unseres Netzwerk wieder ein Bild sein muss, dessen Grauwerte im Invervall $[0, 1]$ liegen muessen. Eine Sigmoidfunktion hat genau dieses Intervall als Wertemenge.

Nun muessen wir noch das Kerasmodell mit diesem Graphen definieren. Also definieren wir das Modell einfach als der Abschnitt des Graphen zwischen der Input und dem Decoding. Und es danach mit kompilieren. Bei der Kompilierung wahlen wir als Optimierungsverfahren einfach das Stochastische-Gradientenverfahren SGD. Als Kostenfunktion waehlen wir den Mittleren-Quadratischen-Fehler C_{MSE}.

```
1  autoencoder = tf.keras.Model(input_data, decoded)
2  autoencoder.compile(optimizer='sgd',loss='mean_squared_error') # SGD und MSE
```

Mithilfe von `tf.keras.Model.summary` kann man eine Zusammenfassung des Modells in Textform erhalten. So kann man Informationen bezueglich der Form der verschiedenen Schichten extrahieren und ueberpruefen, ob alles stimmig ist. Fuer unser Modell erhalten wir folgende Summary:

```
1  _________________________________________________________________
2  Layer (type)                 Output Shape              Param #
3  =================================================================
4  input_1 (InputLayer)         [(None, 28, 28, 1)]       0
5  _________________________________________________________________
6  conv2d (Conv2D)              (None, 28, 28, 32)        320
7  _________________________________________________________________
8  max_pooling2d (MaxPooling2D) (None, 14, 14, 32)        0
9  _________________________________________________________________
10 conv2d_1 (Conv2D)            (None, 14, 14, 32)        9248
11 _________________________________________________________________
12 max_pooling2d_1 (MaxPooling2 (None, 7, 7, 32)          0
13 _________________________________________________________________
14 conv2d_2 (Conv2D)            (None, 7, 7, 32)          9248
15 _________________________________________________________________
16 up_sampling2d (UpSampling2D) (None, 14, 14, 32)        0
17 _________________________________________________________________
18 conv2d_3 (Conv2D)            (None, 14, 14, 32)        9248
19 _________________________________________________________________
20 up_sampling2d_1 (UpSampling2 (None, 28, 28, 32)        0
21 _________________________________________________________________
22 conv2d_4 (Conv2D)            (None, 28, 28, 1)         289
```

```
=================================================================
Total params: 28,353
Trainable params: 28,353
Non-trainable params: 0
_________________________________________________________________
```

Wir sehen hier schoen, dass wir circa 28'000 Hyperparameter haben, welche wir zu trainieren haben.

6.3.4 Training

Nun muessen wir das definierte Modell trainieren. Mit Keras ist das mit einem Funktionsaufruf gemacht. Diese Funktion erwartet einige Argumente. Zuerst muessen wir die Features und die Labels angeben. Die Features haben wir in der Variabel `x_train_noisy` gespeichert. Die Labels sind die unverrauschten MNIST-Bilder, welche in Variable `x_train` sind. Desweiteren spezifizieren wir die groesse eines Mini-Batches als 128. Wir werden fuer 100 Epochen trainieren und geben dies entsprechend an. Wichtig ist auch, dass wir die Trainingssampels vor dem Training mischen, damit keine Mustern innerhalb der Anordnung der Samples erlernt werden. Als letzen geben wir noch den Testdatensatz an.

```
autoencoder.fit(
  x=x_train_noisy, y=x_train, batch_size=128, epochs=100, shuffle=True,validation_data=(x_test_noisy,x_test))
```

Wenn wir jetzt unser Modell trainieren, macht TensorFlow, falls die CUDA-Version von TF installiert ist und eine geeignete Nvidia-Grafikarte gefunden wurde, gebrauch von ihr. Somit sollte das Training innerhalb einiger Minuten beendet sein. Waehrend des Trainings schreibt TensorFlow Informationen zum Fortschritt in die Kommandozeile. Diese Infos haben folgenden Charakter und informieren ueber die aktuelle Epoche und die Werte der Kostenfunktion. Die Kosten fuer den Trainingsdatensatz werden mit `loss` bezeichnet und die fuer den Testdatensatz mit `val_loss`.

```
Epoch 99/100
60000/60000 [==============================] - 3s 57us/sample - loss: 0.1102 - val_loss: 0.1089
```

Das Modell hat nach dem dem Training einen `val_los` von 0.1089, bezueglich dem Testdatensatz.

Jetzt, da das Modell trainiert ist, speichern wir es und alle seine Modellparameter als Datei auf unserem Computer ab. So muessen wir es nicht jedesmal wieder neu trainieren, sondern koennen einfach die Modelldatei einlesen lassen.

```
autoencoder.save('denoiser.model')
```

Um das Modell wieder einzulesen verwendet man folgenden Code:

```
autoencoder = tf.keras.models.load_model('denoiser.model')
```

Mit der Summary-Funktion kann man ueberpruefen, ob das richtige Modell geladen wurde.

6.3.5 Modell ausfuehren

Nun kann das trainierte Modell verwenden werden, um nun eben Bilder zu entrauschen. Wir verwenden dafuer Bilder aus dem Testdatensatz, da diese nicht zum Training verwenden wurden. Mit der Funktion `tf.Keras.Model.predict` erhaelt man alle Vorhersagen unseres Autoencoders zum ganzen Testdatensatz.

```
denoised_imgs = autoencoder.predict(x_test)
```

Mithilfe von der Matplotlib koennen wir nun diese Entrauschten Bilder mit den urspruenliche MNIST-Bildern Seite an Seite darstellen. So koennen wir von Auge beurteilen, wie gut die Ergebnisse sind.

```
n = 10 # jeweils 10 Bilder
plt.figure()
for i in range(n):
  # verrauschte Bilder
  ax = plt.subplot(3, n, i+1)
  plt.imshow(x_test_noisy[i].reshape(28,28))
  plt.gray()

  # entrauschte Bilder
  ax = plt.subplot(3, n, i+1+n)
  plt.imshow(decoded_imgs[i].reshape(28,28))
  plt.gray()

  # Original-Bilder
  ax = plt.subplot(3, n, i+1+2*n)
  plt.imshow(x_test[i].reshape(28,28))
  plt.gray()

plt.show()
```

In der Grafik, welche wir erhalten, sind zuoberst die verrauchsten Bilder, in der Mitte sind die Rekonstruktionen und ganz unten sind Original-Bilder. Die Resultate sind schon verblueffend guet.

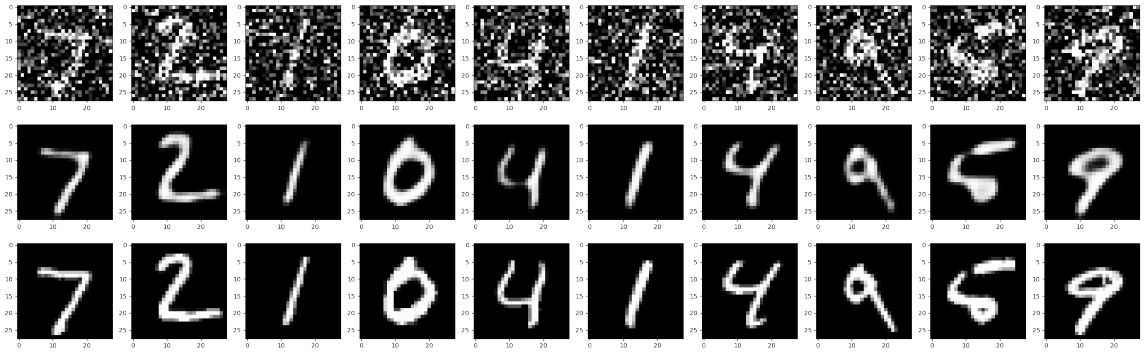


Abbildung 6.4: Verrauschte Bilder, Rekonstruktionen und Orignalbilder

6.3.6 Hyperparameter einstellen

Wir in der Theorie so oft erkaert, muss man um die Hyperparameter einzustellen einfach verschiedene Werte ausprobieren. Wir werden nun unser Programm zu modifizeren, dass er verschiedene Modell mit jeweils unterschiedlichen Hyperparametern trainiert. Beim Training erfassen wir mithilfe von TensorBoard Daten zum Lernfortschritt und den Kosten. Dann koennen wir mithilfe von TensorBoard eine Auswertung ausfuehren und so die besten Hyperparameter finden.

Wir werden unser Programm mithilfe von For-Loops ausbauen, damit es die verschiedenen Modelle trainiert.

6.3.7 TensorBoard Konfiguration

Damit wir den Lernprozess und die Leistungen unseres Modells analysieren koennen, werden wir nun TensorBoard konfigurieren. Um TensorBoard einzurichten, muessen wir zuerst eine entsprechende TensorBoard Variable definieren. Hierbei muessen wir angeben wohin die TensorBoard-Daten geschrieben werden. Da wir die verschiedenten Hyperparameter-Einstellungen haben, welche wir alle vergleichen wollen, benennen wir die Dateien aussagekraeftig.
Desweiteren spezifizieren wir gewisse Optionen fuer die Datenerfassung. Diese muessen bei der Komilierung des Modells unter `metrics` angegeben werden. Wir interessieren uns an der Akkuratesse des Modells.

```
autoencoder.compile(optimizer='sgd',loss=loss, metrics=['accuracy'])
NAME = '{}-{}-batch-{}-filters-{}-denoiser'.format(loss, activation_function, batch_size, num_filters)
tensorboard =
tf.keras.callbacks.TensorBoard(log_dir='logs/{}'.format(NAME))
autoencoder.fit(x=x_train_noisy, y=x_train, batch_size=batch_size, epochs=EPOCHS, shuffle=True,validation_data=(
```

Kapitel 7

Diskussion

7.1 Auswertung mit TensorBoard

Anhang A

Programmcode

```
import numpy as np
import matplotlib.pyplot as plt

import tensorflow as tf

(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()
x_train = x_train.astype('float32') / 255.
x_test = np.reshape(x_test, (len(x_test),28,28,1))
x_test = x_test.astype('float32') / 255.
x_train = np.reshape(x_train, (len(x_train),28,28,1))

noise_factor = 0.5
x_train_noisy = x_train + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=x_train.shape)
x_test_noisy = x_test + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=x_test.shape)
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)


input_data = tf.keras.Input(shape=(28,28,1))

econv0 = tf.keras.layers.Conv2D(filters=32, kernel_size=(3,3), strides=(1,1),padding='same',activation='relu')(inp
emaxpool0 = tf.keras.layers.MaxPooling2D(pool_size=(2,2),strides=None,padding='same')(econv0)
econv1 = tf.keras.layers.Conv2D(filters=32, kernel_size=(3,3), strides=(1,1),padding='same',activation='relu')(ema
emaxpool1 = tf.keras.layers.MaxPooling2D(pool_size=(2,2), strides=None,padding='same')(econv1)
encoded =  emaxpool1 # bottleneck (shape=(7,7,32))
dconv0 = tf.keras.layers.Conv2D(filters=32, kernel_size=(3,3), strides=(1,1), padding='same', activation='relu')(e
dupsample0 = tf.keras.layers.UpSampling2D(size=(2,2), interpolation='nearest')(dconv0)
dconv1 = tf.keras.layers.Conv2D(filters=32, kernel_size=(3,3), strides=(1,1), padding='same', activation='relu')(d
dupsample1 = tf.keras.layers.UpSampling2D(size=(2,2), interpolation='nearest')(dconv1)
dconv2 = tf.keras.layers.Conv2D(filters=1,kernel_size=(3,3), strides=(1,1), padding='same', activation='sigmoid')(
decoded = dconv2

autoencoder = tf.keras.Model(input_data, decoded)
autoencoder.compile(optimizer='adadelta',loss='binary_crossentropy')

print(autoencoder.summary())


autoencoder.fit(x=x_train_noisy, y=x_train, batch_size=128, epochs=100, shuffle=True,validation_data=(x_test_noisy

```

```
decoded_imgs = autoencoder.predict(x_test)

autoencoder.save('denoiser.model')

n = 10
plt.figure(figsize=(20,4))
for i in range(n):
    # original
    ax = plt.subplot(2, n, i+1)
    plt.imshow(x_test_noisy[i].reshape(28,28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)

    # reconstruction
    ax = plt.subplot(2,n,i+1+n)
    plt.imshow(decoded_imgs[i].reshape(28,28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
plt.show()
```

Anhang B

Ausfuehrungen zu TensorFlow

Backend

Anhang C

Intuitiver Beweis fuer UAT?

Literatur

[1] CodeParade. *Computer Generates Human Faces*. [Online; besucht am 21. August 2019]. Youtube. 2017. URL: https://www.youtube.com/watch?v=4VAkrUNLKSo.

[2] Andrew NG - deeplearning.ai. *Convolutional Neural Networks (Course 4 of the Deep Learning Specialization)*. [Online; besucht am 17. Juni 2019]. 2017. URL: https://www.youtube.com/playlist?list=PLkDaE6sCZn6Gl29AoE31iwdVwSG-KnDzF.

[3] Ian Goodfellow, Yoshua Bengio und Aaron Courville. *Deep Learning*. http://www.deeplearningbook.org. MIT Press, 2016.

[4] Many. *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. Software available from tensorflow.org. 2015. URL: https://www.tensorflow.org/.

[5] Mehrere. *Convolution— Wikipedia, The Free Encyclopedia*. [Online; besucht am 25. Juni 2019]. 2019. URL: https://en.wikipedia.org/wiki/Convolution.

[6] Mehrere. *Convolutional neural network — Wikipedia, The Free Encyclopedia*. [Online; besucht am 25. Juni 2019]. 2019. URL: https://en.wikipedia.org/wiki/Convolutional_neural_network.

[7] Mehrere. *Kernel (image processing) — Wikipedia, The Free Encyclopedia*. [Online; besucht am 25. Juni 2019]. 2019. URL: https://en.wikipedia.org/wiki/Kernel_(image_processing).

[8] Mehrere. *Kuenstliches Neuron — Wikipedia, Die freie Enyklopädie*. [Online; besucht am 4. Mai 2019]. 2019. URL: https://de.wikipedia.org/wiki/K%C3%BCnstliches_Neuron.

[9] Mehrere. *Kuenstliches Neuronales Netz — Wikipedia, Die freie Enyklopädie*. [Online; besucht am 04. Mai 2019]. 2019. URL: https://de.wikipedia.org/wiki/K%C3%BCnstliches_neuronales_Netz.

[10] Mehrere. *Linear separability — Wikipedia, The Free Encyclopedia*. [Online; besucht am 9. Juli 2019]. 2019. URL: https://en.wikipedia.org/wiki/Linear_separability.

[11] Mehrere. *Normal distribution — Wikipedia, The Free Encyclopedia*. [Online; besucht am 26. Juni 2019]. 2019. URL: https://en.wikipedia.org/wiki/Normal_distribution.

[12] Mehrere. *Perzeptron — Wikipedia, Die freie Enyklopädie*. [Online; besucht am 28. April 2019]. 2019. URL: https://de.wikipedia.org/wiki/Perzeptron.

[13] Mehrere. *Sigmoidfunktion — Wikipedia, Die freie Enyklopädie*. [Online; besucht am 4. Mai 2019]. 2019. URL: https://de.wikipedia.org/wiki/Sigmoidfunktion.

[14] Mehrere. *Sobel operator — Wikipedia, The Free Encyclopedia*. [Online; besucht am 17. Juni 2019]. 2019. URL: https://en.wikipedia.org/wiki/Sobel_operator.

[15] Mehrere. *Supervised learning — Wikipedia, The Free Encyclopedia*. [Online; besucht am 25. Juni 2019]. 2019. URL: https://en.wikipedia.org/wiki/Supervised_learning.

[16] Mehrere. *Tensor — Wikipedia, Die freie Enyklopädie*. [Online; besucht am 13. Juli 2019]. 2019. URL: https://de.wikipedia.org/wiki/Tensor.

[17] Mehrere. *Universal approximation theorem — Wikipedia, The Free Encyclopedia*. [Online; besucht am 22. August 2019]. 2019. URL: https://en.wikipedia.org/wiki/Universal_approximation_theorem.

[18] Mehrere. *Unsupervised learning — Wikipedia, The Free Encyclopedia*. [Online; besucht am 25. Juni 2019]. 2019. URL: https://en.wikipedia.org/wiki/Unsupervised_learning.

[19] Michael Nielsen. *Neural Networks and Deep Learning*. http://neuralnetworksanddeeplearning.com. 2018.

[20] Elias Saalmann. *Einfuehrung in Autoencoder und Convolutional Neural Networks*. Universitaet Leipzig, 2018.

[21] Josef Steppan. *File:MnistExamples.png — Wikipedia, The Free Encyclopedia*. [Online; besucht am 18. August 2019]. 2017. URL: https://en.wikipedia.org/wiki/File:MnistExamples.png.

[22] Christopher J.C. Burges Yann LeCun Corinna Cortes. *The MNIST Database of handwritten digits*. [Online; besucht am 15. August 2019]. URL: http://yann.lecun.com/exdb/mnist/.

Abbildungsverzeichnis

1.1 Schema eines Modells . . . 3
1.2 ein Skalarfeld als Konturdiagramm mit zugehoerigem Gradientenfeld . . . 5
1.3 Visualisierung des Gradientenabstiegs: ein Ball rollt das Gradientenfeld hinab in das lokale Minimum. . . . 6
1.4 Visualisierung verschiedener Lernraten . . . 6

2.1 Perzeptron mit drei Inputs . . . 9
2.2 erfolgreiche lineare Separierung (links) und das Versagen bei XOR (rechts) . . . 9
2.3 ein kuenstliches Neuron und seine Bestandteile . . . 10
2.4 Definition und Graph der Heaviside-Funktion Θ . . . 10
2.5 Formel und Graph der ReLU-Funktion . . . 11
2.6 Definition, Ableitung und Graph der Sigmoid-Funktion σ . . . 12
2.7 Schichten eines KNNs . . . 12
2.8 zum Verstaentniss der Nomenklatur . . . 14
2.9 zum Verstaentnis der Gewichtebeschriftungen . . . 14
2.10 Graph der Dichtefunktion $\phi(x \mid \mu = 0, \sigma^2 = 1)$ mit ihren wichtigsten Eigenschaften . . . 16
2.11 Computation Graph einer exemplarischen Verkettung von Funktionen . . . 17
2.12 Computation Graph zur Berechnung von $\boldsymbol{\delta}^L$ . . . 18
2.13 Computation Graph zur Berechnung von δ_j^{l-1} . . . 19

3.1 Beispiel einer Bildmatrix . . . 23
3.2 Schichtung eines CNNs . . . 24
3.3 der Instagramfilter: Blablabla . . . 24
3.4 eine 2D-Filtermatrix . . . 24
3.5 Sobel-Filter angewandt auf Beispielsbild . . . 25
3.6 ein Filter und sein rezeptives Feld . . . 26
3.7 Schema, wie ein Filter ueber ein Bild laueft . . . 27
3.8 aufeinander gelegte gefaltene Bilder . . . 28
3.9 Padding . . . 29

4.1 Schichten eines kurzen Autoencoders . . . 34
4.2 Schichten eines tieferen Autoencoders . . . 34
4.3 Darstellung eines Convolutional Autoencoders . . . 37

5.1 TF-Logo . . . 38
5.2 Visualisierung des beschriebenen `tf.graph` . . . 42
5.3 Keras-Logo . . . 47

6.1 grobes Schema des Modells . . . 51
6.2 ein Auszug an MNIST-Bildern (Steppan, *File:MnistExamples.png — Wikipedia, The Free Encyclopedia*) . . . 52
6.3 Verrauschte Bilder neben den Orignalbildern . . . 55
6.4 Verrauschte Bilder, Rekonstruktionen und Orignalbilder . . . 59

Tabellenverzeichnis

Selbstständigkeitserklärung

Ich erkläre hiermit, dass ich diese Arbeit selbständig durchgeführt und keine anderen als die angegebene Quellen, Hilfsmittel und Hilfspersonen beigezogen habe. Alle Textstellen in der Arbeit, die wörtlich oder sinngemäss aus Quellen entnommen wurden, habe ich als solche gekennzeichnet.

Luis Wirth